

Classical Machine Learning

A Napkin-style companion for campus placements:
derivations, hand numerics, traps, and code that checks itself

Philospher

Built October 6, 2026

Preface

[STUB: preface]

Contents

I	Foundations	9
1	The Setup	11
1.1	The zoo of learning problems	11
1.2	Loss, risk and empirical risk	12
1.3	Train, validation, test	14
1.4	Over- and underfitting, the generalisation gap	14
1.5	The i.i.d. assumption and when it breaks	16
1.6	A few problems to think about	16
2	Maths at Your Fingertips	19
2.1	Linear algebra	19
2.2	Calculus and matrix calculus	20
2.3	Probability	21
2.4	Information theory in five lines	23
2.5	Maximum likelihood and MAP	23
2.6	Convexity	24
2.7	Lagrange multipliers and KKT	26
2.8	A few problems to think about	27
II	Supervised Learning	29
3	Linear Regression	33
3.1	Least squares and the normal equations	33
3.2	The projection and MLE views	34
3.3	Five points by hand	35
3.4	Gradient descent, traced	36
3.5	R^2 and adjusted R^2	37
3.6	Assumptions, diagnostics, multicollinearity	38
3.7	Polynomial features and robust losses	39
3.8	A few problems to think about	40
4	Bias, Variance and Regularisation	41
4.1	The decomposition	41
4.2	Ridge regression	42
4.3	Lasso and elastic net	44
4.4	The augmented error	46
4.5	Early stopping and double descent	46
4.6	A few problems to think about	47
5	Logistic Regression and GLMs	49
5.1	From odds to the sigmoid	49
5.2	Log-loss from maximum likelihood	49
5.3	Newton's method and IRLS	51
5.4	Softmax regression and multiclass	52
5.5	Why not MSE for classification?	53

5.6	Imbalance and thresholds	54
5.7	Generalised linear models	54
5.8	The perceptron and XOR	54
5.9	A few problems to think about	56
6	Generative Classifiers	57
6.1	Naive Bayes	57
6.2	Laplace smoothing	57
6.3	Limitations, and why it still works	58
6.4	LDA and QDA	59
6.5	A few problems to think about	61
7	k-Nearest Neighbours and Distances	63
7.1	The algorithm	63
7.2	Distances	64
7.3	Scaling, and choosing k	64
7.4	The curse of dimensionality	65
7.5	KD-trees and approximate search	66
7.6	A few problems to think about	66
8	Support Vector Machines	69
8.1	Margins	69
8.2	Hard and soft margin	69
8.3	Duality	70
8.4	Kernels	72
8.5	SVR and SMO	73
8.6	Convex surrogate losses	75
8.7	A few problems to think about	75
9	Decision Trees	77
9.1	Impurity	77
9.2	A split by hand	77
9.3	Growing a tree	79
9.4	Regression trees	80
9.5	Pruning	80
9.6	What trees are good for	80
9.7	A few problems to think about	82
10	Ensembles	83
10.1	The bootstrap and 63.2 percent	83
10.2	Bagging and random forests	84
10.3	Feature importance and its traps	85
10.4	AdaBoost	86
10.5	Gradient boosting	87
10.6	XGBoost	88
10.7	LightGBM and CatBoost	90
10.8	Stacking and the bias-variance story	91
10.9	A few problems to think about	91

III	Learning Without (Enough) Labels	93
11	Unsupervised Learning	95
11.1	<i>k</i> -means	95
11.2	Hierarchical clustering	97
11.3	DBSCAN	97
11.4	Gaussian mixtures and EM	98
11.5	PCA	99
11.6	ICA, t-SNE and UMAP	100
11.7	Anomaly detection	101
11.8	Association rules	101
11.9	A few problems to think about	102
12	Semi- and Self-Supervised Learning	103
12.1	Assumptions	103
12.2	Self-training	103
12.3	Co-training and label propagation	103
12.4	Consistency regularisation	104
12.5	When unlabelled data hurts	104
12.6	Contrastive learning in one page	105
12.7	A few problems to think about	106
IV	Doing It Properly	107
13	Evaluation and Model Selection	109
13.1	The confusion matrix	109
13.2	ROC and PR curves	110
13.3	Regression and probabilistic metrics	110
13.4	Cross-validation	111
13.5	Leakage	112
13.6	AIC, BIC and hyper-parameter search	112
13.7	Calibration	114
13.8	A few problems to think about	114
14	Statistics Interviewers Ask	117
14.1	Distributions	117
14.2	Estimators and confidence intervals	117
14.3	Hypothesis tests	118
14.4	Chi-square and ANOVA	119
14.5	Multiple testing	119
14.6	Correlation, causation and Simpson	120
14.7	A few problems to think about	120
15	Data Preparation and Feature Engineering	123
15.1	Scaling	123
15.2	Encoding categoricals	124
15.3	Missing values and outliers	124
15.4	Feature selection	125
15.5	Imbalance	125
15.6	Text features and TF-IDF	126

15.7	Time series in one page	127
15.8	Recommenders in one page	127
15.9	A few problems to think about	127
16	Optimisation for ML	129
16.1	Gradient descent on a quadratic	129
16.2	Momentum	129
16.3	Newton and quasi-Newton	130
16.4	Coordinate descent, SGD and duality	130
16.5	A few problems to think about	131
V	Interview Mode	133
17	Implement It	135
17.1	Linear models	135
17.2	Neighbours and Bayes	136
17.3	Trees and forests	136
17.4	Clustering and PCA	138
17.5	SVM	139
18	Rapid-fire	141
18.1	Rapid-fire answers	141
19	Problems	143
19.1	Problems	143
VI	Backmatter	145
A	Hints to problems	147
B	Solutions to problems	151
	Bibliography	157

I

Foundations

Part I: Contents

1	The Setup	11
1.1	The zoo of learning problems	11
1.2	Loss, risk and empirical risk	12
1.3	Train, validation, test	14
1.4	Over- and underfitting, the generalisation gap	14
1.5	The i.i.d. assumption and when it breaks	16
1.6	A few problems to think about	16
2	Maths at Your Fingertips	19
2.1	Linear algebra	19
2.2	Calculus and matrix calculus	20
2.3	Probability	21
2.4	Information theory in five lines	23
2.5	Maximum likelihood and MAP	23
2.6	Convexity	24
2.7	Lagrange multipliers and KKT	26
2.8	A few problems to think about	27

1 The Setup

Every interview question in this book is secretly about one sentence: *we choose a function using a finite sample, and we are paid for how it does on data we have not seen*. This chapter fixes the vocabulary for that sentence. It is short on algorithms and long on the distinctions that one-line MCQs love to blur: loss versus risk, validation versus test, training error versus generalisation gap.

§1.1 The zoo of learning problems

Prototypical example for this section: A bank has two million past loan applications, each with a “defaulted / repaid” label, and wants a score for tomorrow’s applicant.

The type of problem is decided by *what supervision you have at training time*, not by the algorithm you later pick.

Definition 1.1.1 (The five regimes). • **Supervised learning**: each training example is a pair $(\mathbf{x}_i, y_i)$, and we learn f with $f(\mathbf{x}) \approx y$. If y is real-valued it is **regression**; if y lies in a finite set it is **classification** (binary or multiclass; **multilabel** if several labels can be on at once).

- **Unsupervised learning**: only $\mathbf{x}_i$. We look for structure: clusters, low-dimensional coordinates, densities, anomalies.
- **Semi-supervised learning**: a few labelled pairs plus many unlabelled $\mathbf{x}$ ’s, and the goal is still a supervised predictor (**Chapter 12**).
- **Self-supervised learning**: labels are *manufactured from the input itself* (mask a word and predict it; crop two views of an image and ask whether they match). It is trained exactly like supervised learning, but nobody paid an annotator.
- **Reinforcement learning**: an agent acts, the environment returns states and a scalar **reward**, and the goal is a policy that maximises long-run reward. There is no correct action given per example, only delayed, noisy feedback on the actions you happened to take.

A useful everyday picture: supervised learning is a student with an answer key, unsupervised is a librarian sorting books with no catalogue, semi-supervised is the librarian who was shown how fifty books were shelved, self-supervised is a student who covers half a page and tries to recall it, and reinforcement learning is learning to cycle — nobody tells you the correct handlebar angle, you just fall less over time.

Task	Regime	Output
Price of a flat from area, floor, locality	supervised	regression
Is this transaction fraud?	supervised	binary classification
Which of 10 digits is in the image?	supervised	multiclass
Which genres does this film belong to?	supervised	multilabel
Segment customers by purchase behaviour	unsupervised	clustering
Flag unusual server logs (no labels)	unsupervised	anomaly score
Pre-train a language model by predicting masked words	self-supervised	classification over vocabulary
Learn to place bids in an ad auction from revenue	reinforcement	policy

Remark 1.1.2 — Trap. “Predicting the number of goals in a match (0, 1, 2, ...) is classification because the output is discrete.” Counts are ordered and arithmetic on them is meaningful, so the usual answer is *regression* (often Poisson regression, [Chapter 5](#)). Discreteness alone does not make a problem classification; *unordered* categories do. Similarly “customer segmentation” is clustering, not classification, unless someone hands you segment labels.

Remark 1.1.3 — Trap. Self-supervised $\neq$ unsupervised in an MCQ. Both use unlabelled data, but self-supervised learning optimises an ordinary supervised loss on a *pretext task*, and its output is usually a representation reused downstream. If an option says “BERT pre-training is unsupervised”, many keys accept it; the precise term is self-supervised.

§1.1.i Parametric versus non-parametric; discriminative versus generative

A **parametric** model has a fixed number of parameters regardless of n (linear and logistic regression). A **non-parametric** model grows with the data (k -nearest neighbours stores all of it; a fully grown tree can have up to n leaves). A **discriminative** model learns $p(y | \mathbf{x})$ or a decision rule directly (logistic regression, SVM); a **generative** model learns $p(\mathbf{x} | y)$ and $p(y)$ and applies Bayes’ rule (naive Bayes, LDA, [Chapter 6](#)).

The regime is set by the supervision available at training time; the output type (real, class, set of classes) then decides regression versus classification.

§1.2 Loss, risk and empirical risk

Prototypical example for this section: Predict $\hat{y} = 2x + 1$ on five points and average the squared errors.

Definition 1.2.1. A **loss** $\ell(\hat{y}, y) \geq 0$ scores *one* prediction. Common ones: squared loss $(\hat{y} - y)^2$, absolute loss $|\hat{y} - y|$, **0-1 loss** $\mathbb{I}[\hat{y} \neq y]$, log-loss $-\log \hat{p}_y$. Assume data come from an unknown distribution $\mathcal{D}$ over $(\mathbf{x}, y)$. The **risk** (true or expected risk, generalisation error) of f is

$$R(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [\ell(f(\mathbf{x}), y)],$$

and the **empirical risk** on a sample $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ is

$$\hat{R}_S(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(\mathbf{x}_i), y_i).$$

Empirical risk minimisation (ERM) picks $\hat{f} = \arg \min_{f \in \mathcal{H}} \hat{R}_S(f)$ over a **hypothesis class** $\mathcal{H}$.

Loss is per example, risk is an expectation over the population, empirical risk is an average over your sample. MCQs swap these words freely; you should not.

Example 1.2.2 (Empirical risk by hand)

Points $x = 0, 1, 2, 3, 4$ with $y = 1.2, 2.9, 5.1, 7.2, 8.7$ and the predictor $f(x) = 2x + 1$. Predictions are 1, 3, 5, 7, 9, so the residuals $y - f(x)$ are

$$+0.2 \quad -0.1 \quad +0.1 \quad +0.2 \quad -0.3 \quad .$$

Mean squared error = 0.038; mean absolute error = 0.180. For a 0-1 example, the rule “predict 1 iff $x > 2$ ” on $x = (0.5, 1.5, 2.2, 2.8, 3.5, 4.1)$, $y = (0, 0, 1, 0, 1, 1)$ makes 1 mistake (the point $x = 2.8$), so the empirical 0-1 risk is $1/6 = 0.1667$.

§1.2.i The best you can possibly do

Even a perfect learner cannot beat the noise in y given $\mathbf{x}$.

Definition 1.2.3. The **Bayes predictor** minimises the risk over *all* functions. Under squared loss it is $f^*(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}]$; under 0-1 loss it is $f^*(\mathbf{x}) = \arg \max_c \mathbb{P}(y = c \mid \mathbf{x})$. Its risk is the **Bayes risk** (**Bayes error** for 0-1 loss), the irreducible error.

Why the conditional mean is optimal for squared loss. For fixed $\mathbf{x}$ write $m = \mathbb{E}[y \mid \mathbf{x}]$. Then $\mathbb{E}[(c - y)^2 \mid \mathbf{x}] = (c - m)^2 + \text{Var}(y \mid \mathbf{x})$, minimised at $c = m$; the leftover $\text{Var}(y \mid \mathbf{x})$ is the Bayes risk at $\mathbf{x}$. $\square$

Example 1.2.4 (Bayes error of two Gaussians)

Class 0 is $\mathcal{N}(-1, 1)$, class 1 is $\mathcal{N}(+1, 1)$, equal priors. By symmetry the Bayes rule thresholds at 0, and its error is $\Phi(-1) = 0.1587$; a simulation with 200 000 draws gives 0.1599. No classifier, however deep, does better on this distribution. With priors 0.8/0.2 the optimal threshold moves to $\frac{1}{2} \ln(0.8/0.2) = 0.6931$ (towards the rare class) and the error drops to 0.1121; keeping the threshold at 0 would leave it at 0.1587.

Remark 1.2.5 — Trap. “Bayes error is the error of naive Bayes.” No: Bayes error is a property of the *distribution*, the floor for every classifier. Naive Bayes is one particular (usually sub-optimal) classifier.

§1.2.ii No free lunch, inductive bias

Averaged over *all* possible labelling functions, every learner has the same off-training-set error. Learning works only because real problems are not uniformly random: we assume

smoothness, linearity, sparsity, locality, invariance. These assumptions are the model's **inductive bias**, and choosing $\mathcal{H}$ is choosing one. “Which algorithm is best?” has no answer without a data distribution.

§1.3 Train, validation, test

Prototypical example for this section: Fifty models compared on the same validation set; the winner's score is too good to be true.

Definition 1.3.1. The **training set** fits parameters. The **validation set** (or cross-validation, [Chapter 13](#)) chooses among models and **hyper-parameters** (anything not learnt by the fitting procedure itself: degree, k , λ , tree depth). The **test set** is touched *once*, at the end, to report an unbiased estimate of risk.

Why three sets and not two? Every time you pick the best of several candidates by a score, the winning score is biased upwards: you selected partly on luck. The test set must not have taken part in any choice.

Example 1.3.2 (Selection optimism)

Take 50 “models” that are pure coin flips and score each on a balanced validation set of 100 points. Every model has true accuracy 0.5, yet over 2000 repetitions the *best* validation accuracy averages 0.612 (exact value from the distribution of the maximum of 50 binomials: 0.612). On fresh data the winner scores 0.500. A twelve-point “improvement” produced by selection alone.

Remark 1.3.3 — Typical splits: 60/20/20 or 80/10/10 for moderate data; with millions of rows a 1% test set is already tens of thousands of examples. With little data use k -fold cross-validation for selection and keep a held-out test set if you can afford it; nested CV otherwise ([Chapter 13](#)).

Remark 1.3.4 — **Trap.** Tuning on the test set (“I tried a few depths and kept the one with best test accuracy”) turns the test set into a validation set. The reported number is then optimistic, exactly as in the coin-flip example. Interviewers probe this with “how many times did you look at the test set?”

§1.4 Over- and underfitting, the generalisation gap

Prototypical example for this section: Fit polynomials of degree 0 to 12 to fifteen noisy samples of $\sin 2\pi x$.

Definition 1.4.1. The **generalisation gap** of $\hat{f}$ is $R(\hat{f}) - \hat{R}_S(\hat{f})$: test (or validation) error minus training error, or equivalently training accuracy minus test accuracy. A model **underfits** when even its training error is high (too little capacity or too much regularisation), and **overfits** when training error is low but the gap is large (it has memorised noise specific to S).

Example 1.4.2 (OA one-liner)

Training accuracy 0.98, test accuracy 0.81. The generalisation gap is 0.17 (17 percentage points): overfitting. Fixes, in order of what interviewers want to hear: more data, regularisation, simpler model / fewer features, early stopping, ensembling (bagging), data augmentation.

Example 1.4.3 (The degree sweep)

Fifteen training points, $y = \sin(2\pi x) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 0.3^2)$ (so the Bayes risk is 0.09). Median MSE over 200 repetitions:

degree	train MSE	test MSE	gap
0	0.558	0.611	0.053
1	0.238	0.323	0.085
3	0.065	0.124	0.059
9	0.037	0.777	0.740

Degree 0 and 1 underfit (both errors high, gap small); degree 3 is best on test; degree 9 has the lowest training error and a huge gap. The *mean* test MSE at degree 9 is 3768, because a few unlucky fits explode between and beyond the training points — variance in its purest form.

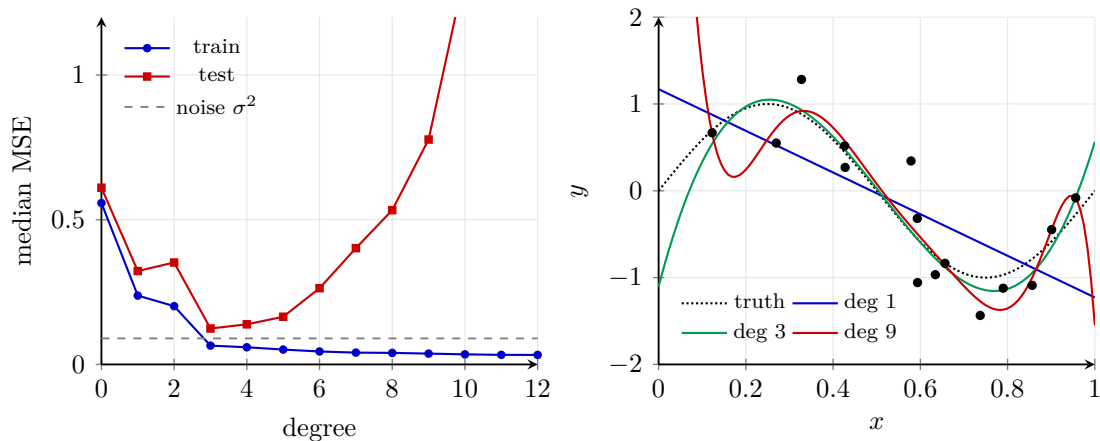


Figure 1.1: Left: training error falls monotonically with capacity; test error is U-shaped. Right: degree 1 underfits, degree 3 is about right, degree 9 threads every noisy point.

Remark 1.4.4 — Trap. “A small generalisation gap means a good model.” A constant predictor has a tiny gap and is useless. You need *both* low training error and a small gap. Conversely a modest gap with the best validation score is fine; deep nets routinely have training error near zero and are still the best model available.

Remark 1.4.5 — Trap. “Training error keeps decreasing, so we are still improving.” Training error is non-increasing in capacity for nested model classes almost by definition; it says nothing about generalisation. Watch validation error.

Question 1.4.6. If you add an irrelevant random feature to a least-squares model, can its training MSE go up? (Answer: no — the old solution is still available with weight 0 on the new feature.)

§1.5 The i.i.d. assumption and when it breaks

Prototypical example for this section: A model trained on 2022 transactions is scored on 2024 transactions after a payment app changes its UI.

Everything above assumes the training points and future points are drawn **independently and identically distributed (i.i.d.)** from the same $\mathcal{D}$. The two halves fail differently.

- **Not independent.** Several rows per patient, per user, per session; consecutive time steps. A random row-level split puts the same patient in train and test, and the model scores well by recognising the patient. Fix: split by group (GroupKFold) or by time.
- **Not identically distributed (distribution shift).** **Covariate shift:** $p(\mathbf{x})$ changes, $p(y | \mathbf{x})$ does not (a new city with different house sizes). **Label shift:** $p(y)$ changes, $p(\mathbf{x} | y)$ does not (fraud rate doubles during a festival sale). **Concept drift:** $p(y | \mathbf{x})$ itself changes (what counts as spam evolves).

Remark 1.5.1 — Trap. Shuffling a time series before a train/test split leaks the future into the past: the model sees Tuesday when predicting Monday. Use forward-chaining splits (Chapter 13).

Training error measures fit; the gap measures luck you mistook for signal; the i.i.d. assumption is what lets either number say anything about tomorrow.

§1.6 A few problems to think about

Problem 1A (Name the regime). Classify each as supervised (regression/classification), unsupervised, self-supervised or RL: (a) predicting tomorrow’s maximum temperature; (b) grouping news articles into topics with no labels; (c) predicting the next frame of a video from previous frames; (d) learning to play Ludo from win/loss outcomes; (e) assigning one of five star ratings to a review.

hint · solution

Problem 1B (Empirical risk). A predictor outputs $\hat{y} = (3, 5, 2, 8)$ for targets $y = (2, 5, 4, 7)$. Compute the mean squared error and the mean absolute error. Which of the two is more affected if the last target were 17 instead of 7?

hint · solution

Problem 1C (Gap arithmetic). Model A: train accuracy 0.99, validation 0.84. Model B: train 0.88, validation 0.86. Model C: train 0.71, validation 0.70. Which overfits, which underfits, which would you ship?

[hint](#) · [solution](#)

Problem 1D (Bayes rule with priors). Class 0 is $\mathcal{N}(0, 1)$ and class 1 is $\mathcal{N}(2, 1)$, with $\mathbb{P}(y = 1) = \pi$. Show that the Bayes classifier predicts 1 iff $x > 1 + \frac{1}{2} \ln \frac{1-\pi}{\pi}$. What happens as $\pi \rightarrow 0$?

[hint](#) · [solution](#)

Problem 1E (Leaky split). A hospital dataset has 10 X-rays each for 500 patients; the label is a diagnosis per patient. A random 80/20 split of the 5000 images gives 97% test accuracy, but the deployed model gets 78%. Explain, and propose a correct validation scheme.

[hint](#) · [solution](#)

Problem 1F (Selection bias, quantified). You evaluate m useless classifiers (each predicts with a fair coin) on a balanced validation set of size n . Using the normal approximation, estimate the expected best validation accuracy for $m = 50$, $n = 100$. (Use $\mathbb{E}[\max \text{ of } 50 \text{ standard normals}] \approx 2.25$.)

[hint](#) · [solution](#)

2 Maths at Your Fingertips

This chapter is the toolkit. Nothing here is decoration: every later derivation (normal equations, ridge, logistic gradients, the SVM dual, PCA, EM) is one of these facts applied once. Read it quickly, then come back when a later chapter says “by the identity in Chapter 2”.

§2.1 Linear algebra

Prototypical example for this section: The hat matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$, which projects $\mathbf{y}$ onto the column space of $\mathbf{X}$.

§2.1.i Vectors, norms, inner products

For $\mathbf{x} \in \mathbb{R}^d$, the ℓ_p **norm** is $\|\mathbf{x}\|_p = (\sum_i |x_i|^p)^{1/p}$. The three you need: $\|\mathbf{x}\|_1 = \sum |x_i|$ (lasso), $\|\mathbf{x}\|_2 = \sqrt{\sum x_i^2}$ (ridge, Euclidean distance), $\|\mathbf{x}\|_\infty = \max |x_i|$. For $\mathbf{x} = (3, -4, 12)$: $\|\mathbf{x}\|_1 = 19$, $\|\mathbf{x}\|_2 = 13$, $\|\mathbf{x}\|_\infty = 12$. Always $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1$.

The inner product $\mathbf{x}^\top \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$; **cosine similarity** is $\cos \theta$, which ignores length — why it is the default for comparing TF-IDF vectors and embeddings, where document length is a nuisance (Chapter 15).

§2.1.ii Rank, inverses, conditioning

The **rank** of $\mathbf{A}$ is the dimension of its column space. $\mathbf{A} \in \mathbb{R}^{d \times d}$ is invertible iff rank d iff $\det \mathbf{A} \neq 0$ iff no eigenvalue is 0. In regression, $\mathbf{X}^\top \mathbf{X}$ is singular exactly when a feature is a linear combination of others (perfect multicollinearity, Chapter 3); when some $n < d$ it is *always* singular. The **condition number** $\kappa(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$ measures how much relative error in $\mathbf{b}$ can be amplified in the solution of $\mathbf{A}\mathbf{x} = \mathbf{b}$; for $\text{diag}(1, 10^{-4})$ it is 10000. Large κ also makes gradient descent zig-zag (Chapter 16).

§2.1.iii Eigenvalues and the spectral theorem

$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ with $\mathbf{v} \neq \mathbf{0}$. A real symmetric matrix has real eigenvalues and an orthonormal eigenbasis: $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ (**spectral theorem**). Useful facts: $\text{Tr } \mathbf{A} = \sum \lambda_i$, $\det \mathbf{A} = \prod \lambda_i$, and eigenvalues of $\mathbf{A}^k$ are λ_i^k .

Example 2.1.1 (A rank-one matrix)

$\mathbf{S} = \begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix}$ has trace 5 and determinant 0.000, so its eigenvalues are 5.000 and 0.000 and its rank is 1 (it is $(2, 1)^\top (2, 1)$). Trace and determinant give a 2×2 's eigenvalues in ten seconds: they are the roots of $\lambda^2 - (\text{Tr})\lambda + \det$.

§2.1.iv Positive semi-definite matrices

Definition 2.1.2. Symmetric $\mathbf{A}$ is **positive semi-definite** (PSD, $\mathbf{A} \succeq 0$) if $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ for all $\mathbf{x}$, and **positive definite** (PD) if > 0 for all $\mathbf{x} \neq \mathbf{0}$. Equivalently: all eigenvalues ≥ 0 (resp. > 0).

Every covariance matrix and every Gram matrix $\mathbf{X}^\top \mathbf{X}$ is PSD, because $\mathbf{v}^\top \mathbf{X}^\top \mathbf{X} \mathbf{v} = \|\mathbf{X} \mathbf{v}\|^2 \geq 0$. Adding $\lambda \mathbf{I}$ with $\lambda > 0$ makes a PSD matrix PD — the whole reason ridge regression always has a unique solution. Kernel matrices must be PSD (Mercer, Chapter 8). $\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$ is PD (eigenvalues 1.000, 3.000); a diagonal-dominance glance tells you so.

§2.1.v Singular value decomposition

Every $\mathbf{A} \in \mathbb{R}^{n \times d}$ factors as $\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$ with orthonormal columns in $\mathbf{U}, \mathbf{V}$ and non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \dots$ on the diagonal of $\mathbf{\Sigma}$. The σ_i^2 are the eigenvalues of $\mathbf{A}^\top \mathbf{A}$ (and of $\mathbf{A} \mathbf{A}^\top$), $\mathbf{V}$ holds the eigenvectors of $\mathbf{A}^\top \mathbf{A}$.

Theorem 2.1.3 (Eckart–Young)

The best rank- k approximation of $\mathbf{A}$ in Frobenius (or spectral) norm is the truncated SVD $\mathbf{A}_k = \sum_{i \leq k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$, with error $\|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sum_{i > k} \sigma_i^2}$.

Example 2.1.4 (SVD by eigenvalues)

$\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \\ 1 & 1 \end{pmatrix}$ gives $\mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 11 & 7 \\ 7 & 11 \end{pmatrix}$ with eigenvalues 11 ± 7 , i.e. 18.000 and 4.000, so $\sigma_1 = \sqrt{18} = 4.2426$ and $\sigma_2 = 2.0000$. The best rank-one approximation misses by exactly $\sigma_2 = 2.0000$ in Frobenius norm. PCA (Chapter 11) is this theorem applied to a centred data matrix.

§2.1.vi Projections

The orthogonal projection onto the column space of a full-column-rank $\mathbf{X}$ is $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$. It is symmetric and **idempotent** ($\mathbf{H}^2 = \mathbf{H}$: projecting twice changes nothing), its eigenvalues are 0 or 1, and $\text{Tr } \mathbf{H} = \text{rank } \mathbf{X}$.

Example 2.1.5 (Projecting onto a line)

With $\mathbf{X} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}$ (intercept and $x = 0, 1, 2$) and $\mathbf{y} = (1, 2, 2)$, the projection is $\hat{\mathbf{y}} = \mathbf{H} \mathbf{y} = (1.167, 1.667, 2.167)$ — the fitted values of the least-squares line — and $\text{Tr } \mathbf{H} = 2.000$, the number of parameters. The residual $\mathbf{y} - \hat{\mathbf{y}}$ is orthogonal to every column of $\mathbf{X}$.

Symmetric $\Rightarrow$ orthogonal eigenvectors; Gram and covariance matrices are PSD; adding $\lambda \mathbf{I}$ makes them invertible; least squares is a projection.

§2.2 Calculus and matrix calculus

Prototypical example for this section: $\nabla_w \|\mathbf{X} \mathbf{w} - \mathbf{y}\|^2 = 2 \mathbf{X}^\top (\mathbf{X} \mathbf{w} - \mathbf{y})$, the gradient behind every linear model.

§2.2.i Gradient, Jacobian, Hessian

For $f : \mathbb{R}^d \rightarrow \mathbb{R}$, the **gradient** $\nabla f \in \mathbb{R}^d$ collects partial derivatives and points in the direction of steepest ascent; $-\nabla f$ is the descent direction. For $\mathbf{f} : \mathbb{R}^d \rightarrow \mathbb{R}^m$ the **Jacobian** $\mathbf{J} \in \mathbb{R}^{m \times d}$ has $J_{ij} = \partial f_i / \partial x_j$; the chain rule is a product of Jacobians, which is all backpropagation is (see *Deep Learning*). The **Hessian** $\nabla^2 f$ is the $d \times d$ matrix of second derivatives; it is symmetric for smooth f .

At a stationary point ($\nabla f = \mathbf{0}$): Hessian PD $\Rightarrow$ local minimum, ND $\Rightarrow$ local maximum, eigenvalues of both signs $\Rightarrow$ **saddle point**. For $f = x^2 + 3xy + y^2$ the Hessian $\begin{pmatrix} 2 & 3 \\ 3 & 2 \end{pmatrix}$ has eigenvalues 5 and -1 , so the origin is a saddle, although the function is a “bowl” along both axes.

§2.2.ii The identities you will use

With $\mathbf{A}$ a constant matrix and $\mathbf{b}$ a constant vector (“numerator layout” gradients as column vectors):

$$\begin{aligned}\nabla_{\mathbf{x}}(\mathbf{b}^\top \mathbf{x}) &= \mathbf{b}, \\ \nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) &= (\mathbf{A} + \mathbf{A}^\top) \mathbf{x} \quad (= 2\mathbf{A} \mathbf{x} \text{ if } \mathbf{A} \text{ is symmetric}), \\ \nabla_{\mathbf{x}} \|\mathbf{A} \mathbf{x} - \mathbf{b}\|^2 &= 2\mathbf{A}^\top (\mathbf{A} \mathbf{x} - \mathbf{b}), \quad \nabla_{\mathbf{x}}^2 \|\mathbf{A} \mathbf{x} - \mathbf{b}\|^2 = 2\mathbf{A}^\top \mathbf{A}, \\ \nabla_{\mathbf{A}} \text{Tr}(\mathbf{A} \mathbf{B}) &= \mathbf{B}^\top, \quad \nabla_{\mathbf{A}} \log \det \mathbf{A} = \mathbf{A}^{-\top}.\end{aligned}$$

All four non-trivial ones are checked numerically against finite differences in `code/ml/c02_maths.py`.

Remark 2.2.1 — Trap. Writing $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = 2\mathbf{A} \mathbf{x}$ for a non-symmetric $\mathbf{A}$ is a common slip; the correct answer is $(\mathbf{A} + \mathbf{A}^\top) \mathbf{x}$. When deriving by hand, check dimensions at every step: a gradient with respect to a d -vector is a d -vector, so terms like $\mathbf{x}^\top \mathbf{A}$ must be transposed.

§2.3 Probability

Prototypical example for this section: A test for a disease with 1% prevalence is 95% sensitive and 90% specific; a patient tests positive.

§2.3.i Bayes’ rule

$$\mathbb{P}(A | B) = \frac{\mathbb{P}(B | A)\mathbb{P}(A)}{\mathbb{P}(B)}, \quad \mathbb{P}(B) = \sum_k \mathbb{P}(B | A_k)\mathbb{P}(A_k).$$

In ML language: posterior $\propto$ likelihood $\times$ prior.

Example 2.3.1 (Base rates)

Prevalence 0.01, sensitivity $\mathbb{P}(+ | D) = 0.95$, specificity $\mathbb{P}(- | \neg D) = 0.90$. Then $\mathbb{P}(+) = 0.95 \cdot 0.01 + 0.10 \cdot 0.99 = 0.1085$ and

$$\mathbb{P}(D | +) = \frac{0.0095}{0.1085} = 0.0876.$$

Under 9%: most positives are false positives because healthy people vastly outnumber sick ones. This is the precision problem of every rare-event classifier

(Chapter 13).

§2.3.ii Expectation, variance, covariance

Expectation is linear always: $\mathbb{E}[aX + bY] = a\mathbb{E}X + b\mathbb{E}Y$, independent or not. $\text{Var } X = \mathbb{E}[X^2] - (\mathbb{E}X)^2$, $\text{Var}(aX + b) = a^2 \text{Var } X$, and

$$\text{Var}(X + Y) = \text{Var } X + \text{Var } Y + 2 \text{Cov}(X, Y), \quad \text{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}X \mathbb{E}Y.$$

For a vector, the **covariance matrix** $\Sigma = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top]$ is PSD and $\text{Var}(\mathbf{w}^\top \mathbf{x}) = \mathbf{w}^\top \Sigma \mathbf{w}$. A fair die has mean 3.5 and variance $35/12 = 2.9167$.

§2.3.iii Correlation and r^2

Definition 2.3.2. The (Pearson) **correlation** is $r = \text{Corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1]$.

Covariance has units (rupees $\times$ kilograms) and changes when you rescale; correlation is unit-free and invariant to positive affine maps of either variable. It measures *linear* association only.

Example 2.3.3 (Covariance and correlation of five points)

$x = (2, 4, 6, 8, 10)$, $y = (3, 7, 5, 11, 14)$. Sample covariance (divide by $n - 1 = 4$) is 13.00; $s_x = 3.1623$, $s_y = 4.4721$, so $r = 13 / (3.1623 \cdot 4.4721) = 0.9192$ and $r^2 = 0.8450$. Rescaling x by 100 makes the covariance 1300 but leaves r at 0.9192.

Theorem 2.3.4 (r^2 is the R^2 of simple regression)

For least-squares regression of y on a single x with an intercept, the coefficient of determination equals the squared correlation: $R^2 = r^2$. (Proof in Chapter 3.)

So the OA question “the correlation between x and y is -0.8 ; what fraction of the variance of y is explained by a linear fit on x ?” has answer $(-0.8)^2 = 0.64$, i.e. 64%. The sign of r disappears.

Remark 2.3.5 — Trap. Three classics. (i) $r = 0$ does not mean independent: for x uniform on $[-1, 1]$ and $y = x^2$, y is a deterministic function of x yet $r = 0.0000$. (ii) $r^2 = 0.64$ does not mean “64% of points are predicted correctly”; it is a fraction of *variance*. (iii) $R^2 = r^2$ holds for *simple* linear regression with intercept; with several features R^2 is the squared correlation between y and $\hat{y}$, not with any one feature.

§2.3.iv Law of large numbers and the central limit theorem

Theorem 2.3.6 (LLN and CLT)

If $X_1, X_2, \dots$ are i.i.d. with mean μ and variance $\sigma^2 < \infty$, then the sample mean $\bar{X}_n \rightarrow \mu$ (LLN), and $\sqrt{n}(\bar{X}_n - \mu)/\sigma \rightarrow \mathcal{N}(0, 1)$ in distribution (CLT). In particular $\text{sd}(\bar{X}_n) = \sigma/\sqrt{n}$, the **standard error**.

The LLN is why empirical risk approximates risk; the CLT is why confidence intervals and z -tests (Chapter 14) work even for non-normal data. Figure 2.1 shows both for an exponential distribution, which is heavily skewed (skewness 1.98); means of 30 draws already have skewness 0.36.

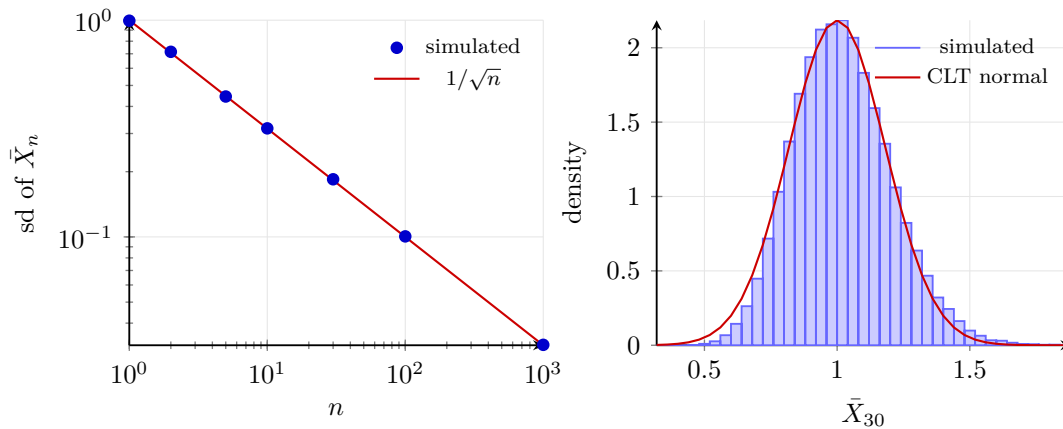


Figure 2.1: Means of Exponential(1) draws: the spread shrinks like $1/\sqrt{n}$ (left) and the shape becomes normal (right, $n = 30$, 50 000 repetitions).

§2.4 Information theory in five lines

Prototypical example for this section: Cross-entropy between the true label distribution and a classifier's predicted probabilities.

For distributions p, q over the same outcomes:

$$\text{entropy } H(p) = -\sum_i p_i \log p_i,$$

$$\text{cross-entropy } H(p, q) = -\sum_i p_i \log q_i = H(p) + \text{KL}(p \parallel q),$$

$$\text{KL divergence } \text{KL}(p \parallel q) = \sum_i p_i \log \frac{p_i}{q_i} \geq 0.$$

With $p = (0.5, 0.25, 0.25)$, $q = (0.4, 0.4, 0.2)$ in bits: $H(p) = 1.5000$, $H(p, q) = 1.5719$, $\text{KL}(p \parallel q) = 0.0719$ but $\text{KL}(q \parallel p) = 0.0781$ — KL is not symmetric and not a metric. Because $H(p)$ does not depend on the model q , minimising cross-entropy *is* minimising KL to the data distribution. Entropy reappears as a tree impurity (Chapter 9).

Remark 2.4.1 (Numerical stability) — $\log \sum_i e^{z_i}$ overflows for $z = (1000, 1001, 1002)$: numpy returns *inf*. Subtract the max first: $\log \sum e^{z_i} = m + \log \sum e^{z_i - m} = 1002.4076$. Every softmax and log-loss implementation does this.

§2.5 Maximum likelihood and MAP

Prototypical example for this section: A coin lands heads 7 times in 10; estimate p with and without a prior.

Definition 2.5.1. Given i.i.d. data and a model $p(\mathbf{x} \mid \boldsymbol{\theta})$, the **likelihood** is $L(\boldsymbol{\theta}) = \prod_i p(\mathbf{x}_i \mid \boldsymbol{\theta})$ and the **maximum likelihood estimate** (MLE) is $\arg \max_{\boldsymbol{\theta}} \sum_i \log p(\mathbf{x}_i \mid \boldsymbol{\theta})$. With a prior $p(\boldsymbol{\theta})$ the **maximum a posteriori** (MAP) estimate is $\arg \max_{\boldsymbol{\theta}} [\log p(\boldsymbol{\theta}) + \sum_i \log p(\mathbf{x}_i \mid \boldsymbol{\theta})]$.

We take logs because products of many small numbers underflow, sums are easier to differentiate, and log is monotone so the maximiser is unchanged.

Example 2.5.2 (Bernoulli MLE and MAP)

$\ell(p) = 7 \log p + 3 \log(1 - p)$; $\ell'(p) = 7/p - 3/(1 - p) = 0$ gives $\hat{p}_{\text{MLE}} = 7/10 = 0.70$. With a $\text{Beta}(3, 3)$ prior (two pseudo-heads and two pseudo-tails) the posterior is $\text{Beta}(10, 6)$, whose mode is $\hat{p}_{\text{MAP}} = (7 + 2)/(10 + 4) = 0.6429$ and whose mean is $10/16 = 0.6250$. The prior pulls the estimate towards $1/2$; with more data the pull fades. Laplace smoothing in naive Bayes (Chapter 6) is exactly this.

§2.5.i Three losses that are really likelihoods

1. **Least squares = Gaussian MLE.** If $y_i = \mathbf{w}^\top \mathbf{x}_i + \varepsilon_i$ with $\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$, then $-\log L(\mathbf{w}) = \frac{1}{2\sigma^2} \sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \frac{n}{2} \log(2\pi\sigma^2)$, so maximising likelihood minimises the sum of squared errors.
2. **Ridge = MAP with a Gaussian prior.** Add $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 \mathbf{I})$: $-\log p(\mathbf{w} | \mathcal{D}) = \frac{1}{2\sigma^2} \sum (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \frac{1}{2\tau^2} \|\mathbf{w}\|^2 + c$, i.e. ridge with $\lambda = \sigma^2/\tau^2$. A Laplace prior gives the lasso. A tight prior (small τ) means large λ .
3. **Cross-entropy = categorical MLE.** If the model outputs probabilities $\hat{p}_{ik}$ and y_i is one-hot, $-\log L = -\sum_i \sum_k y_{ik} \log \hat{p}_{ik}$, the log-loss of logistic and softmax regression (Chapter 5).

Remark 2.5.3 — Trap. The MLE of a Gaussian variance divides by n , not $n - 1$, and is biased. For the sample $(4, 7, 5, 9, 5)$: mean 6.0, MLE variance 3.20, unbiased variance 4.00. `numpy.var` defaults to `ddof=0` (the MLE); `pandas` defaults to `ddof=1`. MLEs are typically consistent and asymptotically efficient, not necessarily unbiased.

§2.6 Convexity

Prototypical example for this section: Why $|x|$ is convex, $\sin x$ is not, and the 0-1 loss is not.

Definition 2.6.1. A set C is **convex** if the segment between any two of its points stays in C . A function f on a convex set is **convex** if for all $\mathbf{x}, \mathbf{y}$ and $t \in [0, 1]$

$$f(t\mathbf{x} + (1 - t)\mathbf{y}) \leq tf(\mathbf{x}) + (1 - t)f(\mathbf{y}) :$$

the chord lies on or above the graph. **Strictly convex:** strict inequality for $\mathbf{x} \neq \mathbf{y}$, $t \in (0, 1)$.

Theorem 2.6.2 (Tests)

For differentiable f : convex $\iff f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x})$ (every tangent is a global under-estimator). For twice-differentiable f : convex $\iff \nabla^2 f \succeq 0$ everywhere; $\nabla^2 f \succ 0$ everywhere implies strictly convex.

Theorem 2.6.3 (Why we care)

For a convex function every local minimum is a global minimum, and the set of minimisers is convex. A strictly convex function has at most one minimiser.

§2.6.i Building convex functions

Convexity is preserved by: non-negative weighted sums; composition with an affine map ($f(\mathbf{Ax} + \mathbf{b})$); and the *pointwise maximum* (or supremum) of convex functions. It is **not** preserved by the pointwise minimum, by products, or by differences.

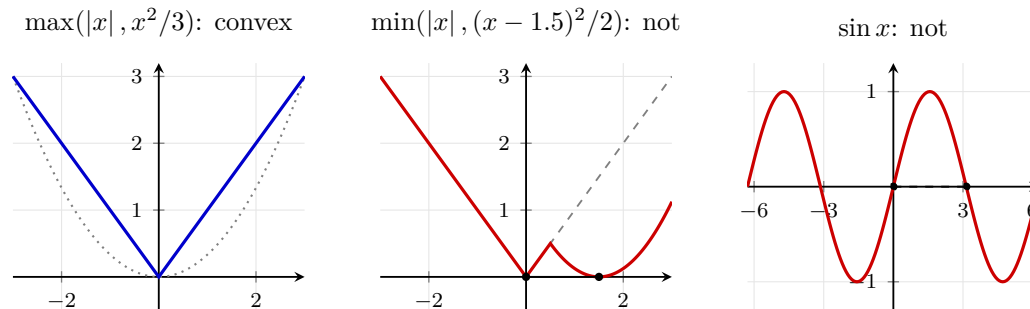


Figure 2.2: The maximum of convex functions is convex; the minimum generally is not (two local minima, at 0 and 1.5). For $\sin x$ the chord from 0 to π lies *below* the graph.

Example 2.6.4 (OA: which are convex?)

Each of these has appeared as a “which statements are true” option.

- $|x|$: convex (a max of x and $-x$, both linear), but not differentiable at 0 and not strictly convex.
- $\sin x$ on $\mathbb{R}$: not convex. Chord test on $[0, \pi]$: the midpoint of the chord is 0, but $\sin(\pi/2) = 1 > 0$. (It is convex on $[\pi, 2\pi]$ where $\sin'' = -\sin \geq 0$, which is why a question may restrict the domain.)
- $e^x, x^2, -\log x, x \log x, \|\mathbf{x}\|, \max_i x_i, \log \sum e^{x_i}$: convex. $\log x, \sqrt{x}$: concave.
- $\max(f, g)$ for convex f, g : convex. $\min(f, g)$: not in general (Figure 2.2).
- Jensen’s inequality $f(\mathbb{E}X) \leq \mathbb{E}f(X)$ for convex f : with X uniform on $\{1, 2, 6\}$ and $f(x) = x^2$, $(\mathbb{E}X)^2 = 9 \leq \mathbb{E}X^2 = 13.67$; this is just $\text{Var } X \geq 0$.

§2.6.ii Why the 0-1 loss is not convex, and what we do about it

Write a binary classifier as $\text{sign}(f(\mathbf{x}))$ with labels $y \in \{-1, +1\}$ and **margin** $m = yf(\mathbf{x})$. The 0-1 loss is $\mathbb{1}[m \leq 0]$, a step. Chord test: at $m = -0.5$ the loss is 1, at $m = 0.5$ it is 0, the chord’s midpoint is 0.5, but the loss at $m = 0$ is 1. It is also flat almost everywhere, so its gradient is zero almost everywhere and gradient methods get no signal; minimising it exactly over linear classifiers is NP-hard in general.

So we minimise a convex **surrogate loss** that upper-bounds the step: the **hinge** $\max(0, 1-m)$ (SVM), the **logistic** loss $\log_2(1+e^{-m})$ (logistic regression), the **exponential**

e^{-m} (AdaBoost), or the squared loss $(1 - m)^2$ (which also penalises being “too right”, a reason it is a poor classification loss).

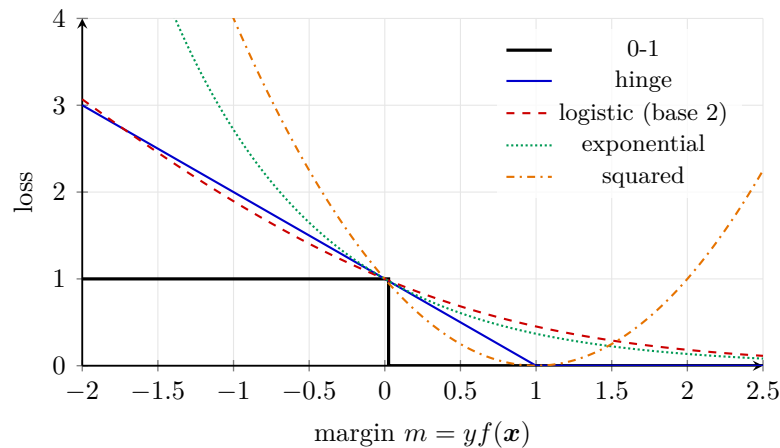


Figure 2.3: The 0-1 loss and its convex surrogates as functions of the margin. All surrogates upper-bound the step; only the squared loss rises again for confident correct predictions.

Remark 2.6.5 — Trap. “Convex loss $\Rightarrow$ convex training objective.” Only if the model is linear in its parameters. Log-loss of a neural network is non-convex in the weights even though log-loss is convex in the logits: composition of a convex function with a non-affine map breaks convexity.

Max of convex is convex, min is not; the 0-1 loss is a non-convex step with zero gradient, so we optimise convex surrogates that bound it from above.

§2.7 Lagrange multipliers and KKT

Prototypical example for this section: Maximise xy subject to $x + y = 10$; then minimise $(x - 2)^2$ subject to $x \leq 1$.

§2.7.i Equality constraints

To optimise $f(\mathbf{x})$ subject to $g(\mathbf{x}) = 0$, form the **Lagrangian** $\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x})$ and set $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{0}$, $g = 0$. Geometrically, at the optimum ∇f is parallel to ∇g : you cannot improve f while staying on the constraint surface.

Example 2.7.1

Maximise xy with $x + y = 10$: $\nabla(xy) = (y, x) = \lambda(1, 1)$ forces $x = y = 5$ (checked with SciPy’s SLSQP). A more ML-flavoured one: the maximum-entropy distribution on $\{1, 2, 3\}$ with mean 2.5 has the exponential-family form $p_i \propto e^{tx_i}$; solving for t gives $p = (0.116, 0.268, 0.616)$, matching a numerical constrained optimiser. The same calculation with an unconstrained mean gives the uniform distribution, and with a variance constraint on $\mathbb{R}$ gives the Gaussian.

§2.7.ii Inequality constraints: KKT

For $\min f(\mathbf{x})$ s.t. $g_j(\mathbf{x}) \leq 0$, $h_k(\mathbf{x}) = 0$, with Lagrangian $f + \sum_j \mu_j g_j + \sum_k \lambda_k h_k$, the **Karush–Kuhn–Tucker** (KKT) conditions are

1. stationarity: $\nabla f + \sum \mu_j \nabla g_j + \sum \lambda_k \nabla h_k = \mathbf{0}$;
2. primal feasibility: $g_j \leq 0$, $h_k = 0$;
3. dual feasibility: $\mu_j \geq 0$;
4. **complementary slackness**: $\mu_j g_j(\mathbf{x}) = 0$ for each j .

For convex problems satisfying a mild regularity condition (Slater: a strictly feasible point exists), KKT is necessary and sufficient.

Example 2.7.2 (An active constraint)

$\min(x-2)^2$ s.t. $x \leq 1$. Unconstrained the answer would be 2, which is infeasible, so the constraint is active: $x^* = 1$. Stationarity $2(x-2) + \mu = 0$ gives $\mu = 2 > 0$. Complementary slackness reads: either the constraint is tight ($x = 1$) or its multiplier is zero. In SVMs (Chapter 8) this is exactly why only the *support vectors* have non-zero α_i .

Multipliers price constraints; complementary slackness says a constraint either binds or costs nothing.

§2.8 A few problems to think about

Problem 2A (r^2 from r). The correlation between hours studied and marks is -0.3 in one batch and 0.6 in another. In each, what fraction of the variance of marks is explained by a straight-line fit on hours? Which batch has the “stronger” linear relationship?

hint · solution

Problem 2B (Which are convex?). Decide convexity on the stated domain: (a) $f(x) = \max(x^2, 2x+1)$; (b) $f(x) = \min(x^2, 1)$; (c) $f(\mathbf{w}) = \sum_i \log(1 + e^{-y_i \mathbf{w}^\top \mathbf{x}_i}) + \lambda \|\mathbf{w}\|_1$; (d) $f(x) = x^3$ on $\mathbb{R}$; (e) $f(x) = x^3$ on $[0, \infty)$; (f) $f(x, y) = xy$.

hint · solution

Problem 2C (Gradient of a quadratic form). Let $f(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \mathbf{A} \mathbf{w} - \mathbf{b}^\top \mathbf{w} + c$ with $\mathbf{A}$ symmetric PD. Find the minimiser and the minimum value. What goes wrong if $\mathbf{A}$ has a zero eigenvalue?

hint · solution

Problem 2D (Bayes with two tests). In the disease example (prevalence 1%, sensitivity 95%, specificity 90%) the patient takes a second, conditionally independent test with the same characteristics and is positive again. What is $\mathbb{P}(D \mid ++)$?

hint · solution

Problem 2E (MAP shrinkage). A click-through rate is estimated from 2 clicks in 3 impressions. Compare the MLE with the MAP under a Beta(2, 20) prior. Which would you show on a dashboard ranking ads?

hint · solution

Problem 2F (KKT in one dimension). Solve $\min_x (x - a)^2$ subject to $0 \leq x \leq 1$ for any real a using KKT, and identify which multiplier is positive in each case.

[hint](#) · [solution](#)



II

Supervised Learning

Part II: Contents

3	Linear Regression	33
3.1	Least squares and the normal equations	33
3.2	The projection and MLE views	34
3.3	Five points by hand	35
3.4	Gradient descent, traced	36
3.5	R^2 and adjusted R^2	37
3.6	Assumptions, diagnostics, multicollinearity	38
3.7	Polynomial features and robust losses	39
3.8	A few problems to think about	40
4	Bias, Variance and Regularisation	41
4.1	The decomposition	41
4.2	Ridge regression	42
4.3	Lasso and elastic net	44
4.4	The augmented error	46
4.5	Early stopping and double descent	46
4.6	A few problems to think about	47
5	Logistic Regression and GLMs	49
5.1	From odds to the sigmoid	49
5.2	Log-loss from maximum likelihood	49
5.3	Newton's method and IRLS	51
5.4	Softmax regression and multiclass	52
5.5	Why not MSE for classification?	53
5.6	Imbalance and thresholds	54
5.7	Generalised linear models	54
5.8	The perceptron and XOR	54
5.9	A few problems to think about	56
6	Generative Classifiers	57
6.1	Naive Bayes	57
6.2	Laplace smoothing	57
6.3	Limitations, and why it still works	58
6.4	LDA and QDA	59
6.5	A few problems to think about	61
7	k-Nearest Neighbours and Distances	63
7.1	The algorithm	63
7.2	Distances	64
7.3	Scaling, and choosing k	64
7.4	The curse of dimensionality	65
7.5	KD-trees and approximate search	66
7.6	A few problems to think about	66
8	Support Vector Machines	69
8.1	Margins	69
8.2	Hard and soft margin	69
8.3	Duality	70
8.4	Kernels	72
8.5	SVR and SMO	73
8.6	Convex surrogate losses	75
8.7	A few problems to think about	75
9	Decision Trees	77
9.1	Impurity	77
9.2	A split by hand	77

9.3	Growing a tree	79
9.4	Regression trees	80
9.5	Pruning	80
9.6	What trees are good for	80
9.7	A few problems to think about	82
10	Ensembles	83
10.1	The bootstrap and 63.2 percent	83
10.2	Bagging and random forests	84
10.3	Feature importance and its traps	85
10.4	AdaBoost	86
10.5	Gradient boosting	87
10.6	XGBoost	88
10.7	LightGBM and CatBoost	90
10.8	Stacking and the bias–variance story	91
10.9	A few problems to think about	91

3 Linear Regression

Linear regression is the one model every interviewer assumes you can derive in three ways (calculus, geometry, probability), fit by hand in an OA, and criticise in a design round. It is also the template for everything else: a hypothesis class, a loss, a closed form or a gradient, and a list of assumptions whose violation you should be able to diagnose.

§3.1 Least squares and the normal equations

Prototypical example for this section: Predict flat price from area: $\hat{y} = w_0 + w_1 \cdot \text{area}$.

§3.1.i The model

Stack n examples as rows of the **design matrix** $\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$, whose first column is all ones (the intercept), and the targets as $\mathbf{y} \in \mathbb{R}^n$. The model is $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ and **ordinary least squares** (OLS) minimises the **residual sum of squares**

$$\text{RSS}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 = \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2.$$

“Linear” means linear *in the parameters* $\mathbf{w}$: $y = w_0 + w_1x + w_2x^2 + w_3 \log x$ is still linear regression.

§3.1.ii The closed form

Using $\nabla_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$ from **Chapter 2** and setting it to zero gives the **normal equations**

$$\mathbf{X}^\top \mathbf{X} \hat{\mathbf{w}} = \mathbf{X}^\top \mathbf{y} \quad \implies \quad \hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

whenever $\mathbf{X}^\top \mathbf{X}$ is invertible (i.e. $\mathbf{X}$ has full column rank). The Hessian $2\mathbf{X}^\top \mathbf{X}$ is PSD, so RSS is convex and any stationary point is a global minimum; if $\mathbf{X}$ has full column rank the Hessian is PD and the minimiser is unique.

Remark 3.1.1 (How it is actually computed) — Nobody inverts $\mathbf{X}^\top \mathbf{X}$. Forming it squares the condition number: for an intercept column plus a feature ranging over $[1000, 1001]$, $\kappa(\mathbf{X}) = 3.4 \times 10^6$ but $\kappa(\mathbf{X}^\top \mathbf{X}) = 1.2 \times 10^{13}$, so you lose about twice as many digits. Libraries use a QR decomposition ($\mathbf{X} = \mathbf{Q}\mathbf{R}$, then solve $\mathbf{R}\mathbf{w} = \mathbf{Q}^\top \mathbf{y}$) or the SVD (`numpy.linalg.lstsq`, `scikit-learn`). On a 200-row, 3-feature example the normal equations, QR, `lstsq` and `LinearRegression` all give $\hat{\mathbf{w}} = (2.982, 1.955, -1.048, 0.527)$ to machine precision (`code/ml/c03_linreg.py`). Cost: $O(nd^2 + d^3)$; for very large d or streaming data, use gradient descent instead.

Remark 3.1.2 — **Trap.** “The normal equations fail when features are correlated.” They fail when features are *perfectly* collinear (or $n < d + 1$), because $\mathbf{X}^\top \mathbf{X}$ is singular. Strong but imperfect correlation still gives a unique solution, only an unstable one (§3.6). Centring and scaling features do not change the fitted values

of OLS, only the coefficients' scale and the numerical conditioning.

§3.2 The projection and MLE views

Prototypical example for this section: $\hat{\mathbf{y}}$ is the shadow of $\mathbf{y}$ on the plane spanned by the feature columns.

§3.2.i Geometry

$\mathbf{X}\mathbf{w}$ ranges over the column space $\mathcal{C}(\mathbf{X})$, a $(d + 1)$ -dimensional subspace of $\mathbb{R}^n$. The closest point to $\mathbf{y}$ in it is the orthogonal projection $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ with $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ (the **hat matrix**: it puts the hat on $\mathbf{y}$). The normal equations say $\mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}$: the residual vector is orthogonal to every column of $\mathbf{X}$. Two corollaries an interviewer can ask:

- With an intercept, residuals sum to zero (orthogonal to the ones column) and the fitted line passes through $(\bar{x}, \bar{y})$.
- Residuals are uncorrelated with every feature and with $\hat{\mathbf{y}}$ in the sample. A residual plot that shows a trend against $\hat{\mathbf{y}}$ therefore signals *non-linearity*, not a coding bug.

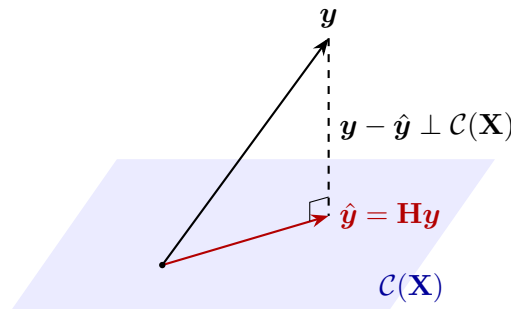


Figure 3.1: Least squares projects $\mathbf{y}$ onto the column space of $\mathbf{X}$; the residual is perpendicular to it.

§3.2.ii Probability

Assume $y_i = \mathbf{w}^\top \mathbf{x}_i + \varepsilon_i$ with $\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$. The log-likelihood is $-\frac{1}{2\sigma^2} \text{RSS}(\mathbf{w}) - \frac{n}{2} \log(2\pi\sigma^2)$, so the MLE of $\mathbf{w}$ is OLS (as derived in **Chapter 2**), and the MLE of σ^2 is RSS/n (biased; the unbiased estimate divides by $n - d - 1$). Squared loss is therefore the natural loss when noise is Gaussian; with Laplace noise the MLE is least *absolute* deviations, which estimates a conditional median.

Theorem 3.2.1 (Gauss–Markov)

If $\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$ and $\text{Cov}(\boldsymbol{\varepsilon}) = \sigma^2 \mathbf{I}$ (zero mean, equal variance, uncorrelated — normality *not* required), then OLS is the **BLUE**: the best (minimum-variance) linear unbiased estimator of $\mathbf{w}$. Its covariance is $\sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$.

“Best among *linear unbiased*” is the catch: ridge is biased and can have lower mean squared error (Chapter 4).

OLS = calculus (normal equations) = geometry (projection) = statistics (Gaussian MLE).

§3.3 Five points by hand

Prototypical example for this section: The OA question: “fit $y = b_0 + b_1x$ to these five points”.

For one feature the normal equations reduce to

$$\hat{b}_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} = r \frac{s_y}{s_x}, \quad \hat{b}_0 = \bar{y} - \hat{b}_1 \bar{x}.$$

The middle form is what you use with raw sums when the means are messy; the centred form is faster when $\bar{x}$ is an integer.

Example 3.3.1 (The OA fit)

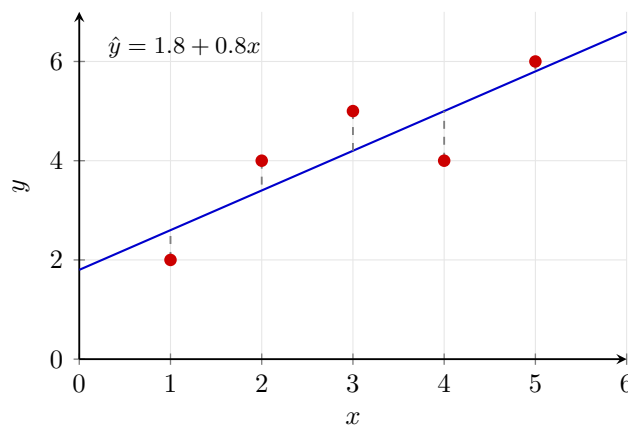
Data: $x = (1, 2, 3, 4, 5)$, $y = (2, 4, 5, 4, 6)$. Raw sums: $\sum x = 15$, $\sum y = 21$, $\sum xy = 71$, $\sum x^2 = 55$.

$$\hat{b}_1 = \frac{5 \cdot 71 - 15 \cdot 21}{5 \cdot 55 - 15^2} = \frac{40}{50} = 0.80, \quad \hat{b}_0 = 4.2 - 0.8 \cdot 3 = 1.80.$$

Centred check: $\sum(x - \bar{x})(y - \bar{y}) = 8$ and $\sum(x - \bar{x})^2 = 10$, ratio 0.8. Fitted values and residuals:

x	1	2	3	4	5
y	2	4	5	4	6
$\hat{y}$	2.6	3.4	4.2	5.0	5.8
$y - \hat{y}$	-0.6	+0.6	+0.8	-1.0	+0.2

The residuals sum to 0.000 and $\sum x_i(y_i - \hat{y}_i) = 0.000$, as the normal equations demand. RSS = 2.40, TSS = $\sum(y - \bar{y})^2 = 8.8$, so $R^2 = 1 - 2.4/8.8 = 0.7273$, which equals r^2 with $r = 0.8528$. Prediction at $x = 6$: 6.60. (Checked against `LinearRegression`, `numpy.polyfit` and `scipy.stats.linregress`.)



Example 3.3.2 (Is the slope significant?)

Residual variance $s^2 = \text{RSS}/(n-2) = 0.8$; standard error $\text{SE}(\hat{b}_1) = \sqrt{s^2/\sum(x-\bar{x})^2} = 0.2828$; $t = 0.8/0.2828 = 2.828$ on 3 degrees of freedom, two-sided $p = 0.0663$. With five points, an R^2 of 0.73 is *not* significant at the 5% level. Small samples make large R^2 cheap.

Remark 3.3.3 — Trap. Regressing x on y does *not* give slope $1/\hat{b}_1$. It gives $r s_x/s_y$, and the product of the two slopes is $r^2 \leq 1$. Both lines pass through $(\bar{x}, \bar{y})$ and coincide only if $|r| = 1$. Also: the slope has units of y per unit of x , so “a bigger coefficient means a more important feature” is meaningless unless features are on a common scale.

§3.4 Gradient descent, traced

Prototypical example for this section: Three gradient steps on the five points, by hand.

For MSE $J(\mathbf{w}) = \frac{1}{n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$ the gradient is $\frac{2}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$ and the update is $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$. In coordinates, with $e_i = \hat{y}_i - y_i$:

$$w_0 \leftarrow w_0 - \eta \cdot \frac{2}{n} \sum e_i, \quad w_1 \leftarrow w_1 - \eta \cdot \frac{2}{n} \sum e_i x_i.$$

Example 3.4.1 (The first iterations, $\eta = 0.05$, from $\mathbf{w} = \mathbf{0}$)

t	w_0	w_1	MSE	$\partial J/\partial w_0$	$\partial J/\partial w_1$
0	0.0000	0.0000	19.4000	-8.4000	-28.4000
1	0.4200	1.4200	1.4792	0.9600	5.3600
2	0.3720	1.1520	0.8662	-0.7440	-0.8240
3	0.4092	1.1932	0.8338	-0.4224	0.3056

Step 0: all predictions are 0, errors are $-y$, so $\partial J/\partial w_0 = -2 \cdot 21/5 = -8.4$ and $\partial J/\partial w_1 = -2 \cdot 71/5 = -28.4$; the step moves to $(0.42, 1.42)$. After 500 steps $\mathbf{w} = (1.7997, 0.8001)$, the closed form to three decimals, and the MSE floor is $\text{RSS}/n = 0.4800$.

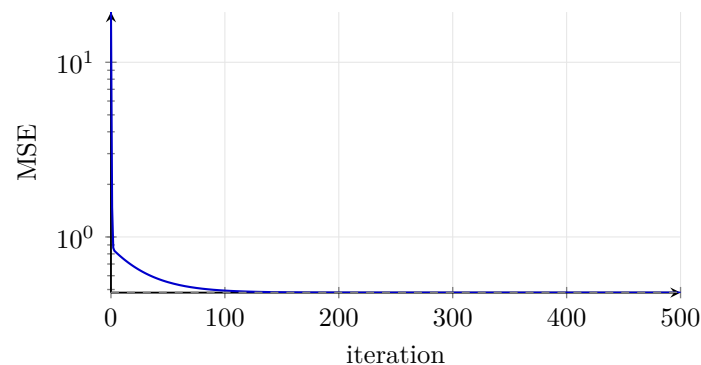


Figure 3.2: GD on the five points: a fast drop along the steep direction, then slow progress along the shallow one (the intercept), typical of an ill-conditioned quadratic.

On a quadratic, GD converges iff $\eta < 2/\lambda_{\max}$ of the Hessian $\frac{2}{n} \mathbf{X}^\top \mathbf{X}$; here that bound

is 0.0845, so $\eta = 0.05$ is safe and $\eta = 0.1$ would diverge (Chapter 16). Unscaled features make $\lambda_{\max}$ huge, forcing a tiny η and slow convergence along the other directions: the practical reason to standardise before gradient descent, even though OLS's solution does not care.

§3.5 R^2 and adjusted R^2

Prototypical example for this section: Adding twenty columns of random noise raises R^2 from 0.64 to 0.83.

Definition 3.5.1. $R^2 = 1 - \text{RSS} / \text{TSS}$ with $\text{TSS} = \sum (y_i - \bar{y})^2$: the fraction of variance of y explained by the model, relative to predicting $\bar{y}$. With p features (excluding the intercept),

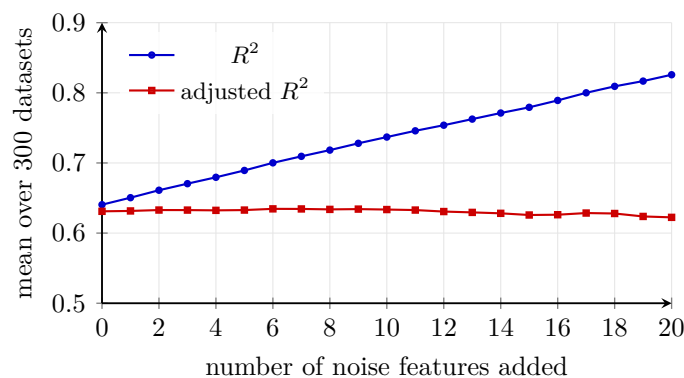
$$R^2_{\text{adj}} = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}.$$

Facts worth stating crisply:

- On the training data with an intercept, $0 \leq R^2 \leq 1$, and R^2 never decreases when you add a feature (the old fit is still available).
- On *test* data R^2 can be negative: the model is worse than predicting the training mean. `sklearn's r2_score` returns such values.
- Adjusted R^2 rises when a new feature's $|t| > 1$ and falls otherwise; it penalises features but is not a substitute for validation.

Example 3.5.2 (Junk features)

$n = 40$, one real feature, then $k = 0, \dots, 20$ added columns of pure noise, averaged over 300 datasets. R^2 climbs from 0.641 to 0.826; adjusted R^2 stays flat ($0.631 \rightarrow 0.622$). OA version: $R^2 = 0.80$, $n = 50$, $p = 4$ gives $R^2_{\text{adj}} = 1 - 0.2 \cdot 49/45 = 0.7822$.



Remark 3.5.3 — Trap. “High R^2 means the model is correct” and “low R^2 means it is useless” are both wrong. A curved relationship fitted by a line can have $R^2 = 0.9$ and systematic residuals; a noisy but real effect (returns on a stock) can have $R^2 = 0.02$ and be valuable. R^2 also depends on the spread of x in the sample: restrict the range and R^2 drops with the same underlying slope.

§3.6 Assumptions, diagnostics, multicollinearity

Prototypical example for this section: Two nearly identical features — area in square feet and in square metres rounded differently — get coefficients +300 and −290.

§3.6.i The assumptions, and what breaks when each fails

Assumption	Diagnostic	If violated
Linearity of $\mathbb{E}[y \mid \mathbf{x}]$	residuals vs fitted show a curve	biased predictions; add features/transforms or a non-linear model
Independent errors	residuals vs time/order, Durbin–Watson	standard errors too small; use time-series models or clustered errors
Homoscedasticity (constant variance)	funnel-shaped residual plot	OLS still unbiased but SEs wrong; log-transform y , weighted LS, robust SEs
Normal errors	Q–Q plot of residuals	only matters for small-sample tests and intervals
No perfect collinearity	rank of $\mathbf{X}$, VIF	no unique solution; drop/combine features, ridge
No influential outliers	leverage h_{ii} , Cook’s distance	a single point drags the line; robust losses

Remark 3.6.1 — Trap. “Linear regression requires x and y to be normally distributed.” No assumption is made about the distribution of x at all, and the normality assumption (needed only for exact t/F inference) is on the *errors*, not on y .

§3.6.ii Multicollinearity and VIF

When features are highly correlated, $\mathbf{X}^\top \mathbf{X}$ is nearly singular, $(\mathbf{X}^\top \mathbf{X})^{-1}$ has huge entries, and since $\text{Cov}(\hat{\mathbf{w}}) = \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$ the individual coefficients have huge variance. Predictions inside the data range are fine; *interpretation* of individual coefficients is not.

Definition 3.6.2. The **variance inflation factor** of feature j is $\text{VIF}_j = 1/(1 - R_j^2)$, where R_j^2 is the R^2 of regressing feature j on all the other features. Equivalently, VIF_j is the j -th diagonal entry of the inverse of the feature correlation matrix. Rules of thumb: above 5 worrying, above 10 serious.

Example 3.6.3 (VIF and unstable coefficients)

Features a, b, c with $\text{Corr}(a, b) = 0.950$ and c independent. VIFs are (10.34, 10.32, 1.00) (computed both by auxiliary regressions and from the inverse correlation matrix). Fit $y = 1 + a + b + c + \varepsilon$ on 500 bootstrap resamples: the standard deviation of $\hat{w}_a$ is 0.196, about 3.7 times that of $\hat{w}_c$ (0.053), roughly $\sqrt{\text{VIF}}$ as theory predicts. But the *sum* $\hat{w}_a + \hat{w}_b$ has standard deviation only 0.062: the data pin down the combined effect, not how to split it.

Remark 3.6.4 — Trap. Multicollinearity does not bias OLS and does not hurt prediction on data like the training data. It inflates the variance of individual coefficients, flips signs, and makes “feature importance from coefficients” meaningless. Remedies: drop one of a correlated pair, combine them, use ridge (which splits weight between them), or PCA regression.

§3.6.iii Interpreting coefficients

$\hat{w}_j$ is the change in predicted y per unit change in x_j *holding the other features fixed*. With $\log y$ as the target, $100\hat{w}_j$ is approximately the percentage change in y per unit of x_j ; with both logged, $\hat{w}_j$ is an elasticity. A dummy variable’s coefficient is the shift relative to the omitted (reference) category; including all K dummies plus an intercept is the **dummy variable trap** (perfect collinearity).

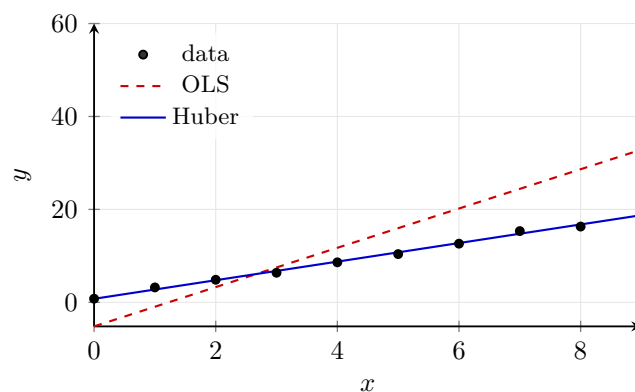
§3.7 Polynomial features and robust losses

Prototypical example for this section: Fit $y = w_0 + w_1x + w_2x^2$ with the same OLS machinery.

Replacing $\mathbf{x}$ by a feature map $\phi(\mathbf{x})$ (powers, interactions, splines, logs) keeps the problem linear in $\mathbf{w}$, so all of the above applies. The cost is dimension: all monomials of degree $\leq D$ in p variables number $\binom{p+D}{D}$. For $p = 10$: degree 2 gives 66 features and degree 3 gives 286; $p = 100$ at degree 2 already gives 5151 (counts checked against `PolynomialFeatures`). High degrees overfit and extrapolate wildly (Figure 1.1); scale x first, because x^9 for $x \sim 1000$ wrecks conditioning. Kernels (Chapter 8) are a way to use enormous feature maps without forming them.

Example 3.7.1 (One outlier)

Ten points on $y \approx 2x + 1$, with the last y replaced by 60. The OLS slope becomes 4.23; the Huber loss (quadratic for small residuals, linear for large ones) gives 2.00. Squared loss lets one point with a large residual dominate the fit.



Linear in the parameters, not in the inputs: feature maps buy flexibility, squared loss buys a closed form and sensitivity to outliers.

§3.8 A few problems to think about

Problem 3A (Fit a line). Fit $y = b_0 + b_1x$ to $(1, 1), (2, 3), (3, 2), (4, 5), (5, 4)$. Report b_0, b_1, R^2 , and the prediction at $x = 7$.

[hint · solution](#)

Problem 3B (Slope from summary statistics). For 100 students, the correlation between study hours and score is 0.6, the standard deviation of hours is 2 and of scores is 10. Mean hours 5, mean score 60. Give the regression line of score on hours and the predicted score for 8 hours. What is the slope of the regression of hours on score?

[hint · solution](#)

Problem 3C (Duplicated feature). You accidentally include the same feature twice, $x_2 = x_1$. What happens to (a) the normal equations, (b) `sklearn`'s `LinearRegression`, (c) the predictions, (d) ridge?

[hint · solution](#)

Problem 3D (Adjusted R^2). Model A has 3 features and $R^2 = 0.70$; model B has 10 features and $R^2 = 0.74$; $n = 60$. Which has higher adjusted R^2 ?

[hint · solution](#)

Problem 3E (Residual orthogonality). Prove that for OLS with an intercept, $\sum_i (y_i - \hat{y}_i) = 0$ and $\sum_i \hat{y}_i (y_i - \hat{y}_i) = 0$. Deduce $\text{TSS} = \text{RSS} + \text{ESS}$ where $\text{ESS} = \sum (\hat{y}_i - \bar{y})^2$.

[hint · solution](#)

Problem 3F (Learning-rate bound). For $J(\mathbf{w}) = \frac{1}{n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$, show that gradient descent converges for every starting point iff $0 < \eta < 2/\lambda_{\max}$ where $\lambda_{\max}$ is the largest eigenvalue of $\frac{2}{n} \mathbf{X}^\top \mathbf{X}$ (assume full column rank).

[hint · solution](#)

4 Bias, Variance and Regularisation

Chapter 1 showed test error as a U-shape in model capacity. This chapter explains the U (bias falls, variance rises), then gives you the dial that moves along it without changing the model class: a penalty λ on the size of the weights. Nearly every “effect of λ ” and “bagging versus boosting” question in an OA is this chapter in disguise.

§4.1 The decomposition

Prototypical example for this section: Refit the same degree-9 polynomial on 200 different training sets and look at the spread of the 200 curves (variance) and at how far their average is from the truth (bias).

§4.1.i Setting

Let $y = f(\mathbf{x}) + \varepsilon$ with $\mathbb{E}\varepsilon = 0$, $\text{Var } \varepsilon = \sigma^2$, independent of $\mathbf{x}$. A learning algorithm applied to a random training set S produces $\hat{f}_S$. Fix a test input $\mathbf{x}_0$ and take expectations over both S and the test noise.

Theorem 4.1.1 (Bias–variance decomposition)

$$\mathbb{E}_{S,\varepsilon}[(y_0 - \hat{f}_S(\mathbf{x}_0))^2] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{(f(\mathbf{x}_0) - \mathbb{E}_S \hat{f}_S(\mathbf{x}_0))^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_S (\hat{f}_S(\mathbf{x}_0) - \mathbb{E}_S \hat{f}_S(\mathbf{x}_0))^2}_{\text{variance}}.$$

Proof. Write $\bar{f} = \mathbb{E}_S \hat{f}_S(\mathbf{x}_0)$ and $f = f(\mathbf{x}_0)$. Then $y_0 - \hat{f}_S = \varepsilon + (f - \bar{f}) + (\bar{f} - \hat{f}_S)$. Square and take expectations. The three cross terms vanish: ε is independent of S with mean zero, $f - \bar{f}$ is a constant, and $\mathbb{E}_S(\bar{f} - \hat{f}_S) = 0$ by definition of $\bar{f}$. $\square$

Read the three terms as an archer. *Noise* is wind you cannot control. *Bias* is a sight that is misaligned: the average arrow lands off-centre. *Variance* is a shaky hand: the arrows scatter around their own average. A rigid model (low degree, big λ) has a steady hand and a bad sight; a flexible one has a good sight and a shaky hand.

Remark 4.1.2 — The decomposition is exact for squared loss. For 0-1 loss there is no clean additive version (several competing definitions exist), which is why MCQs that say “boosting reduces bias” are using the words loosely.

Example 4.1.3 (Measured, not assumed)

Degree-9 polynomial features on 25 noisy points from $\sin 2\pi x$ ($\sigma^2 = 0.09$), ridge penalty swept from $\lambda = 10^{-8}$ to 10^2 , 200 training sets. Averaged over test points:

	tiny $\lambda = 10^{-8}$	best $\lambda \approx 10^{-2}$	big $\lambda = 10^2$
bias ²	0.000	0.004	0.308
variance	0.554	0.025	0.029

The script also draws fresh noisy test targets and checks that $\text{bias}^2 + \text{variance} + \sigma^2$ matches the simulated test MSE at every λ .

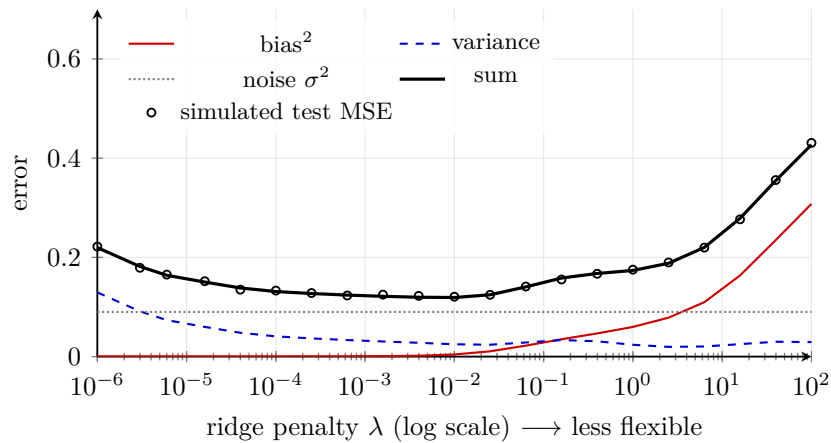


Figure 4.1: Bias–variance trade-off along the ridge path. Moving right (more regularisation) trades variance for bias; the sum is minimised in between.

§4.1.ii What moves which term

Change	bias	variance
more training data (same model)	$\approx$ same	$\downarrow$
more features / higher degree / deeper tree	$\downarrow$	$\uparrow$
larger λ (ridge, lasso, weight decay)	$\uparrow$	$\downarrow$
larger k in k -NN	$\uparrow$	$\downarrow$
bagging / random forest (averaging many high-variance models)	$\approx$ same	$\downarrow$
boosting (sequentially fitting residuals of weak models)	$\downarrow$	$\uparrow$ (slowly)
early stopping (fewer iterations)	$\uparrow$	$\downarrow$

Remark 4.1.4 — Trap. “More data reduces bias.” More data shrinks variance; it cannot fix a model class that cannot represent f (a line fitted to a parabola stays wrong with a billion points). If training error is high, more data will not help — that is the diagnostic. If the gap is large, more data will.

Underfitting is bias, overfitting is variance; regularisation and averaging buy lower variance with a little bias.

§4.2 Ridge regression

Prototypical example for this section: Shrinking the five-point slope from 0.8 towards 0 as λ grows.

§4.2.i Definition and closed form

Definition 4.2.1. Ridge regression (Tikhonov, ℓ_2 regularisation) minimises

$$\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_2^2, \quad \lambda \geq 0,$$

with the intercept *not* penalised (centre $\mathbf{X}$ and $\mathbf{y}$ first, or exclude it explicitly).

The gradient is $-2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) + 2\lambda\mathbf{w}$, so

$$\hat{\mathbf{w}}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}.$$

$\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ is positive definite for any $\lambda > 0$, so the solution exists and is unique even when $\mathbf{X}^\top \mathbf{X}$ is singular ($n < d$, duplicated features). Historically, that was the point.

Example 4.2.2 (One feature, by hand)

Centre the five points of [Chapter 3](#): $\sum x_i y_i = 8$, $\sum x_i^2 = 10$. Without intercept, $\hat{w} = \sum x_i y_i / (\sum x_i^2 + \lambda) = 8 / (10 + \lambda)$:

λ	0	1	10	40	1000
$\hat{w}$	0.8000	0.7273	0.4000	0.1600	0.0079

At $\lambda = \sum x_i^2$ the slope is exactly halved. All non-zero cases match `Ridge(fit_intercept=False)`.

§4.2.ii The SVD view: shrinkage by direction

With $\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^\top$ (singular values s_j),

$$\mathbf{X}\hat{\mathbf{w}}_{\text{ridge}} = \sum_j \mathbf{u}_j \frac{s_j^2}{s_j^2 + \lambda} \mathbf{u}_j^\top \mathbf{y}.$$

OLS keeps every principal direction fully (factor 1); ridge multiplies direction j by $s_j^2 / (s_j^2 + \lambda)$, shrinking the low-variance directions most — exactly the directions in which the data say least and the estimate is noisiest. The **effective degrees of freedom** $\text{df}(\lambda) = \sum_j s_j^2 / (s_j^2 + \lambda) = \text{Tr } \mathbf{H}_\lambda$ falls from d to 0.

Example 4.2.3 (Diabetes data, $\lambda = 50$)

On scikit-learn's diabetes data ($n = 442$, $d = 10$, standardised), $\lambda = 50$ keeps the top principal direction at factor 0.973 and the bottom one at 0.070; $\text{df} = 7.51$. The closed form, the SVD formula and `Ridge` agree to 10^{-6} .

§4.2.iii The Bayesian view

As derived in [Chapter 2](#), ridge is the MAP estimate under $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 \mathbf{I})$ with Gaussian noise σ^2 , and $\lambda = \sigma^2 / \tau^2$. “Large λ ” means “I strongly believe the weights are small”; $\lambda \rightarrow 0$ is a flat prior and recovers OLS.

Remark 4.2.4 — Trap. Ridge is *not* scale-invariant. Penalising w_j^2 treats a feature measured in rupees and one measured in crores identically, so the crores feature

(tiny values, huge coefficient) is punished far more. Standardise features before ridge, lasso, elastic net, SVMs, k -NN and PCA (Chapter 15). OLS and trees do not need it.

§4.3 Lasso and elastic net

Prototypical example for this section: With an orthonormal design, lasso turns OLS coefficients $(3, -1.2, 0.4, -0.1)$ into $(2.5, -0.7, 0, 0)$.

Definition 4.3.1. The **lasso** minimises $\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_1$ (scikit-learn's scaling). The **elastic net** minimises $\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda(\alpha \|\mathbf{w}\|_1 + \frac{1-\alpha}{2} \|\mathbf{w}\|_2^2)$.

§4.3.i Why the lasso gives exact zeros

Three explanations, each enough for an interview.

1. **Subgradient.** In one dimension with standardised feature, the optimality condition is $w = z - \lambda \partial |w|$ where z is the OLS coefficient. The subgradient of $|w|$ at 0 is the whole interval $[-1, 1]$, so $w = 0$ is optimal whenever $|z| \leq \lambda$. The solution is the **soft-thresholding** operator $S_\lambda(z) = \text{sign}(z) \max(|z| - \lambda, 0)$. Ridge's penalty w^2 has derivative $2w$, which vanishes at 0, so it only rescales: $z/(1 + \lambda)$.
2. **Geometry.** Equivalently minimise RSS subject to $\|\mathbf{w}\|_1 \leq t$. The RSS contours are ellipses; the ℓ_1 ball is a diamond with corners on the axes, so the first contact typically happens at a corner, where some coordinates are zero. The ℓ_2 ball is round and has no corners (Figure 4.2).
3. **Prior.** Lasso is MAP under a Laplace prior, which has a sharp peak at zero.

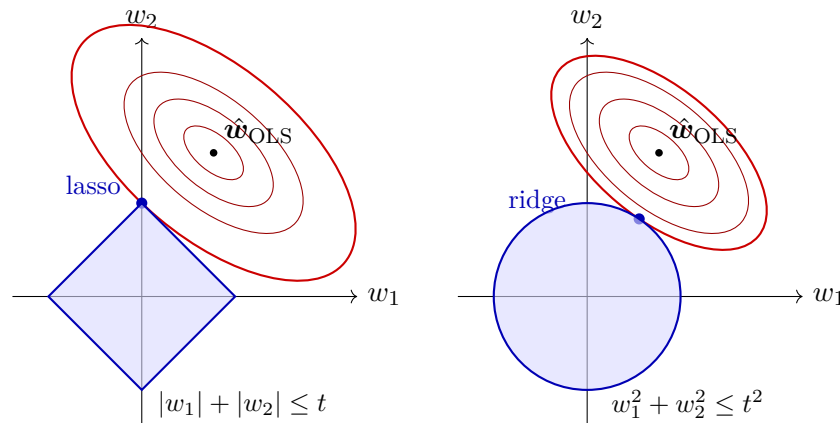


Figure 4.2: Constrained forms: the RSS ellipses first touch the ℓ_1 diamond at a corner (a sparse solution, $w_1 = 0$), and the ℓ_2 disc at a generic point (both weights non-zero). The contact points are computed in `code/ml/c04_biasvar.py`.

Example 4.3.2 (Soft versus proportional shrinkage)

Orthonormal design, OLS coefficients $(3, -1.2, 0.4, -0.1)$. Lasso with threshold 0.5 gives $(2.5, -0.7, 0, 0)$ — verified by running `Lasso` on a design with orthonormal

columns. Ridge with $\lambda = 1$ gives $(1.5, -0.6, 0.2, -0.05)$: everything halved, nothing zero. Lasso shrinks every surviving coefficient by the same *amount*; ridge by the same *factor*.

§4.3.ii Coordinate descent

Because the one-dimensional problem has the closed-form soft-threshold solution, the lasso is solved by cycling through coordinates, each time soft-thresholding the partial residual correlation. This is what scikit-learn does; our dozen-line version matches `Lasso` on the diabetes data to 10^{-4} :

code/ml/c04_biasvar.py: lasso by coordinate descent

```
1 def lasso_cd(X, y, lam, iters=500):
2     """min 1/(2n)||y - Xw||^2 + lam ||w||_1 by cyclic coordinate descent."""
3     n, d = X.shape
4     w = np.zeros(d)
5     col_sq = (X ** 2).sum(0) / n
6     r = y - X @ w
7     for _ in range(iters):
8         for j in range(d):
9             r += X[:, j] * w[j]                # remove j's contribution
10            rho = X[:, j] @ r / n
11            w[j] = np.sign(rho) * max(abs(rho) - lam, 0) / col_sq[j]
12            r -= X[:, j] * w[j]
13     return w
```

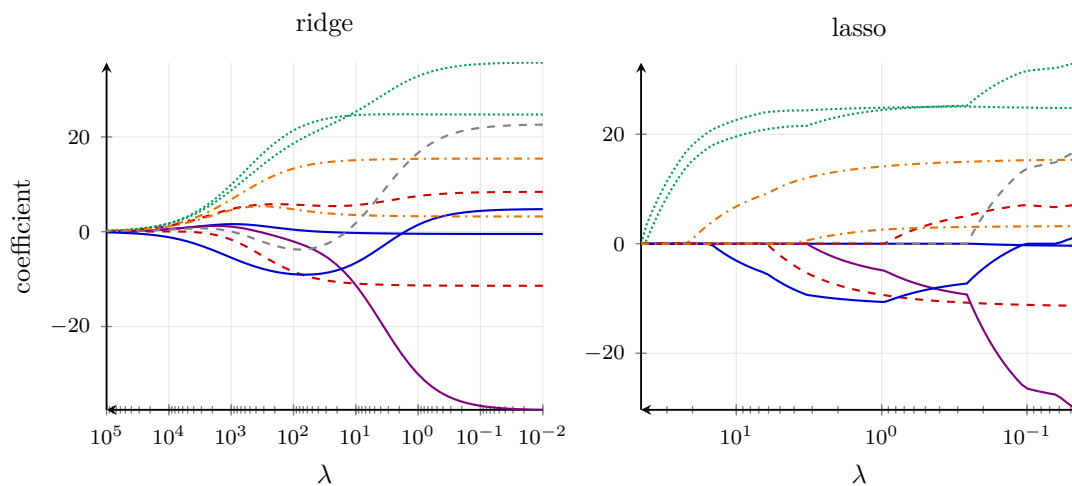


Figure 4.3: Regularisation paths on the diabetes data (penalty decreasing to the right). Ridge coefficients shrink smoothly and are never exactly zero; lasso coefficients enter one at a time.

On the diabetes data, lasso keeps 7, 5 and 3 of the 10 features at $\lambda = 1, 5, 20$; ridge with $\lambda = 10^4$ still has all 10 non-zero.

§4.3.iii Correlated features: why elastic net exists

Lasso is unstable with groups of correlated features: it tends to pick one arbitrarily and zero the rest, and which one can flip between bootstrap samples. Ridge spreads weight

evenly. Elastic net does both: sparse across groups, shared within a group.

Example 4.3.3 (Two near-copies)

x_1 and x_2 are noisy copies of the same signal z (correlation > 0.99), x_3 is independent, $y = 2z + x_3 + \varepsilon$. Coefficients on (x_1, x_2, x_3) : lasso (1.92, 0.00, 0.90), elastic net (0.96, 0.96, 0.90), ridge (0.99, 0.98, 0.95).

Remark 4.3.4 — Trap. “Lasso’s zero coefficients prove those features are irrelevant.” A zeroed feature may be strongly predictive but redundant given a correlated one that was kept. Also, the lasso can select at most n features when $d > n$; elastic net removes that limit.

§4.4 The augmented error

Prototypical example for this section: OA: “In $E_{\text{aug}}(\mathbf{w}) = E_{\text{in}}(\mathbf{w}) + \lambda \sum_j w_j^2$, what happens as $\lambda \rightarrow 0$ and as $\lambda \rightarrow \infty$?”

The penalised objective $E_{\text{aug}} = E_{\text{in}} + \lambda \Omega(\mathbf{w})$ is the general form: data fit plus complexity. Answers to memorise:

- $\lambda = 0$: no regularisation; the unconstrained minimiser of E_{in} ; lowest training error, highest variance; overfits if the model is flexible.
- $\lambda \rightarrow \infty$: the penalty dominates; $\mathbf{w} \rightarrow \mathbf{0}$ (the intercept, unpenalised, goes to $\bar{y}$): the model predicts a constant; maximal bias, zero variance; underfits.
- Increasing λ : training error *never decreases*; validation error typically falls then rises; the effective number of parameters decreases; weights shrink.
- λ is a hyper-parameter: chosen by validation or cross-validation, never on the training set (the training error would always pick $\lambda = 0$).
- In neural networks, the same penalty is called **weight decay**, because the gradient step becomes $\mathbf{w} \leftarrow (1 - 2\eta\lambda)\mathbf{w} - \eta\nabla E_{\text{in}}$: the weights decay geometrically each step. (Under Adam the two are not equivalent; see *Deep Learning*.)

Remark 4.4.1 — Trap. Three plausible-sounding wrong options. “Regularisation reduces training error” (it increases it). “Large λ causes overfitting” (it causes underfitting). “ λ is learnt by gradient descent along with $\mathbf{w}$ ” (then it would go to 0).

λ is a bias dial: zero gives you the raw fit, infinity gives you a constant, validation finds the middle.

§4.5 Early stopping and double descent

Prototypical example for this section: Gradient descent from $\mathbf{w} = \mathbf{0}$ on 60 points with 50 features: validation error is best after about 1000 steps, then creeps up.

§4.5.i Early stopping is implicit regularisation

Gradient descent started at $\mathbf{0}$ moves first along high-variance principal directions (large s_j) and only later fits the low-variance ones. Stopping after t steps therefore acts like ridge with $\lambda \approx 1/(\eta t)$: the number of iterations is a regulariser.

Example 4.5.1 (Where to stop)

$n = 60$ training points, $d = 50$ features of which 5 matter, noise variance 4. Validation MSE is minimal (8.79) at iteration 1370; by iteration 3000 training MSE is 0.71 (below the noise level — it is fitting noise) and validation MSE has risen to 9.69.

§4.5.ii Double descent

The classical U-curve is not the whole story. Fit *minimum-norm* least squares (what gradient descent from zero converges to) with p of the available features on n points. As p approaches n the model can just barely interpolate, the solution is forced through every noisy point with huge weights, and test error spikes. Past $p = n$ there are infinitely many interpolating solutions and the minimum-norm one becomes smoother as p grows, so test error falls again — **double descent** [Be+19].

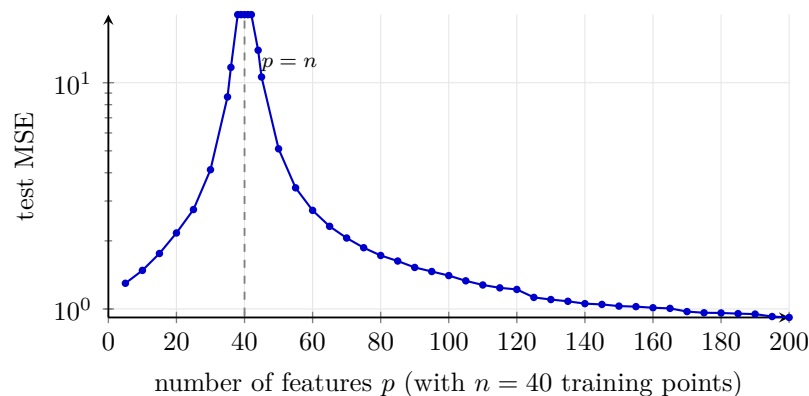


Figure 4.4: Double descent for minimum-norm least squares (100 repetitions, peak clipped at 20 for display). Error peaks at the interpolation threshold $p = n$ and falls again beyond it.

In our run the peak sits at $p = 40$; at $p = 200$ the error is 0.92, better than the best under-parameterised model (1.30). Adding a little ridge removes the spike entirely. The interview point: “more parameters always means more overfitting” is false for interpolating models trained with implicit or explicit norm control, which is part of why huge neural networks generalise.

§4.6 A few problems to think about

Problem 4A (Effect of λ , multi-select). For ridge regression with penalty $\lambda \sum_j w_j^2$, which are true? (a) Training MSE is non-decreasing in λ . (b) As $\lambda \rightarrow \infty$ every prediction tends to the mean of the training targets. (c) The solution is unique for every $\lambda > 0$ even if $d > n$. (d) Ridge sets some coefficients exactly to zero for large enough λ . (e) Ridge results do not depend on the units of the features.



[hint · solution](#)

Problem 4B (One-dimensional ridge and lasso). With a single standardised feature and no intercept, the OLS coefficient is $\hat{w} = 2.4$. Give the lasso solution for thresholds $\lambda = 1$ and $\lambda = 3$, and the ridge solution $\hat{w}/(1 + \lambda)$ for $\lambda = 1$ and $\lambda = 3$.

[hint · solution](#)

Problem 4C (Diagnose from numbers). A model has training MSE 4.1 and validation MSE 4.3; the noise variance is believed to be about 1. Another has training MSE 0.2 and validation MSE 6.0. For each, would you increase or decrease λ , and would collecting more data help?

[hint · solution](#)

Problem 4D (Ridge from MAP). Show that with $y_i = \mathbf{w}^\top \mathbf{x}_i + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ and prior $w_j \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \tau^2)$, the MAP estimate is ridge with $\lambda = \sigma^2/\tau^2$. What prior gives the lasso, and what is the corresponding λ ?

[hint · solution](#)

Problem 4E (Bias and variance of a shrunk mean). Estimate μ from n i.i.d. samples with variance σ^2 using $\hat{\mu}_c = c\bar{X}$ for $0 \leq c \leq 1$. Compute its bias, variance and MSE, and find the c minimising MSE. Why can a biased estimator win?

[hint · solution](#)

Problem 4F (Lasso subgradient condition). For the objective $\frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_1$, show that $\hat{\mathbf{w}} = \mathbf{0}$ is optimal if and only if $\lambda \geq \|\mathbf{X}^\top \mathbf{y}\|_\infty$. What is the practical use of this fact?

[hint · solution](#)

5 Logistic Regression and GLMs

Logistic regression is the most-asked classifier in Indian placement interviews, because it is simultaneously a linear model, a probabilistic model, a one-neuron neural network and the baseline every production team actually ships first. You should be able to derive its loss from likelihood, its gradient in one line, its Newton step, and explain why it is not trained with MSE.

§5.1 From odds to the sigmoid

Prototypical example for this section: A loan model with $z = -3 + 0.8 \cdot \text{income} + 1.2 \cdot \text{has_job}$; an applicant with income 2 and a job.

We want $\mathbb{P}(y = 1 \mid \mathbf{x}) \in (0, 1)$ from a linear score $z = \mathbf{w}^\top \mathbf{x} + b \in \mathbb{R}$. Linear regression on 0/1 targets produces values outside $[0, 1]$ and is dragged around by far-away points. Instead model the **log-odds** (**logit**) as linear:

$$\log \frac{p}{1-p} = \mathbf{w}^\top \mathbf{x} + b \quad \Longleftrightarrow \quad p = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The **sigmoid** squashes $\mathbb{R}$ to $(0, 1)$ with $\sigma(0) = 0.5000$, $\sigma(2) = 0.8808$, $\sigma(-2) = 0.1192 = 1 - \sigma(2)$, $\sigma(4) = 0.9820$. Two identities used constantly:

$$\sigma(-z) = 1 - \sigma(z), \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}.$$

Example 5.1.1 (Score an applicant)

$z = -3 + 0.8 \cdot 2 + 1.2 \cdot 1 = -0.20$, so $p = \sigma(-0.2) = 0.4502$ and the odds are $e^{-0.2} = 0.8187$. With threshold 0.5 the applicant is predicted “no”.

§5.1.i Reading coefficients

A unit increase in x_j adds w_j to the log-odds, i.e. *multiplies the odds* by e^{w_j} (the **odds ratio**). $w = 0.7$ roughly doubles the odds ($e^{0.7} = 2.014$); $w = -0.5$ multiplies them by 0.607. The effect on the *probability* is not constant: it is largest near $p = 0.5$ and tiny near 0 or 1, because of the factor $p(1-p)$.

Remark 5.1.2 — Trap. “Logistic regression is a regression algorithm because of its name.” It is a classifier; it regresses the log-odds. And its decision boundary $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$ is a *hyperplane*: logistic regression is a linear classifier, despite the non-linear sigmoid. The sigmoid is monotone, so thresholding $\sigma(z)$ at 0.5 is thresholding z at 0.

§5.2 Log-loss from maximum likelihood

Prototypical example for this section: Four points on a line, one gradient step from $\mathbf{w} = \mathbf{0}$.

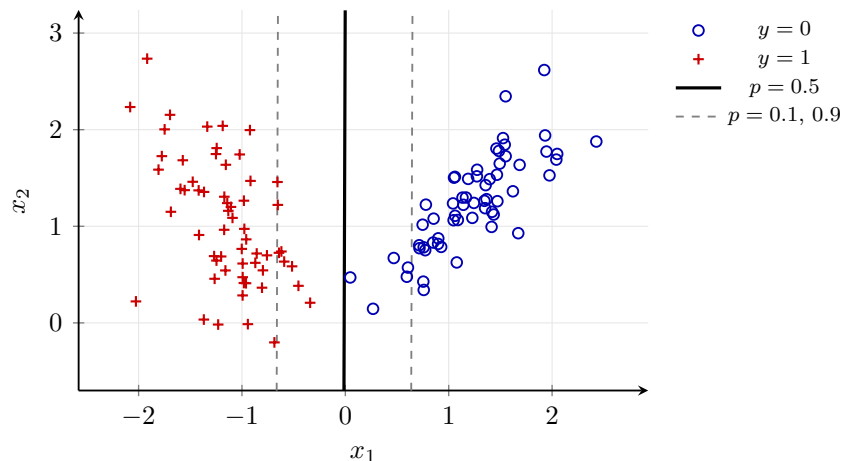


Figure 5.1: A fitted logistic regression: the contours of p are parallel lines, and the decision boundary is the $p = 0.5$ line.

With $p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$ (absorb b into $\mathbf{w}$ with a ones feature), each label is Bernoulli(p_i), so $\mathbb{P}(y_i | \mathbf{x}_i) = p_i^{y_i} (1 - p_i)^{1-y_i}$. The negative log-likelihood, averaged, is the **log-loss** (**binary cross-entropy**):

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

With labels $y \in \{-1, +1\}$ the same loss reads $\frac{1}{n} \sum \log(1 + e^{-y_i \mathbf{w}^\top \mathbf{x}_i})$, the logistic surrogate of Figure 2.3.

Theorem 5.2.1 (The gradient)

$$\begin{aligned} \nabla J(\mathbf{w}) &= \frac{1}{n} \sum_i (p_i - y_i) \mathbf{x}_i = \frac{1}{n} \mathbf{X}^\top (\mathbf{p} - \mathbf{y}), \\ \nabla^2 J &= \frac{1}{n} \mathbf{X}^\top \mathbf{S} \mathbf{X}, \quad \mathbf{S} = \text{diag}(p_i(1 - p_i)). \end{aligned}$$

Proof. For one example, $\partial/\partial z$ of $-[y \log \sigma(z) + (1-y) \log(1-\sigma(z))]$ is $-y(1-\sigma) + (1-y)\sigma = \sigma - y$ using $\sigma' = \sigma(1-\sigma)$; then the chain rule with $\partial z/\partial \mathbf{w} = \mathbf{x}$. Differentiating $(\sigma(z) - y)\mathbf{x}$ once more gives $\sigma(1-\sigma)\mathbf{x}\mathbf{x}^\top$. $\square$

The Hessian is PSD, so J is *convex*: gradient descent cannot get stuck in a bad local minimum. Note the gradient has exactly the same form “(prediction – target) $\times$ input” as linear regression; this is the hallmark of the canonical link in a GLM (§5.7), and the same $(\hat{y} - y)$ reappears for softmax with cross-entropy in neural networks.

Example 5.2.2 (One step by hand)

Points $\mathbf{x} = (0.5, 1.5, 2.5, 3.5)$ with labels $(0, 0, 1, 1)$, features $(1, \mathbf{x})$, start at $\mathbf{w} = \mathbf{0}$. All $p_i = 0.5$, so the loss is $\log 2 = 0.6931$. Errors $p - y = (0.5, 0.5, -0.5, -0.5)$ give gradient $\frac{1}{4} \mathbf{X}^\top (\mathbf{p} - \mathbf{y}) = (0.000, -0.500)$. With learning rate 1, $\mathbf{w}_1 = (0.000, 0.500)$, new probabilities $(0.562, 0.679, 0.777, 0.852)$ and loss 0.5937 (matches

`sklearn.metrics.log_loss`; the gradient formula is also checked against finite differences).

§5.2.i Separable data and why we regularise

If a hyperplane separates the classes perfectly, scaling $\mathbf{w}$ up pushes every p_i towards its label and the loss towards 0, so the MLE does not exist: $\|\mathbf{w}\| \rightarrow \infty$. On four separable points, gradient descent's $\|\mathbf{w}\|$ after 10, 100, 1000, 10^4 steps is (2.10, 4.02, 6.23, 8.52) — growing like $\log t$ forever, with ever more over-confident probabilities. An ℓ_2 penalty fixes it: scikit-learn's default $C=1$ (that is, $\lambda = 1/C$) gives slope 1.01; $C = 10^4$ gives 7.54.

Remark 5.2.3 — Trap. In scikit-learn `LogisticRegression` is *regularised by default* ($C=1.0$, ℓ_2), and C is the *inverse* of the regularisation strength: larger C means weaker regularisation. Comparing its coefficients to a textbook MLE without noticing this is a classic interview slip.

§5.3 Newton's method and IRLS

Prototypical example for this section: Six Newton steps reach the maximum likelihood to machine precision; gradient descent needs about two hundred.

Newton's update is $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{H}^{-1} \nabla J = \mathbf{w} - (\mathbf{X}^\top \mathbf{S} \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{p} - \mathbf{y})$. Rearranging, $\mathbf{w}_{\text{new}} = (\mathbf{X}^\top \mathbf{S} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{S} \mathbf{z}$ with $\mathbf{z} = \mathbf{X} \mathbf{w} + \mathbf{S}^{-1}(\mathbf{y} - \mathbf{p})$: each Newton step is a *weighted least-squares* fit to a “working response” $\mathbf{z}$, hence **iteratively reweighted least squares** (IRLS). Points with p_i near 0.5 get the largest weight $p_i(1 - p_i)$.

code/ml/c05_logistic.py: Newton / IRLS

```
1 def logistic_newton(X, y, iters=10):
2     w = np.zeros(X.shape[1])
3     hist = []
4     for _ in range(iters):
5         p = 1 / (1 + np.exp(-X @ w))
6         grad = X.T @ (p - y)                # gradient of the negative log-
        ↪ likelihood
7         H = X.T @ (X * (p * (1 - p))[:, None])    # Hessian  $X^T S X$ ,  $S = \text{diag}(p$ 
        ↪  $(1-p)$ )
8         w = w - np.linalg.solve(H, grad)
9         p = 1 / (1 + np.exp(-X @ w))
10        hist.append(-np.sum(y * np.log(p) + (1 - y) * np.log(1 - p)))
11    return w, hist
```

Example 5.3.1 (Quadratic convergence)

On 300 points with three features, the negative log-likelihood after each Newton step:

step	NLL
1	185.285552
2	184.958723
3	184.958161
4	184.958161
5	184.958161
6	184.958161
7	184.958161

The weights $(-0.361, -0.432, 0.976, 0.372)$ agree with `LogisticRegression(C=np.inf)` to 10^{-5} . Plain gradient descent (step 0.5) needs 191 iterations to reach the same NLL within 10^{-6} . Newton costs $O(nd^2 + d^3)$ per step, so it wins for small d ; for large d use L-BFGS (scikit-learn's default solver) or stochastic methods ([Chapter 16](#)).

§5.4 Softmax regression and multiclass

Prototypical example for this section: Three iris species from four measurements.

§5.4.i Softmax

For K classes, keep one weight vector per class and set

$$\mathbb{P}(y = k \mid \mathbf{x}) = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad z_k = \mathbf{w}_k^\top \mathbf{x}.$$

The loss is the categorical cross-entropy $-\frac{1}{n} \sum_i \log p_{i,y_i}$, and the gradient with respect to the weight matrix is $\frac{1}{n} \mathbf{X}^\top (\mathbf{P} - \mathbf{Y})$ where $\mathbf{Y}$ is one-hot: the same “prediction minus target” form. Softmax is invariant to adding a constant to all logits, so the parametrisation has one redundant class; with $K = 2$ softmax reduces exactly to the sigmoid of $z_1 - z_0$. Implementations subtract $\max_k z_k$ before exponentiating ([Chapter 2](#)).

code/ml/c05_logistic.py: softmax regression

```
1 def softmax_regression(X, Y, lam, lr=0.5, iters=20000):
2     """min -1/n sum log p_{i,y_i} + lam/2 ||W[1:]||^2 by gradient descent."""
3     n, d = X.shape
4     W = np.zeros((d, Y.shape[1]))
5     for _ in range(iters):
6         Z = X @ W
7         Z -= Z.max(axis=1, keepdims=True) # stable softmax
8         P = np.exp(Z); P /= P.sum(axis=1, keepdims=True)
9         G = X.T @ (P - Y) / n # the famous (P - Y)
10        G[1:] += lam * W[1:] # do not penalise the intercept
11        W -= lr * G
12    return W
```

On standardised iris with $\lambda = 0.1$ these probabilities match scikit-learn's multinomial `LogisticRegression` (with $C = 1/(n\lambda)$) to 2×10^{-4} ; training accuracy 0.920.

§5.4.ii One-vs-rest and one-vs-one

Binary-only learners (classical SVMs, perceptrons) are extended by:

- **One-vs-rest** (OvR): K classifiers, class k against all others; predict the most confident. Each sees all n points; classes are imbalanced inside each problem.
- **One-vs-one** (OvO): $\binom{K}{2} = K(K-1)/2$ classifiers, one per pair, each trained on two classes only; predict by majority vote. For $K = 10$ that is 45 classifiers, each on about $2n/K$ points.

On iris ($K = 3$, so OvR and OvO both train 3 models) training accuracies are 0.947 (OvR) and 0.973 (OvO). Softmax regression needs neither: it is natively multiclass and its probabilities are coherent.

Remark 5.4.1 — Trap. Multilabel is not multiclass. If an article can be both “sports” and “business”, use K independent sigmoids with binary cross-entropy, not a softmax (which forces the probabilities to compete and sum to one).

§5.5 Why not MSE for classification?

Prototypical example for this section: The model is confidently wrong ($z = -5$ for a positive): how hard does each loss push?

Three reasons, in the order interviewers like:

1. **Vanishing gradient when confidently wrong.** With $p = \sigma(z)$ and $y = 1$, the cross-entropy gradient in z is $p - 1$; the MSE gradient is $2(p - 1)p(1 - p)$. At $z = -5$ these are -0.9933 versus -0.01321 : MSE barely moves a badly wrong prediction, because the sigmoid’s slope multiplies in.
2. **Non-convexity.** $(1 - \sigma(z))^2$ is not convex in z (its second derivative changes sign), so MSE◦sigmoid can have flat regions and poor local optima in $\mathbf{w}$; log-loss is convex.
3. **Wrong likelihood.** MSE is the Gaussian log-likelihood; labels are Bernoulli. Log-loss is a proper scoring rule that heavily penalises confident mistakes ($-\log p \rightarrow \infty$ as $p \rightarrow 0$), which is what produces calibrated probabilities.

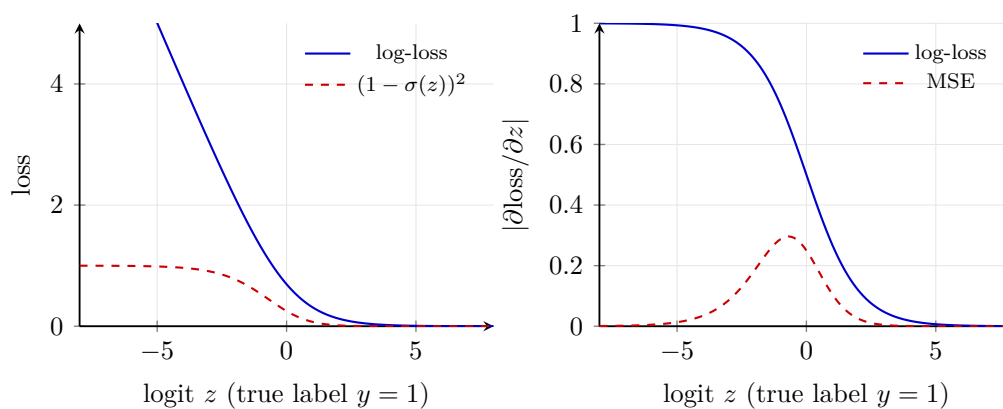


Figure 5.2: For a positive example, log-loss keeps a gradient near 1 when the model is badly wrong ($z \ll 0$); MSE’s gradient dies there.

§5.6 Imbalance and thresholds

Prototypical example for this section: 5% positives: a model that never predicts “positive” is 95% accurate.

Logistic regression estimates probabilities; the *decision* is a separate step. With costs c_{FN} for missing a positive and c_{FP} for a false alarm, predict positive when $p c_{FN} > (1 - p) c_{FP}$, i.e. threshold $t = c_{FP} / (c_{FP} + c_{FN})$. The default 0.5 assumes equal costs.

Example 5.6.1 (Threshold versus class weights)

Synthetic data, 0.051 positives in the test set:

model	accuracy	precision	recall
plain, $t = 0.5$	0.964	1.000	0.294
plain, $t = 0.2$	0.961	0.636	0.549
plain, $t = 0.1$	0.907	0.312	0.686
balanced weights, $t = 0.5$	0.799	0.174	0.784
always predict 0	0.949	0.000	0.000

Lowering the threshold and using `class_weight="balanced"` both trade precision for recall; “always predict 0” has near-top accuracy and zero recall. Choose the operating point from the business cost, using precision–recall curves (Chapter 13).

Remark 5.6.2 — Trap. “With imbalanced data, always oversample.” Re-weighting or resampling changes the model’s probabilities (they no longer estimate the true $\mathbb{P}(y = 1 \mid \mathbf{x})$) and is often equivalent to moving the threshold. First try: keep the model, pick the threshold on validation data, report PR-AUC rather than accuracy. Resampling must happen *inside* the training fold only (Chapter 15).

§5.7 Generalised linear models

Prototypical example for this section: Counts of app crashes per day modelled with $\log \mathbb{E}[y] = \mathbf{w}^\top \mathbf{x}$.

A **generalised linear model** has three parts: a distribution for y from the exponential family, a linear predictor $\eta = \mathbf{w}^\top \mathbf{x}$, and a **link** g with $g(\mathbb{E}[y \mid \mathbf{x}]) = \eta$.

model	distribution	link $g(\mu)$	typical target
linear regression	Gaussian	identity μ	real values
logistic regression	Bernoulli	logit $\log \frac{\mu}{1-\mu}$	binary
Poisson regression	Poisson	$\log \mu$	counts
softmax regression	categorical	multinomial logit	K classes

With the canonical link, the gradient of the negative log-likelihood is always $\mathbf{X}^\top (\boldsymbol{\mu} - \mathbf{y})$ and Newton’s method is IRLS. A Poisson regression fitted by our 6-line Newton loop gives (0.289, 0.937, −0.342), identical to scikit-learn’s `PoissonRegressor`.

§5.8 The perceptron and XOR

Prototypical example for this section: AND is learnable by one neuron; XOR is not.

The **perceptron** (Rosenblatt) predicts $\text{sign}(\mathbf{w}^\top \mathbf{x})$ and, on each mistake ($y_i \mathbf{w}^\top \mathbf{x}_i \leq 0$), updates $\mathbf{w} \leftarrow \mathbf{w} + y_i \mathbf{x}_i$. It is stochastic (sub)gradient descent on the loss $\max(0, -y \mathbf{w}^\top \mathbf{x})$, a hinge with the margin set to zero.

code/ml/c05_logistic.py: perceptron

```

1 def perceptron(X, y, epochs=20, lr=1.0):
2     """y in {-1,+1}; X includes a bias column. Returns weights and #mistakes per
3     ↪ epoch."""
4     w = np.zeros(X.shape[1])
5     mistakes = []
6     for _ in range(epochs):
7         m = 0
8         for xi, yi in zip(X, y):
9             if yi * (w @ xi) <= 0:      # misclassified (or on the boundary)
10                w += lr * yi * xi
11                m += 1
12        mistakes.append(m)
13        if m == 0:
14            break
15    return w, mistakes

```

Theorem 5.8.1 (Perceptron convergence (Novikoff))

If the data are linearly separable with margin γ by a unit vector $\mathbf{w}^*$ ($y_i \mathbf{w}^{*\top} \mathbf{x}_i \geq \gamma$) and $\|\mathbf{x}_i\| \leq R$, the perceptron makes at most $(R/\gamma)^2$ mistakes, whatever the order of examples.

Proof. Each mistake increases $\mathbf{w}^\top \mathbf{w}^*$ by at least γ and increases $\|\mathbf{w}\|^2$ by at most R^2 (because $y_i \mathbf{w}^\top \mathbf{x}_i \leq 0$ on a mistake). After M mistakes, $M\gamma \leq \mathbf{w}^\top \mathbf{w}^* \leq \|\mathbf{w}\| \leq R\sqrt{M}$, so $M \leq R^2/\gamma^2$. $\square$

Example 5.8.2 (AND converges, XOR cycles)

Inputs $(x_1, x_2) \in \{0, 1\}^2$ with a bias feature, labels ± 1 . For AND, mistakes per epoch are (2, 3, 3, 2, 2, 3, 2, 1, 0) and the final weights (bias first) are $(-4, 3, 2)$: predict 1 iff $3x_1 + 2x_2 > 4$. For XOR the perceptron never has an error-free epoch in 50 epochs; scikit-learn's **Perceptron** ends at accuracy 0.50, and a brute-force search over all lines shows no linear classifier exceeds 0.75.

Why no line works: the line would need $w_0 < 0$ (for $(0, 0) \mapsto -1$), $w_0 + w_1 > 0$ and $w_0 + w_2 > 0$ (for the two +1 points), and $w_0 + w_1 + w_2 < 0$. Adding the middle two gives $2w_0 + w_1 + w_2 > 0$, i.e. $w_0 + w_1 + w_2 > -w_0 > 0$, a contradiction.

Two ways out, both interview staples. Add a feature: with $x_1 x_2$ appended the perceptron converges to $(-1, 2, 2, -5)$, i.e. predict 1 iff $2x_1 + 2x_2 - 5x_1 x_2 > 1$ — a linear classifier in a lifted space (the idea behind kernels, [Chapter 8](#)). Or add a hidden layer: XOR = (OR) AND NOT (AND), two neurons feeding a third, the opening example of *Deep Learning*.

Remark 5.8.3 — Trap. “A single-layer perceptron can learn any Boolean function.” Only linearly separable ones (AND, OR, NAND, NOR, NOT), not XOR or XNOR. And “the perceptron converges on any data” is false: on non-separable data it never

stops changing; use the averaged perceptron or logistic regression.

Logistic regression = linear score + sigmoid + Bernoulli likelihood: convex loss, gradient $(p-y)x$, linear boundary, probabilities you threshold by cost.

§5.9 A few problems to think about

Problem 5A (Neuron output and log-loss). A logistic model has $w = (0.5, -1.0)$, $b = 0.25$. For $x = (2, 1)$ with true label 1, compute z , p , the predicted class at threshold 0.5, and the log-loss (natural log).

[hint · solution](#)

Problem 5B (Odds ratio). In a churn model the coefficient on “number of complaints” is 0.4. A customer with 2 complaints has churn probability 0.2. What is the probability with 5 complaints, other features equal?

[hint · solution](#)

Problem 5C (Gradient step). One feature plus intercept, $w = (0, 1)$ (intercept first). Data: $x = 1, y = 0$ and $x = -1, y = 1$. Compute the average log-loss gradient and the weights after one step with $\eta = 0.5$.

[hint · solution](#)

Problem 5D (Which statements are true?). (a) Logistic regression’s decision boundary is linear in the features. (b) Log-loss for logistic regression is convex in the weights. (c) Logistic regression requires features to be normally distributed. (d) With perfectly separable data and no regularisation, the weights diverge. (e) Doubling every feature value halves the fitted coefficients (no regularisation).

[hint · solution](#)

Problem 5E (Softmax gradient). Show that for $p = \text{softmax}(z)$ and one-hot y , $\partial(-\sum_k y_k \log p_k) / \partial z_j = p_j - y_j$.

[hint · solution](#)

Problem 5F (Cost-based threshold). Missing a fraudulent transaction costs Rs 5000; blocking a genuine one costs Rs 100 in customer friction. Above what fraud probability should a transaction be blocked? Suppose instead the model was trained on 1% fraud with balanced class weights (`class_weight="balanced"`). Can you use this threshold directly?

[hint · solution](#)

6 Generative Classifiers

A discriminative model learns the boundary; a generative model learns what each class looks like and lets Bayes' rule draw the boundary. Placement papers test the generative family through naive Bayes numericals (a spam posterior with Laplace smoothing) and through “which statement about naive Bayes is false”. LDA appears less often but is the cleanest bridge to logistic regression.

§6.1 Naive Bayes

Prototypical example for this section: Is “free meeting money” spam, given five labelled emails?

§6.1.i The model

By Bayes' rule $\mathbb{P}(y = c \mid \mathbf{x}) \propto \mathbb{P}(y = c)p(\mathbf{x} \mid y = c)$. Modelling $p(\mathbf{x} \mid y)$ for a d -dimensional $\mathbf{x}$ is hard; **naive Bayes** assumes the features are *conditionally independent given the class*:

$$p(\mathbf{x} \mid y = c) = \prod_{j=1}^d p(x_j \mid y = c) \quad \implies \quad \hat{y} = \arg \max_c \left[\log \pi_c + \sum_j \log p(x_j \mid c) \right].$$

Each factor is a one-dimensional estimate (a count ratio, or a mean and variance), so training is a single pass over the data, $O(nd)$, and the model works with tiny datasets and huge vocabularies. Always compute in log space: a product of a few hundred word probabilities underflows to 0.0.

The variants differ only in $p(x_j \mid c)$:

variant	feature type	$p(x_j \mid c)$
Multinomial	word counts	$\theta_{cj}^{x_j}$, θ_{cj} = share of class- c tokens that are word j
Bernoulli	word present/absent	$\theta_{cj}^{x_j} (1 - \theta_{cj})^{1-x_j}$ (absence counts as evidence)
Gaussian	continuous	$\mathcal{N}(x_j; \mu_{cj}, \sigma_{cj}^2)$
Categorical	categories	table of $\mathbb{P}(x_j = v \mid c)$

§6.2 Laplace smoothing

Prototypical example for this section: The word “money” never appeared in a legitimate email; should a single occurrence make “ham” impossible?

The MLE $\hat{\theta}_{cj} = N_{cj}/N_c$ (count of word j in class c over total tokens in class c) is zero for any word unseen in that class, and one zero factor kills the whole product. **Additive (Laplace) smoothing** adds α pseudo-counts to every word:

$$\hat{\theta}_{cj} = \frac{N_{cj} + \alpha}{N_c + \alpha V},$$

with V the vocabulary size; $\alpha = 1$ is Laplace, $0 < \alpha < 1$ is Lidstone. It is exactly the MAP estimate under a symmetric Dirichlet prior (the Beta-prior coin of [Chapter 2](#), generalised).

Example 6.2.1 (Spam posterior by hand)

Training emails (spam: “free offer win money”, “win free prize now”; ham: “meeting schedule today”, “project meeting notes”, “free lunch meeting today”). Vocabulary size $V = 12$; total tokens: ham 10, spam 8. Priors $\pi_{\text{spam}} = 0.4$, $\pi_{\text{ham}} = 0.6$. Smoothed likelihoods ($\alpha = 1$):

word	count in ham	$\hat{\theta}_{\text{ham}}$	count in spam	$\hat{\theta}_{\text{spam}}$
free	1	$2/22 = 0.0909$	2	$3/20 = 0.1500$
meeting	3	$4/22 = 0.1818$	0	$1/20 = 0.0500$
money	0	$1/22 = 0.0455$	1	$2/20 = 0.1000$

For “free meeting money”: spam score $0.4 \times 0.15 \times 0.05 \times 0.10 = 0.0003000$, ham score $0.6 \times 0.0909 \times 0.1818 \times 0.0455 = 0.0004508$, so $\mathbb{P}(\text{spam} \mid \text{doc}) = 0.3996$: ham, narrowly. This matches `MultinomialNB(alpha=1)` exactly. Without smoothing, “money” has zero probability under ham and the ham score is 0.0, so the email would be called spam on the strength of one word. The Bernoulli variant, which also uses the *absence* of the other nine words, gives $\mathbb{P}(\text{spam}) = 0.2543$.

Remark 6.2.2 — Trap. The denominator adds αV , not α and not $\alpha \cdot$ (number of classes): every word in the vocabulary gets a pseudo-count, so the probabilities still sum to one. For Bernoulli NB it is $(N_{cj} + \alpha)/(N_c + 2\alpha)$ with N_c the number of *documents* in class c , because each word has two outcomes.

§6.2.i Gaussian naive Bayes

For continuous features, estimate a mean and variance per feature per class.

Example 6.2.3 (Height and weight)

Four “male” rows with means (5.83, 176) and variances (0.02188, 92.19); four “female” rows with means (5.4, 132) and variances (0.065, 418.8) (MLE variances, divide by n). For a query (5.8, 160) multiply the two univariate normal densities per class and the equal priors: $\mathbb{P}(\text{male} \mid \mathbf{x}) = 0.8796$, matching `GaussianNB`.

§6.3 Limitations, and why it still works

Prototypical example for this section: Duplicate a feature ten times and watch naive Bayes become 95% sure.

Example 6.3.1 (Double counting)

One feature x , classes $\mathcal{N}(0, 1)$ and $\mathcal{N}(1, 1)$, equal priors, observation $x = 0.8$. The correct posterior of class 1 is 0.5744. If the same feature is copied k times, naive

Bayes multiplies the same evidence k times:

copies k	NB posterior of class 1
1	0.5744
2	0.6457
5	0.8176
10	0.9526

Correlated features (“free”, “free!!!”, “FREE”) behave the same way.

The standard list of limitations, each a common MCQ option:

- The independence assumption is almost always false; correlated features are double-counted, so the *probabilities* are over-confident (pushed towards 0 and 1) and poorly calibrated.
- Zero-frequency problem without smoothing.
- Gaussian NB assumes each feature is unimodal and normal within a class; a skewed or bimodal feature needs a transform or binning.
- It cannot learn interactions: a rule like “spam if (A and B) but not A alone” is invisible to a model that factorises over features.
- It is a high-bias, low-variance learner: excellent with little data, overtaken by logistic regression as data grow.

Why it still classifies well: classification needs only the *argmax* to be right, not the probability. Over-confidence that pushes the right class from 0.6 to 0.99 costs nothing in accuracy. This is why naive Bayes remains a strong text baseline.

Remark 6.3.2 — Trap. “Naive Bayes assumes features are independent.” Precisely: *conditionally* independent given the class. Words like “machine” and “learning” are strongly dependent overall (both frequent in ML documents), which naive Bayes allows; it assumes that *within* the ML class one does not inform the other. And “naive Bayes requires features to be normally distributed” is true only of Gaussian NB.

Naive Bayes is count, smooth, multiply in log space; it ranks well and calibrates badly.

§6.4 LDA and QDA

Prototypical example for this section: Two Gaussian blobs: shared covariance gives a straight boundary, separate covariances a curved one.

§6.4.i The models

Gaussian class-conditionals without the naive assumption: $\mathbf{x} \mid y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$.

- **Quadratic discriminant analysis** (QDA) estimates a separate Σ_c per class. The log-posterior contains $-\frac{1}{2}\mathbf{x}^\top \Sigma_c^{-1} \mathbf{x}$, which differs between classes, so the boundary is a quadratic surface.
- **Linear discriminant analysis** (LDA) assumes a shared Σ . The quadratic terms cancel between classes, leaving the linear discriminant

$$\delta_c(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1} \boldsymbol{\mu}_c - \frac{1}{2} \boldsymbol{\mu}_c^\top \Sigma^{-1} \boldsymbol{\mu}_c + \log \pi_c,$$

and a hyperplane boundary.

code/ml/c06_generative.py: LDA scores

```
1 def lda_scores(X, mus, Sigma, priors):
2     """Linear discriminant: delta_k(x) = x^T S^-1 mu_k - mu_k^T S^-1 mu_k / 2 + log
   ↪ pi_k."""
3     Si = np.linalg.inv(Sigma)
4     return np.column_stack([X @ Si @ m - 0.5 * m @ Si @ m + np.log(p) for m, p in
   ↪ zip(mus, priors)])
```

With MLE estimates (class means, pooled covariance divided by n) these posteriors equal scikit-learn's `LinearDiscriminantAnalysis` to 10^{-8} ; QDA with per-class MLE covariances matches `QuadraticDiscriminantAnalysis`.

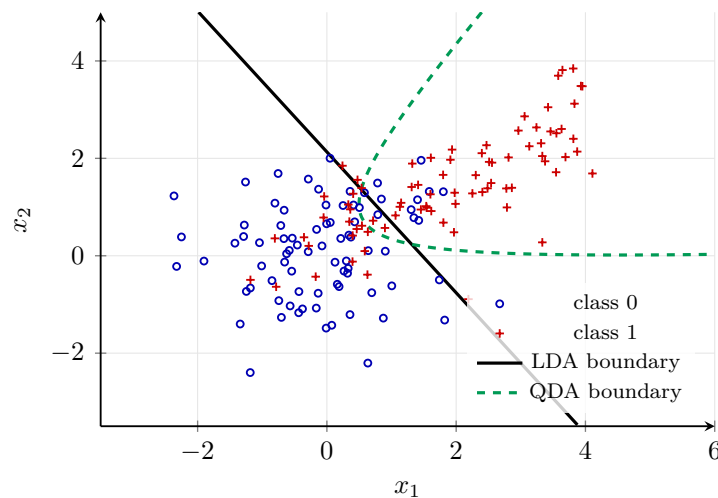


Figure 6.1: Class 0 is round, class 1 is elongated and tilted. LDA (shared covariance) draws a line; QDA follows the different shapes with a conic. Training accuracies: LDA 0.807, QDA 0.810.

§6.4.ii Parameter counts: the bias–variance view

With $K = 2$ classes in d dimensions (ignoring priors): Gaussian NB has $2d$ means and $2d$ variances; LDA has $2d$ means and $d(d+1)/2$ shared covariance entries; QDA has $2d$ means and $2 \cdot d(d+1)/2$ covariance entries. For $d = 10$ that is 40, 75 and 130; for $d = 100$, 400, 5250 and 10300. QDA's flexibility costs variance, and its per-class covariance is singular whenever a class has fewer than $d+1$ points; regularised variants shrink Σ_c towards the pooled or diagonal estimate.

§6.4.iii LDA versus logistic regression

For two classes LDA's posterior is $\sigma(\mathbf{w}^\top \mathbf{x} + b)$ — the same functional form as logistic regression. The difference is how $\mathbf{w}$ is fitted: LDA plugs in Gaussian MLEs of means and covariance (generative, uses $p(\mathbf{x})$), logistic regression maximises the conditional likelihood (discriminative, ignores $p(\mathbf{x})$). If the Gaussian assumption holds, LDA is more data-efficient; if not (heavy tails, outliers, binary features), logistic regression is more robust. LDA is also a dimensionality reduction: projecting onto the at most $K - 1$ discriminant directions that maximise between-class over within-class scatter (Fisher's criterion).

Remark 6.4.1 — Trap. LDA here is *linear discriminant analysis*, not latent Dirichlet allocation (the topic model). And “generative models are always better with little data” is a tendency, not a law: it holds when the generative assumptions are close to true.

Shared covariance $\Rightarrow$ linear boundary (LDA); separate covariances $\Rightarrow$ quadratic (QDA); diagonal covariances $\Rightarrow$ Gaussian naive Bayes.

§6.5 A few problems to think about

Problem 6A (Naive Bayes posterior). $\mathbb{P}(\text{spam}) = 0.3$. Word probabilities (already smoothed): $\mathbb{P}(\text{“offer”} \mid \text{spam}) = 0.05$, $\mathbb{P}(\text{“offer”} \mid \text{ham}) = 0.005$, $\mathbb{P}(\text{“team”} \mid \text{spam}) = 0.002$, $\mathbb{P}(\text{“team”} \mid \text{ham}) = 0.01$. Find $\mathbb{P}(\text{spam} \mid \text{“offer team”})$ with multinomial NB.

hint · solution

Problem 6B (Laplace smoothing arithmetic). In the “positive review” class there are 500 tokens and a vocabulary of $V = 1000$. The word “refund” occurs 0 times there and 12 times among 800 tokens of the negative class. Give the smoothed probabilities with $\alpha = 1$ and their ratio.

hint · solution

Problem 6C (True or false). (a) Naive Bayes can learn XOR of two binary features. (b) Doubling a feature changes naive Bayes' predicted probabilities. (c) Naive Bayes needs feature scaling. (d) LDA's boundary between two classes is linear. (e) QDA has fewer parameters than LDA.

hint · solution

Problem 6D (Gaussian NB boundary). One feature; class $A \sim \mathcal{N}(0, 1)$, class $B \sim \mathcal{N}(0, 4)$, equal priors. Where is the decision boundary, and why is it not a single threshold?

hint · solution

7 k -Nearest Neighbours and Distances

k -NN has no training step, one hyper-parameter and a one-sentence definition, which makes it a favourite for “what happens if...” questions: if $k = 1$, if $k = n$, if you forget to scale, if you add 100 useless features. Answer those well and you have also explained the curse of dimensionality, which is really a chapter about every distance-based method.

§7.1 The algorithm

Prototypical example for this section: Six labelled points, one query at $(3, 3)$, and a prediction that flips as k grows.

Definition 7.1.1. Given training points (x_i, y_i) , a distance d and an integer k , the **k -nearest neighbour** classifier predicts the majority label among the k training points closest to the query; k -NN *regression* predicts the mean (or a distance-weighted mean) of their targets.

It is a **lazy** (instance-based, non-parametric) learner: “training” stores the data, and all the cost moves to prediction — $O(nd)$ per query by brute force, plus $O(nd)$ memory. That is the opposite of most production constraints, which is why approximate search (§7.5) exists.

Example 7.1.2 (By hand)

Query $(3, 3)$; distances to the six training points:

point	label	Euclidean	Manhattan
$(1, 1)$	A	2.828	4
$(2, 1)$	A	2.236	3
$(4, 3)$	B	1.000	1
$(6, 4)$	B	3.162	4
$(3, 5)$	B	2.000	2
$(0, 3)$	A	3.000	3

Sorted by Euclidean distance the labels read B, B, A, A, A, B. So $k = 1$ predicts B, $k = 3$ (votes ABB) predicts B, and $k = 5$ (votes AAABB) predicts A — all three agree with `KNeighborsClassifier`. If the targets of the six points are 10, 12, 20, 24, 22, 14, 3-NN regression predicts $(20 + 22 + 12)/3 = 18.00$; weighting by inverse distance gives 18.68.

Remark 7.1.3 — Practicalities interviewers probe: use an odd k for binary problems to avoid ties (ties in *distance* still need a rule; scikit-learn uses the order in the training data); class imbalance biases the vote towards the majority, so weight by distance or by inverse class frequency; categorical features need a suitable distance (Hamming, or one-hot then scale).

§7.2 Distances

Prototypical example for this section: Two customers as vectors: how far apart are they, and in which sense?

A **metric** satisfies $d(\mathbf{x}, \mathbf{y}) \geq 0$ with equality iff $\mathbf{x} = \mathbf{y}$, symmetry, and the triangle inequality. For $\mathbf{a} = (1, 2, 3)$ and $\mathbf{b} = (4, 0, 3)$:

name	formula	value	use
Euclidean (ℓ_2)	$\sqrt{\sum (a_i - b_i)^2}$	3.6056	default; continuous, scaled features
Manhattan (ℓ_1)	$\sum a_i - b_i $	5.0000	grids, robust to one large difference
Chebyshev (ℓ_∞)	$\max a_i - b_i $	3.0000	worst coordinate matters
Minkowski $p = 3$	$(\sum a_i - b_i ^p)^{1/p}$	3.2711	family containing the three above
cosine distance	$1 - \frac{\mathbf{a}^\top \mathbf{b}}{\ \mathbf{a}\ \ \mathbf{b}\ }$	0.3051	text, embeddings: direction not length

Cosine similarity here is 0.6949. Cosine distance is not a metric (it fails the triangle inequality), but on unit-normalised vectors it is a monotone function of Euclidean distance: $\|\mathbf{a} - \mathbf{b}\|^2 = 2(1 - \cos \theta)$. Hamming distance counts differing positions: 101101 and 110100 differ in 3 places.

The **Mahalanobis distance** $\sqrt{(\mathbf{x} - \mathbf{y})^\top \Sigma^{-1} (\mathbf{x} - \mathbf{y})}$ measures distance in units of the data's own spread and correlation. With $\Sigma = \begin{pmatrix} 4 & 1.5 \\ 1.5 & 1 \end{pmatrix}$, the point (2, 1) is at Euclidean distance 2.2361 from the origin but Mahalanobis distance only 1.0690, because it lies along the direction in which the data naturally vary. Euclidean distance on standardised and decorrelated (whitened) features is Mahalanobis distance.

§7.3 Scaling, and choosing k

Prototypical example for this section: Income in rupees and age in years: who is the nearest neighbour?

Example 7.3.1 (Scaling decides the neighbour)

Customers 1 = (Rs 50,000, 58 yrs) and 2 = (Rs 52,000, 25 yrs); query (Rs 51,800, 57 yrs). Raw Euclidean distances are 1800.0 and 202.5: the nearest is customer 2, a 25-year-old, because a Rs 1600 income gap dwarfs a 32-year age gap. After standardising both features the nearest is customer 1. Distance-based methods (k -NN, k -means, SVM with RBF kernel, PCA) need features on comparable scales.

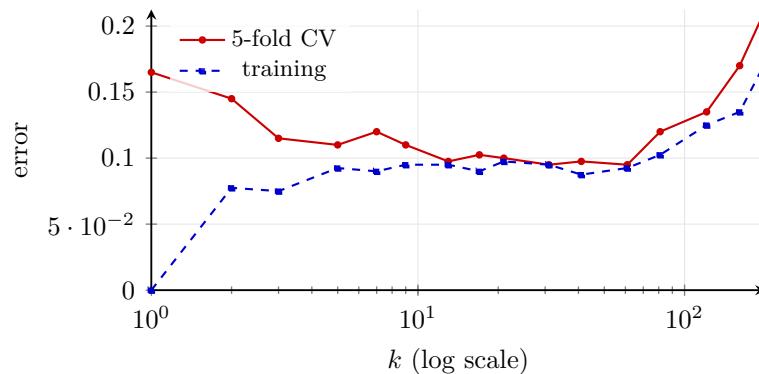
Remark 7.3.2 — Trap. “Fit the scaler on the whole dataset, then split.” The test set's mean and variance then leak into training. Fit the scaler on the training fold only, inside a pipeline (Chapter 15).

§7.3.i k as a bias–variance dial

$k = 1$: training error is exactly 0 (each point is its own neighbour), the boundary is jagged (the Voronoi diagram of the training set), variance is maximal. $k = n$: every query gets the majority class, a constant predictor with maximal bias. Effective number of parameters is roughly n/k . A classical result (Cover and Hart) bounds the asymptotic error of 1-NN by twice the Bayes error.

Example 7.3.3 (Cross-validated k on two moons)

400 points, 5-fold CV error: $k = 1$ gives 0.165 (training error 0.000); the best is $k = 31$ with 0.095; $k = 201$ gives 0.212.



§7.4 The curse of dimensionality

Prototypical example for this section: In 1000 dimensions your nearest and farthest neighbours are almost equally far away.

Three facts, all simulated in `code/ml/c07_knn.py`:

1. **Distances concentrate.** For 500 uniform points in $[0, 1]^d$, the relative contrast $(d_{\max} - d_{\min})/d_{\min}$ from a random query is 34.52 in 2 dimensions and 0.101 in 1000. When every point is about equally far, “nearest” carries little information.
2. **Neighbourhoods are not local.** To capture 10% of uniform data in $[0, 1]^d$ with a sub-cube you need edge length $0.1^{1/d}$: 0.794 of each axis’s range at $d = 10$ and 0.977 at $d = 100$. The “local” neighbourhood spans almost everything.
3. **Volume lives near the surface.** The fraction of a unit ball’s volume within 0.01 of its boundary is $1 - 0.99^d$: 0.634 at $d = 100$, 0.99996 at $d = 1000$.

The data needed to keep the same density of samples grows exponentially in d . Irrelevant features are the practical killer: adding uniform noise dimensions to the two-moons problem drops 7-NN test accuracy from 0.972 to 0.630 with 100 noise features.

Why k -NN still works on images or embeddings: real data lie near a low-dimensional manifold (the *intrinsic* dimension is small), and learned embeddings are built so that distance means something. Remedies when it does not: feature selection, PCA or learned embeddings, metric learning.

Remark 7.4.1 — Trap. “The curse of dimensionality means more features always hurt.” More *informative* features help any model; the curse bites distance-based and density-based methods through irrelevant or redundant dimensions, and more generally through the sample size needed to cover the space. Linear models with regularisation and tree ensembles suffer far less from irrelevant features.

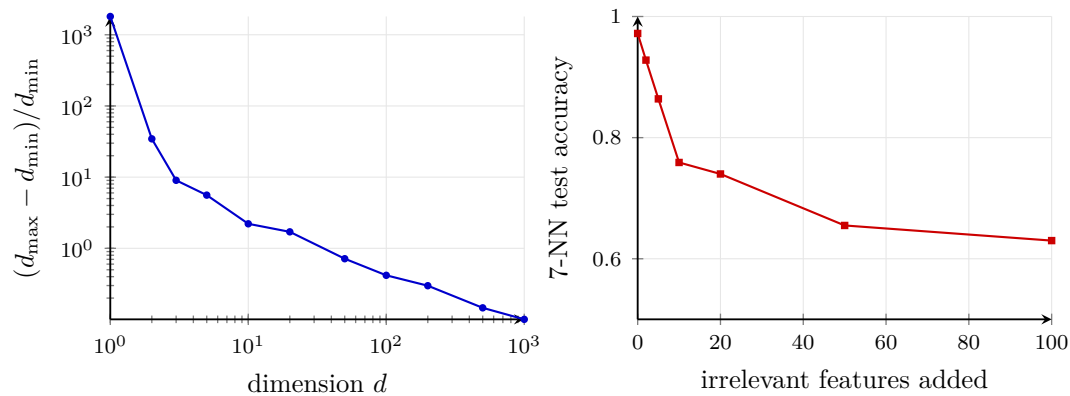


Figure 7.1: Left: distance contrast collapses with dimension. Right: noise features drown the two informative ones.

§7.5 KD-trees and approximate search

Prototypical example for this section: Find the nearest of 4000 points without measuring all 4000 distances.

A **KD-tree** splits space recursively on one coordinate at a time at the median. To find a nearest neighbour, descend to the query’s leaf, then backtrack, exploring the other side of a split only if the splitting plane is closer than the best distance so far. In low dimensions this prunes almost everything; in high dimensions the “ball” around the query crosses nearly every splitting plane and pruning fails. Our 30-line KD-tree (which agrees with scikit-learn’s `KDTree` on every query) visits on average:

d	nodes visited (of 4000)	fraction
2	18	0.5%
5	98	2.5%
10	2104	52.6%
20	3997	99.9%

Beyond about 10–20 dimensions exact trees degrade to brute force. Production systems (embedding search for RAG, recommendation, image retrieval) use **approximate nearest neighbour** (ANN) methods that trade a little recall for orders of magnitude in speed: locality-sensitive hashing (random projections whose hash collisions favour nearby points), graph-based indices such as HNSW (navigable small-world graphs, greedy search from coarse to fine layers), and inverted-file indices with product quantisation (cluster the space, then compress vectors into short codes). Ball trees replace axis-aligned splits by nested hyperspheres and handle non-Euclidean metrics better.

k -NN is only as good as its distance: scale features, drop irrelevant ones, pick k by cross-validation, and expect exact search to fail in high dimension.

§7.6 A few problems to think about

Problem 7A (Vote by hand). Training points: (0,0):red, (1,0):red, (0,2):blue, (2,2):blue, (3,0):blue. Classify (1,1) with $k = 1$ and $k = 3$ using Manhattan distance, listing ties.

hint · solution



Problem 7B (Edge length). Data are uniform in $[0, 1]^{20}$. What edge length must a cube around a query have to contain 1% of the data on average? 50%?

[hint](#) · [solution](#)

Problem 7C (True or false). (a) k -NN with $k = 1$ has zero training error (no duplicate points with different labels). (b) Increasing k increases variance. (c) k -NN needs feature scaling. (d) k -NN training is $O(1)$ apart from storing the data. (e) k -NN cannot do regression.

[hint](#) · [solution](#)

Problem 7D (Cosine versus Euclidean). Documents as term-count vectors: $A = (2, 0, 1)$, $B = (20, 0, 10)$, $C = (1, 1, 1)$. Which is closer to A under Euclidean distance, and under cosine distance? Which is more sensible for documents?

[hint](#) · [solution](#)

8 Support Vector Machines

SVMs are where interviewers test whether you understand optimisation rather than recite it: why maximise the margin, where the dual comes from, why only some points matter, what the kernel trick actually saves, and what C and γ do. OA papers ask the conceptual layer (“which loss is convex”, “what are support vectors”, “effect of C ”); interviews ask the derivation.

§8.1 Margins

Prototypical example for this section: Three negatives near the origin, three positives near (2,2): of all separating lines, pick the one farthest from both groups.

For a linear classifier $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ with labels $y \in \{-1, +1\}$, the signed distance of $\mathbf{x}_i$ from the hyperplane $\{\mathbf{w}^\top \mathbf{x} + b = 0\}$ is $y_i(\mathbf{w}^\top \mathbf{x}_i + b) / \|\mathbf{w}\|$, its **geometric margin**. Many hyperplanes separate separable data; the perceptron returns whichever it hits first. The SVM picks the one maximising the smallest margin, on the argument that a boundary far from all training points is robust to small perturbations of them — the street analogy: build the widest road that keeps the two neighbourhoods on opposite sides.

Because $(\mathbf{w}, b)$ and $(c\mathbf{w}, cb)$ define the same hyperplane, fix the scale by requiring $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$ with equality for the closest points. The margin is then $1 / \|\mathbf{w}\|$ on each side and the street has width $2 / \|\mathbf{w}\|$. Maximising width is minimising $\|\mathbf{w}\|$.

§8.2 Hard and soft margin

Prototypical example for this section: Hard margin: solve the six-point example by hand. Soft margin: let C decide how many points may sit inside the street.

Definition 8.2.1 (Hard-margin SVM).

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i.$$

A convex quadratic programme with linear constraints: one global optimum, no local-minimum worries.

Example 8.2.2 (Six points by hand)

Negatives $(0, 0)$, $(1, 0)$, $(0, 1)$; positives $(2, 2)$, $(3, 3)$, $(2, 3)$. By symmetry $\mathbf{w} = (a, a)$. The closest negatives are $(1, 0)$ and $(0, 1)$, the closest positive is $(2, 2)$, so $a + b = -1$ and $4a + b = 1$, giving $a = 2/3$, $b = -5/3$. Check the rest: $(0, 0)$ gives $-5/3 \leq -1$, $(2, 3)$ gives $5/3 \geq 1$. Street width $2 / \|\mathbf{w}\| = 3 / \sqrt{2} = 2.1213$. Support vectors: $(0, 1)$, $(1, 0)$, $(2, 2)$. `SVC(kernel="linear", C=1e8)` returns the same $\mathbf{w}$, b and support vectors.

§8.2.i Soft margin and the hinge loss

Real data overlap. Introduce slacks $\xi_i \geq 0$ that let point i violate the margin:

Definition 8.2.3 (Soft-margin SVM).

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

At the optimum $\xi_i = \max(0, 1 - y_i f(\mathbf{x}_i))$, the **hinge loss**, so the problem is the unconstrained, convex

$$\min_{\mathbf{w}, b} \sum_i \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)) + \frac{1}{2C} \|\mathbf{w}\|^2 :$$

hinge loss plus ℓ_2 regularisation with $\lambda = 1/(2C)$ — structurally ridge-penalised logistic regression with the logistic loss replaced by the hinge. $\xi_i = 0$: outside the street; $0 < \xi_i \leq 1$: inside the street but correctly classified; $\xi_i > 1$: misclassified. Solving this primal QP directly on 80 overlapping points reproduces `SVC(kernel="linear", C=1)` to 2×10^{-3} , and the optimal slacks equal the hinge losses; 14 points violate the margin and 5 are misclassified.

Example 8.2.4 (What C does)

Same 80 points, linear kernel; test set of 2000:

C	street width $2/\ \mathbf{w}\ $	#support vectors	train acc	test acc
0.01	3.86	54	0.912	0.939
0.1	2.19	25	0.900	0.940
1	1.16	17	0.938	0.927
10	0.92	14	0.925	0.932
100	0.92	14	0.925	0.932

Small C : violations are cheap, the street is wide, many points are support vectors, the model is heavily regularised (high bias). Large C : violations are expensive, the street narrows to fit the training data (high variance); as $C \rightarrow \infty$ we approach the hard margin.

Remark 8.2.5 — Trap. C is the *inverse* of regularisation strength (as in scikit-learn’s logistic regression). “Increasing C increases regularisation” is the wrong option. Also, support vectors are not just the points on the margin: in the soft-margin SVM every point with $y_i f(\mathbf{x}_i) \leq 1$ — on the margin, inside it, or misclassified — is a support vector.

§8.3 Duality

Prototypical example for this section: The six-point example has dual weights $\alpha = (2/9, 2/9, 4/9)$ on its three support vectors and 0 everywhere else.

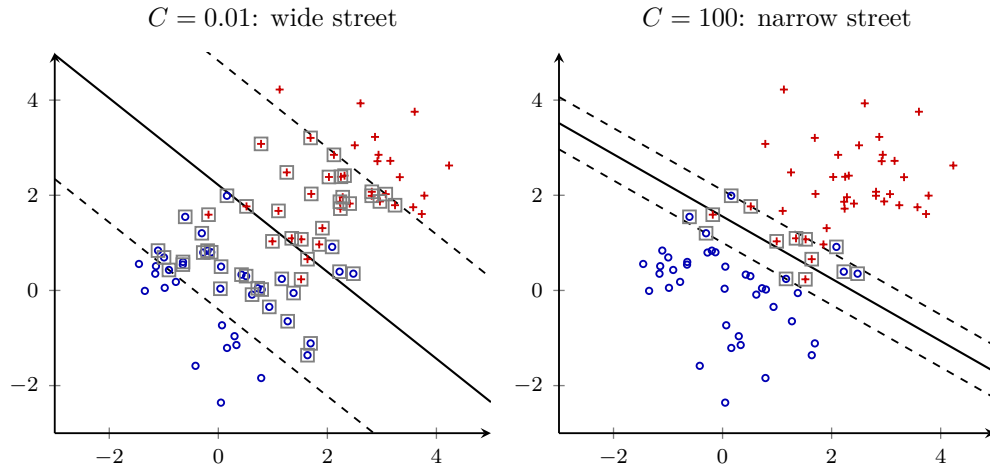


Figure 8.1: Soft-margin linear SVMs; squares mark support vectors, dashed lines are $f(\mathbf{x}) = \pm 1$.

§8.3.i Deriving the dual

Lagrangian of the soft-margin problem, with $\alpha_i \geq 0$ for the margin constraints and $\mu_i \geq 0$ for $\xi_i \geq 0$:

$$\mathcal{L} = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum \xi_i - \sum \alpha_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1 + \xi_i] - \sum \mu_i \xi_i.$$

Stationarity:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i, \quad \frac{\partial \mathcal{L}}{\partial b} = 0 \Rightarrow \sum_i \alpha_i y_i = 0, \quad \frac{\partial \mathcal{L}}{\partial \xi_i} = 0 \Rightarrow \alpha_i = C - \mu_i \leq C.$$

Substituting back eliminates $\mathbf{w}$, b and ξ :

Theorem 8.3.1 (SVM dual)

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0,$$

and the classifier is $f(\mathbf{x}) = \sum_i \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b$.

Strong duality holds (convex objective, affine constraints), so solving the dual solves the primal.

§8.3.ii Complementary slackness explains support vectors

KKT says $\alpha_i [y_i f(\mathbf{x}_i) - 1 + \xi_i] = 0$ and $\mu_i \xi_i = (C - \alpha_i) \xi_i = 0$. Hence:

- $\alpha_i = 0$: $y_i f(\mathbf{x}_i) \geq 1$, the point is outside the street and does not affect the solution at all. Delete it and retrain: same classifier.
- $0 < \alpha_i < C$: $\xi_i = 0$ and $y_i f(\mathbf{x}_i) = 1$, exactly on the margin. Use these to compute b .
- $\alpha_i = C$: inside the street or misclassified ($\xi_i \geq 0$).

For the six points, $\mathbf{w} = \sum \alpha_i y_i \mathbf{x}_i$ with $\alpha = 2/9$ on $(1, 0)$ and $(0, 1)$ and $4/9$ on $(2, 2)$ gives $\frac{4}{9}(2, 2) - \frac{2}{9}(1, 0) - \frac{2}{9}(0, 1) = (\frac{2}{3}, \frac{2}{3})$. A generic QP solver on the dual (below) returns these α 's, matching `SVC.dual_coef_`.

code/ml/c08_svm.py: the dual as a QP

```

1 def svm_dual(K, y, C=np.inf):
2     """max sum(a) - 1/2 a^T (yy^T * K) a s.t. 0 <= a_i <= C, sum a_i y_i = 0."""
3     n = len(y)
4     Q = (y[:, None] * y[None, :]) * K
5     obj = lambda a: 0.5 * a @ Q @ a - a.sum()
6     jac = lambda a: Q @ a - 1
7     bounds = [(0, None if np.isinf(C) else C)] * n
8     res = minimize(obj, np.zeros(n), jac=jac, bounds=bounds, method="SLSQP",
9                   constraints=[{"type": "eq", "fun": lambda a: a @ y, "jac":
10                               lambda a: y}],
11                   options={"ftol": 1e-12, "maxiter": 1000})
12     return res.x

```

Remark 8.3.2 — Why bother with the dual? (i) The data enter only through inner products $\mathbf{x}_i^\top \mathbf{x}_j$, which is what makes kernels possible. (ii) The solution is sparse in α . (iii) It has one variable per *example* instead of per feature, which helps when $d \gg n$. For large n with linear kernels, primal solvers (`LinearSVC`, SGD on the hinge loss) are faster.

§8.4 Kernels

Prototypical example for this section: Points inside a circle versus outside: no line separates them in $\mathbb{R}^2$, but adding $x_1^2 + x_2^2$ as a third coordinate makes them linearly separable.

§8.4.i The trick

Map inputs through a feature map ϕ and run the linear SVM on $\phi(\mathbf{x})$. The dual needs only $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$; if a function $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$ computes that inner product directly, we never form ϕ . The prediction is $f(\mathbf{x}) = \sum_{i \in \text{SV}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$.

Example 8.4.1 (The quadratic kernel unpacked)

For $\mathbf{x}, \mathbf{z} \in \mathbb{R}^2$, $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + 1)^2$ equals $\phi(\mathbf{x})^\top \phi(\mathbf{z})$ with $\phi(\mathbf{x}) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)$. For $\mathbf{x} = (1, 2)$, $\mathbf{z} = (3, 1)$: $\mathbf{x}^\top \mathbf{z} = 5$ and $K = (5 + 1)^2 = 36$; explicitly $\phi(\mathbf{x}) = (1, 1.41, 2.83, 1, 4, 2.83)$ and the 6-dimensional dot product is also 36. One 2-dimensional dot product replaced a 6-dimensional one; for $d = 100$ and degree 3 the explicit map has $\binom{103}{3} = 176851$ coordinates, while the kernel still costs one 100-dimensional dot product.

Common kernels:

kernel	$K(\mathbf{x}, \mathbf{z})$	feature space
linear	$\mathbf{x}^\top \mathbf{z}$	$\mathbb{R}^d$
polynomial	$(\gamma \mathbf{x}^\top \mathbf{z} + r)^p$	all monomials up to degree p
RBF (Gaussian)	$\exp(-\gamma \ \mathbf{x} - \mathbf{z}\ ^2)$	infinite-dimensional
sigmoid	$\tanh(\gamma \mathbf{x}^\top \mathbf{z} + r)$	not always a valid kernel

For the RBF kernel with $\gamma = 0.5$ and the same $\mathbf{x}, \mathbf{z}$: $\|\mathbf{x} - \mathbf{z}\|^2 = 5$ and $K = e^{-2.5} = 0.0821$. The RBF kernel is a similarity: 1 for identical points, decaying with distance at a rate set by γ (so it needs scaled features).

Theorem 8.4.2 (Mercer, informally)

K is a valid kernel (an inner product in some feature space) iff every Gram matrix $[K(\mathbf{x}_i, \mathbf{x}_j)]_{ij}$ it produces is symmetric positive semi-definite.

Numerically: the RBF Gram matrix of 30 random points has smallest eigenvalue $8.8 \times 10^{-6} \geq 0$, while the “negative distance” $-\|\mathbf{x} - \mathbf{z}\|$ yields a smallest eigenvalue of -48.16 — not a kernel, and the dual stops being convex.

§8.4.ii γ and C for the RBF kernel

γ is an inverse length scale: large γ makes each support vector’s influence very local, so the boundary can wrap around individual points (low bias, high variance); small γ gives smooth, almost linear boundaries.

	γ	#support vectors	train acc	test acc
Example 8.4.3 (Two moons, $C = 1$)	0.1	92	0.830	0.855
	1	63	0.940	0.931
	10	94	0.950	0.931
	100	183	0.990	0.889

At $\gamma = 100$ nearly every training point is a support vector and training accuracy is near perfect while test accuracy drops: memorisation. On concentric circles a linear SVM scores 0.583 (chance), the RBF SVM 1.000, and a *linear* SVM on the lifted features $(x_1, x_2, x_1^2 + x_2^2)$ also 1.000 — the kernel trick made explicit.

Remark 8.4.4 — Trap. “Kernels map data to a higher-dimensional space, so kernel SVMs are slow because of the dimension.” The dimension never appears; the cost is in n : the Gram matrix is $n \times n$ and training is roughly between $O(n^2)$ and $O(n^3)$, which is why kernel SVMs are rarely used beyond a few hundred thousand points. Prediction costs one kernel evaluation per support vector.

§8.5 SVR and SMO

Prototypical example for this section: Fit $\sin x$ with a tube of half-width ε in which errors are free.

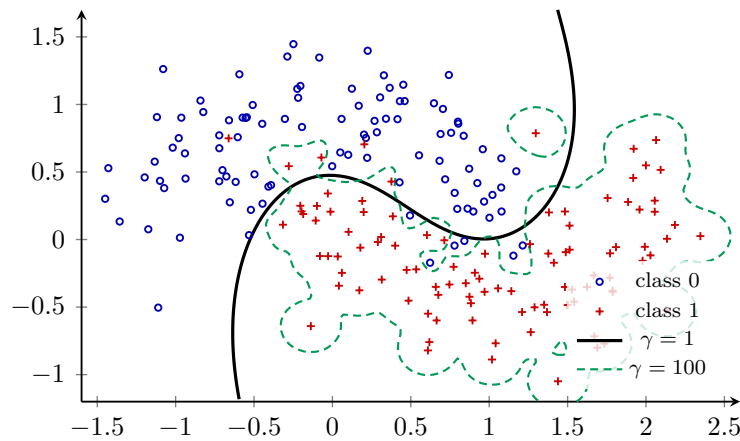


Figure 8.2: RBF-SVM decision boundaries on two moons: $\gamma = 1$ is smooth; $\gamma = 100$ draws islands around individual points.

§8.5.i Support vector regression

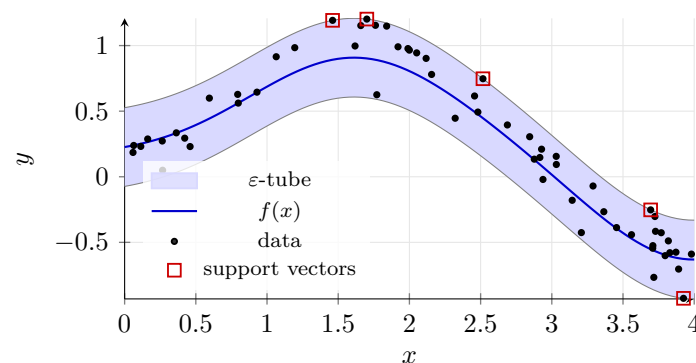
Replace the hinge by the **ϵ -insensitive loss** $\max(0, |y - f(\mathbf{x})| - \epsilon)$: errors smaller than ϵ cost nothing, larger ones cost linearly. Points strictly inside the tube have $\alpha_i = 0$; only points on or outside it are support vectors.

Example 8.5.1 (Width of the tube)

RBF SVR ($C = 10$, $\gamma = 1$) on 60 noisy samples of $\sin x$:

ϵ	#support vectors	training MSE
0.01	54	0.0174
0.1	24	0.0164
0.3	5	0.0247
0.6	3	0.0886

A wider tube means a sparser, smoother model and a larger training error.



§8.5.ii SMO in one paragraph

The dual has a box constraint per variable and one equality $\sum \alpha_i y_i = 0$, so you cannot change a single α_i without breaking the equality. **Sequential minimal optimisation** (Platt) repeatedly picks a *pair* (α_i, α_j) violating KKT, solves the two-variable sub-problem

in closed form (a one-dimensional quadratic along the line $\alpha_i y_i + \alpha_j y_j = \text{const}$, then clipped to the box $[0, C]$ — the KKT clipping of [Chapter 2](#)), updates b and the cached errors, and repeats until all points satisfy KKT within a tolerance. LIBSVM, behind scikit-learn's SVC, is an SMO-type solver with clever pair selection and kernel caching.

§8.5.iii Probabilities

SVMs output scores $f(\mathbf{x})$, not probabilities. **Platt scaling** fits a one-dimensional logistic regression $\sigma(Af(\mathbf{x}) + B)$ on held-out scores (scikit-learn's `probability=True` does this with internal cross-validation, which is slow and can disagree with `predict`).

§8.6 Convex surrogate losses

Prototypical example for this section: OA: “Which of these losses is convex in the model output?”

With margin $m = yf(\mathbf{x})$ ([Figure 2.3](#)):

loss	formula	convex?	used by
0-1	$\mathbb{I}[m \leq 0]$	no	the true objective, never optimised directly
hinge	$\max(0, 1 - m)$	yes (kink at 1)	SVM
squared hinge	$\max(0, 1 - m)^2$	yes, smooth	LinearSVC default
logistic	$\log(1 + e^{-m})$	yes, smooth	logistic regression
exponential	e^{-m}	yes	AdaBoost
perceptron	$\max(0, -m)$	yes	perceptron
ε -insensitive	$\max(0, y - f - \varepsilon)$	yes	SVR
ramp / truncated hinge	$\min(1, \max(0, 1 - m))$	no	robust SVM variants

Convex in the *output* f ; with a linear f also convex in $(\mathbf{w}, b)$. Hinge and logistic loss behave alike for misclassified points (linear growth); hinge is exactly zero once $m \geq 1$, which is what produces sparsity in support vectors, while logistic loss keeps pushing all points and yields probabilities.

Remark 8.6.1 — Trap. “The hinge loss is not convex because it is not differentiable.” Convexity does not need differentiability: $|x|$ and $\max(0, 1 - m)$ are convex. Use subgradients.

SVM = hinge loss + ℓ_2 penalty; the dual depends on data only through inner products, so kernels give non-linear boundaries, and complementary slackness makes the solution depend only on support vectors.

§8.7 A few problems to think about

Problem 8A (Margin from weights). A trained linear SVM has $\mathbf{w} = (3, 4)$, $b = -2$. What is the width of its street? On which side, and how far from the hyperplane, is $\mathbf{x} = (1, 1)$?

[hint · solution](#)

Problem 8B (Kernel value). Compute $K(\mathbf{x}, \mathbf{z})$ for $\mathbf{x} = (1, 0, 2)$, $\mathbf{z} = (2, 1, 1)$ under (a) the linear kernel, (b) $(\mathbf{x}^\top \mathbf{z} + 1)^3$, (c) the RBF kernel with $\gamma = 0.1$.

[hint · solution](#)

Problem 8C (Support vectors and deletion). After training a soft-margin SVM, you delete all training points with $\alpha_i = 0$ and retrain with the same C . What happens? What if you delete a point with $\alpha_i = C$?

[hint](#) · [solution](#)

Problem 8D (Which are true?). (a) The SVM objective is convex in $(\mathbf{w}, b)$. (b) Increasing C widens the margin. (c) An RBF SVM with very large γ behaves like 1-nearest-neighbour on the support vectors. (d) The number of support vectors is an upper bound on leave-one-out errors. (e) SVMs need feature scaling.

[hint](#) · [solution](#)

Problem 8E (Hinge-loss subgradient step). Primal objective $\frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum \max(0, 1 - y_i \mathbf{w}^\top \mathbf{x}_i)$ (no bias), $\lambda = 0.1$, $\mathbf{w} = (0.5, -0.5)$. One example $\mathbf{x} = (1, 2)$, $y = +1$. Compute the stochastic subgradient and the update with step size 0.1 (Pegasos-style, $n = 1$).

[hint](#) · [solution](#)

9 Decision Trees

Trees are the most interview-friendly model: you can compute a split by hand in two minutes, draw the result, and explain it to a non-technical manager. They are also the building block of the models that win most tabular competitions ([Chapter 10](#)), so their quirks — greediness, instability, bias towards many-valued features — matter far beyond this chapter.

§9.1 Impurity

Prototypical example for this section: A node with 8 candidates who accepted an offer and 6 who declined: how “mixed” is it?

A classification tree splits so that child nodes are purer than the parent. With class proportions $p_1, \dots, p_K$ in a node:

Definition 9.1.1.

$$\begin{aligned} \text{entropy } H &= - \sum_k p_k \log_2 p_k, & \text{Gini } G &= 1 - \sum_k p_k^2 = \sum_k p_k(1 - p_k), \\ \text{misclassification error} &= 1 - \max_k p_k. \end{aligned}$$

All three are 0 for a pure node and maximal for a uniform one (binary: entropy 1 bit, Gini 0.5). Gini is the probability that two labels drawn at random from the node disagree; entropy is the expected number of bits to encode a label. They almost always choose the same split; Gini avoids a logarithm and is scikit-learn’s default.

Definition 9.1.2. The **information gain** of a split of node S into children S_v is

$$\text{IG} = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v),$$

the drop from parent impurity to the size-weighted child impurity. The same construction with Gini gives the *Gini decrease*.

Remark 9.1.3 — Trap. Misclassification error is a poor splitting criterion: it is not strictly concave, so many splits that clearly improve purity leave it unchanged (e.g. $(40, 40) \rightarrow (30, 10) + (10, 30)$ and $(40, 40) \rightarrow (20, 40) + (20, 0)$ both have error 0.25, but the second creates a pure node). Use it for pruning, not growing.

§9.2 A split by hand

Prototypical example for this section: Which of CTC, city and role best predicts whether a candidate accepts an offer?

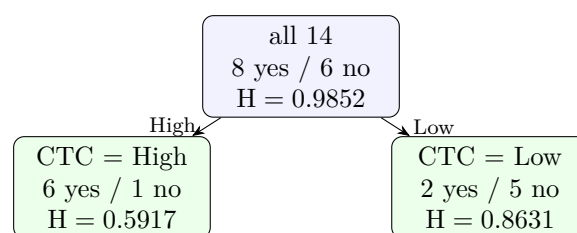
#	CTC	City	Role	Accepted?
1	High	Metro	Dev	yes
2	High	Metro	Data	yes
3	High	Tier2	Dev	yes
4	High	Tier2	Ops	no
5	High	Metro	Ops	yes
6	High	Tier2	Data	yes
7	Low	Metro	Dev	yes
8	Low	Metro	Data	yes
9	Low	Tier2	Dev	no
10	Low	Tier2	Data	no
11	Low	Metro	Ops	no
12	Low	Tier2	Ops	no
13	High	Metro	Dev	yes
14	Low	Metro	Dev	no

Example 9.2.1 (Information gain of each feature)

Parent: 8 yes, 6 no, so $H = -\frac{8}{14} \log_2 \frac{8}{14} - \frac{6}{14} \log_2 \frac{6}{14} = 0.9852$ and $Gini = 1 - (8/14)^2 - (6/14)^2 = 0.4898$.

- **CTC** (*High* : 6 + /1−; *Low* : 2 + /5−): child entropies 0.5917 and 0.8631, weighted $\frac{7}{14} \cdot 0.5917 + \frac{7}{14} \cdot 0.8631 = 0.7274$, so $IG = 0.2578$.
- **City** (*Metro* : 6 + /2−; *Tier2* : 2 + /4−): weighted child entropy 0.8571, $IG = 0.1281$.
- **Role** (*Data* : 3 + /1−; *Dev* : 4 + /2−; *Ops* : 1 + /3−): weighted child entropy 0.8571, $IG = 0.1281$.

The root splits on **CTC**. Gini decreases rank the same way (0.1633, 0.0850, 0.0850). The parent Gini and the CTC gain match scikit-learn's impurity values for a depth-1 tree.



§9.2.i The bias towards many-valued features: gain ratio

City and Role have *identical* information gain, but Role splits three ways. Taken to the extreme, a candidate-ID feature with 14 distinct values gives 14 pure leaves and the maximal gain 0.9852 while being useless for prediction. C4.5 corrects this with the **gain ratio** $IG / \text{SplitInfo}$, where $\text{SplitInfo} = -\sum_v \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$ is the entropy of the split itself. SplitInfo is 0.9852 for City, 1.5567 for Role and $\log_2 14 = 3.8074$ for the ID, giving gain ratios 0.1300, 0.0823 and 0.2588. (Gain ratio over-corrects towards near-trivial splits with tiny SplitInfo, so C4.5 only considers splits with above-average gain.)

Remark 9.2.2 (ID3, C4.5, CART) — **ID3**: categorical features, multiway splits, information gain, no pruning. **C4.5**: adds continuous thresholds, gain ratio, missing-value handling and error-based pruning. **CART** (what scikit-learn implements): strictly *binary* splits, Gini for classification and squared error for regression, cost-complexity pruning. A binary tree can still express a multiway categorical split as a sequence of binary ones.

§9.3 Growing a tree

Prototypical example for this section: Eight values of a continuous feature: only seven thresholds are worth trying.

§9.3.i Continuous features

Sort the values; only midpoints between consecutive *distinct* values can change the partition, so a feature with n distinct values has at most $n - 1$ candidate thresholds (and only those between differently-labelled neighbours can be optimal). With pre-sorting, scanning all thresholds for one feature costs $O(n)$ by updating class counts incrementally.

Example 9.3.1 (Threshold scan)

$x = (2, 3, 4.5, 5, 6.5, 7, 8, 9.5)$ with labels $(0, 0, 0, 1, 0, 1, 1, 1)$; parent Gini 0.5000.

threshold	left (positives/size)	right	weighted Gini
2.5	0/1	4/7	0.4286
3.75	0/2	4/6	0.3333
4.75	0/3	4/5	0.2000
5.75	1/4	3/4	0.3750
6.75	1/5	3/3	0.2000
7.5	2/6	2/2	0.3333
8.75	3/7	1/1	0.4286

Two thresholds tie at 0.2000; scikit-learn's stump picks $x \leq 4.75$, the first.

§9.3.ii Categorical features and missing values

A categorical feature with q levels admits $2^q - 1$ binary partitions. For binary classification (and regression) there is a shortcut: order the levels by their positive rate (mean target) and only consider the $q - 1$ splits in that order — optimal for Gini and squared error. That ordering is a form of target encoding, with the same leakage risk if done on the full data (Chapter 15). Libraries that take only numbers need one-hot or ordinal encoding first; LightGBM and CatBoost handle categories natively.

Missing values: C4.5 sends a fractional example down every branch; CART uses **surrogate splits** (another feature mimicking the primary split); XGBoost and LightGBM learn a *default direction* per split; recent scikit-learn trees accept NaN and learn which side missing values go to. With 10% of entries set to NaN in both train and test, a depth-5 tree on the breast-cancer data still reaches test accuracy 0.912.

§9.3.iii Stopping

Recursion stops when a node is pure, or hits `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`, or a minimum impurity decrease. Greedy growth is not optimal: finding the smallest consistent tree is NP-hard, and a split with zero gain (XOR!) may be needed to enable great splits below it.

§9.4 Regression trees

Prototypical example for this section: Six points that jump from about 5 to about 21: split where the jump is.

Each leaf predicts the mean of its targets; the split criterion is the reduction in the sum of squared errors (equivalently, variance reduction).

Example 9.4.1 (One regression split)

$x = 1, \dots, 6$, $y = (5, 6, 5, 20, 22, 21)$; total SSE 370.83. The best threshold is $x \leq 3.5$ with leaf means 5.33 and 21.00 and remaining SSE 2.67 — identical to `DecisionTreeRegressor(max_depth=1)`.

A regression tree is a piecewise-constant function: it cannot extrapolate beyond the range of the training targets and approximates a straight line by a staircase. That is why trees forecast trending time series badly unless you model differences, and why “trees for regression” answers should mention it.

§9.5 Pruning

Prototypical example for this section: An unpruned tree reaches 100% training accuracy; its test accuracy peaked several levels earlier.

Grown to purity, a tree memorises noise. **Pre-pruning** stops early (depth, leaf size); **post-pruning** grows fully and cuts back. CART’s **cost-complexity pruning** minimises

$$R_\alpha(T) = R(T) + \alpha |T|$$

(training error plus α per leaf). As α grows, the subtree whose removal increases error least per leaf removed (the “weakest link”) is collapsed first, giving a nested sequence of subtrees; pick α by cross-validation. Note the parallel with $E + \lambda\Omega$ in [Chapter 4](#).

Example 9.5.1 (Breast-cancer data)

The unpruned tree has depth 6, 14 leaves and test accuracy 0.939. Along the pruning path, the best test accuracy 0.943 is reached at $\alpha = 0.0029$ with 12 leaves.

§9.6 What trees are good for

Prototypical example for this section: OA: “Which of the following can decision trees be used for?”

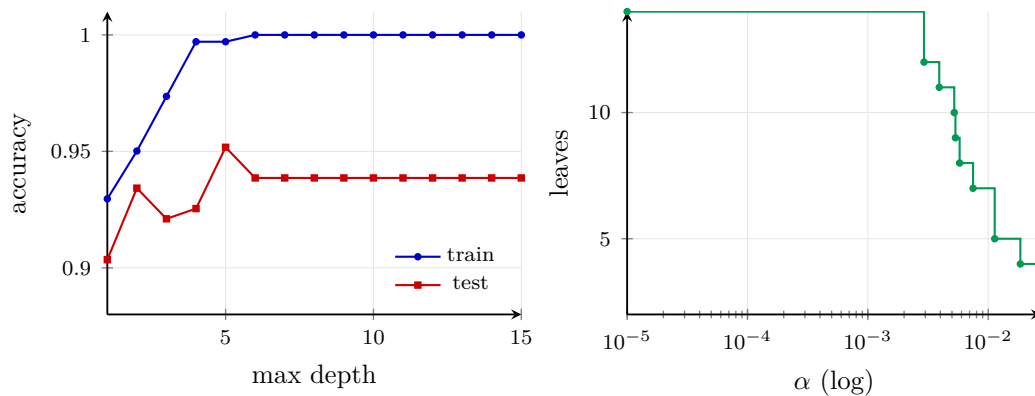


Figure 9.1: Left: training accuracy rises to 1 with depth while test accuracy plateaus. Right: cost-complexity pruning removes leaves as α grows.

Strengths. Classification and regression (and multi-output); mixed numeric and categorical features; no scaling needed — predictions are invariant to any monotone transform of a feature (log-transforming every feature leaves the breast-cancer tree’s test predictions identical, checked in the script); automatic interactions (a path is a conjunction of conditions); missing-value strategies; interpretable rules; built-in feature-importance scores; fast prediction ($O(\text{depth})$); feature selection and discretisation as by-products; base learners for ensembles.

Weaknesses. High variance: resampling the training set changes the root split — over 200 bootstrap samples of the breast-cancer data the depth-1 tree chose 6 different root features, the commonest only 0.44 of the time. Axis-aligned boundaries approximate a diagonal by a staircase (Figure 9.2). No extrapolation. Greedy and myopic. Biased towards features with many split points. Probability estimates from small leaves are coarse.

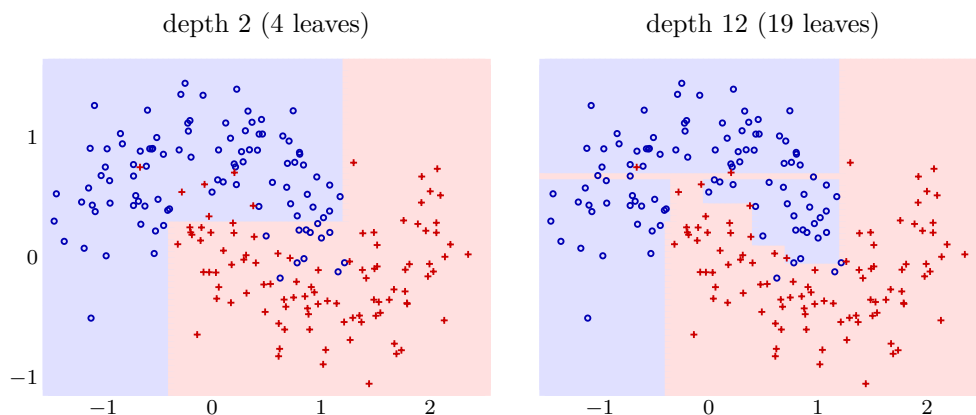


Figure 9.2: Decision regions are unions of axis-aligned rectangles. A deep tree carves small boxes around individual noisy points.

Remark 9.6.1 — Trap. “Decision trees cannot handle categorical data” (they can in principle; scikit-learn’s implementation needs encoding). “Decision trees need normalised features” (they do not). “Decision trees are linear classifiers” (no: piecewise-constant, axis-aligned). “A deeper tree always generalises better” (no:

training accuracy rises, test accuracy plateaus or falls).

Split greedily on the largest impurity drop, stop or prune before the leaves memorise noise, and expect high variance — which is exactly what ensembles fix.

§9.7 A few problems to think about

Problem 9A (Entropy and Gini). A node holds 30 samples: 10 of class A, 10 of B, 10 of C. Compute its entropy (bits) and Gini. It is split into (10, 10, 0) and (0, 0, 10); compute the information gain.

[hint · solution](#)

Problem 9B (Best split). 12 emails, 6 spam. Feature A splits them into (5 spam, 1 ham) and (1, 5). Feature B splits them into (3, 0) and (3, 6). Which split does information gain prefer? Which does Gini prefer?

[hint · solution](#)

Problem 9C (Regression split). $x = (1, 2, 3, 4)$, $y = (2, 4, 10, 12)$. Find the best single split by squared error and the two leaf predictions.

[hint · solution](#)

Problem 9D (True or false). (a) A fully grown tree on data without duplicate inputs has zero training error. (b) Standardising features changes a decision tree's predictions. (c) Trees can capture feature interactions without explicit product features. (d) A regression tree can predict a value larger than every training target. (e) Gini impurity of a binary node is at most 0.5.

[hint · solution](#)

10 Ensembles

If a placement OA has one question on classical ML beyond linear models, it is usually here: the 63.2% bootstrap number, “bagging reduces variance, boosting reduces bias”, “which is true of random forests”, or a feature-importance trap. Interviews go further: derive AdaBoost’s weight, explain gradient boosting as gradient descent in function space, and derive XGBoost’s leaf weight and split gain. All of those are below, each checked against the library that implements it.

§10.1 The bootstrap and 63.2 percent

Prototypical example for this section: Draw n rows with replacement from n ; which fraction of the original rows appears?

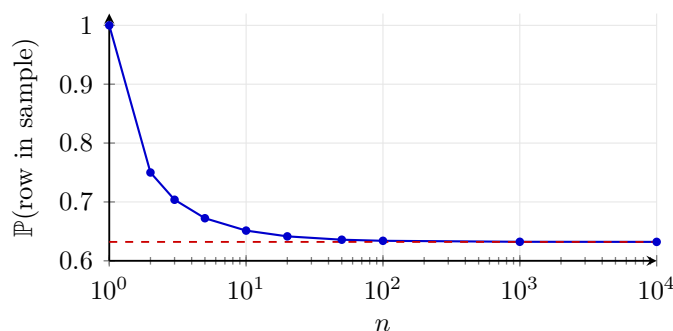
A **bootstrap sample** draws n rows uniformly *with replacement* from the n training rows. A fixed row is missed by one draw with probability $1 - \frac{1}{n}$, by all n independent draws with probability $(1 - \frac{1}{n})^n$. Hence

$$\mathbb{P}(\text{row appears}) = 1 - \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} 1 - e^{-1} = 0.6321,$$

using $\lim(1 - \frac{1}{n})^n = e^{-1}$ (take logs: $n \log(1 - \frac{1}{n}) = n(-\frac{1}{n} - \frac{1}{2n^2} - \dots) \rightarrow -1$).

Example 10.1.1 (How fast it converges)

$n = 2$: 0.7500; $n = 5$: 0.6723; $n = 10$: 0.6513; $n = 100$: 0.6340; $n = 1000$: 0.6323. Simulating 2000 bootstraps of 1000 rows gives an average unique fraction of 0.6321. So each bootstrap sample contains about 63.2% of the distinct rows, the other 36.8% are **out-of-bag** (OOB) for that sample, and the number of copies of a given row is approximately Poisson(1): it appears twice with probability $e^{-1}/2 = 0.1839$.



Remark 10.1.2 — Trap. The 63.2% is the fraction of *distinct original rows* in a bootstrap sample, not the sample size (which is n) and not “each row has a 63.2% chance per draw” (that is $1/n$). For a quick OA: “probability that a given observation is *not* in a bootstrap sample of size n ” is $(1 - 1/n)^n \approx 0.3679$.

§10.2 Bagging and random forests

Prototypical example for this section: Average 100 deep trees, each grown on its own bootstrap sample.

§10.2.i Why averaging helps

If B models each have variance σ^2 and pairwise correlation ρ , their average has variance

$$\text{Var}\left(\frac{1}{B} \sum_b f_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Averaging kills the second term as B grows, but the first, $\rho\sigma^2$, stays.

ρ	B	formula	simulated
0	10	0.100	0.099
0	100	0.010	0.010
0.3	10	0.370	0.368
0.3	100	0.307	0.310
0.7	10	0.730	0.745
0.7	100	0.703	0.703

Two consequences drive the design of every averaging ensemble: (1) the base models should have low bias and high variance (deep trees), since averaging does not touch bias; (2) the models should be as decorrelated as possible.

Definition 10.2.1. Bagging (bootstrap aggregating): fit the same learner on B bootstrap samples; average (regression) or vote/average probabilities (classification). A **random forest** bags deep trees and *additionally* considers only a random subset of `max_features` features at each split (typically $\sqrt{d}$ for classification, $d/3$ or all for regression), which decorrelates the trees.

Example 10.2.2 (Decorrelation, measured)

On a 20-feature problem, the mean pairwise correlation between the predicted probabilities of 200 trees is 0.556 for plain bagging and 0.435 for a random forest with `max_features="sqrt"`. Test accuracies: 0.893 and 0.892.

§10.2.ii Out-of-bag error

Each tree has not seen about 36.8% of the rows; predicting each row using only the trees for which it was OOB gives a nearly free validation estimate. Computing this by hand from the forest's stored bootstrap indices reproduces scikit-learn's `oob_score_` exactly (0.890); 5-fold cross-validation gives 0.889.

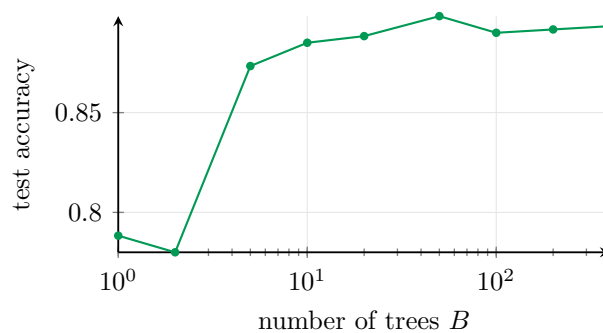
Example 10.2.3 (Bias and variance of each ensemble)

Regression on $f(\mathbf{x}) = \sin 3x_1 + 0.5x_2^2$ with noise variance 0.09, 200 training points, 20 independent training sets:

model	bias ²	variance	expected test MSE
stump	0.0879	0.0324	0.2103
deep tree	0.0062	0.0954	0.1916
bagged deep trees (100)	0.0038	0.0265	0.1203
random forest (100)	0.0071	0.0229	0.1199
boosted stumps (300, $\nu = 0.1$)	0.0025	0.0134	0.1059

A deep tree has low bias and high variance; bagging and random forests cut the variance by about a factor of four with bias roughly unchanged. A stump has high bias; boosting 300 stumps removes most of it.

Remark 10.2.4 — Trap. “Random forests reduce bias.” The standard (and correct) answer is that bagging and random forests reduce *variance*; the forest’s bias is about that of one deep tree (slightly higher, since feature subsampling weakens each tree). “Adding more trees to a random forest causes overfitting” is also false: test error stabilises as B grows; more trees cost compute, not accuracy. What *can* overfit a forest is very deep trees on noisy data with too few features per split rarely mattering — tune `min_samples_leaf` and `max_features`, not B .



§10.3 Feature importance and its traps

Prototypical example for this section: A column of random noise is ranked the second most important feature.

Mean decrease in impurity (MDI, scikit-learn’s `feature_importances_`) sums, over all splits on a feature, the weighted impurity decrease, averaged over trees and normalised to sum to 1. **Permutation importance** shuffles one feature’s column in held-out data and measures the drop in score.

Example 10.3.1 (MDI favours high-cardinality features)

Two informative features (one continuous, one binary) and two pure-noise features (one continuous, one binary); random forest, importances:

feature	MDI (training)	permutation (held-out)
informative (cont.)	0.533	0.095
informative (binary)	0.094	0.018
noise (cont.)	0.343	-0.013
noise (binary)	0.029	-0.011

MDI gives the continuous noise 0.343, far more than the genuinely informative binary feature (0.094): a continuous column offers hundreds of thresholds, and deep trees use them to fit noise in training data, collecting impurity decrease that never generalises [St+07]. Permutation importance on held-out data puts the noise at about zero. (Our hand-rolled permutation importance agrees with `sklearn.inspection.permutation_importance` to 0.01.)

Example 10.3.2 (Correlated features split the credit)

The informative continuous feature alone gets MDI 0.944; add a near-copy and the two receive 0.468 and 0.474. Each looks half as important although either alone carries all the signal. Permutation importance has the mirror problem: shuffling one copy barely hurts, because the model reads the other.

Remark 10.3.3 — Trap. Feature importance is not causal effect, and MDI is not reliable for features of different cardinality or for correlated groups. Prefer permutation importance on a validation set, drop-column importance, or SHAP values, and permute correlated groups together. An interviewer who asks “your random forest says `customer_id` is the top feature — why?” wants exactly this answer (plus: check for leakage).

§10.4 AdaBoost

Prototypical example for this section: Ten points on a line, stumps as weak learners, three rounds by hand.

Boosting builds an additive model $F(\mathbf{x}) = \sum_m \alpha_m h_m(\mathbf{x})$ sequentially, each weak learner focusing on what the previous ones got wrong. AdaBoost (binary, $y \in \{\pm 1\}$):

Algorithm: AdaBoost

1. Start with weights $w_i = 1/n$.
2. For $m = 1, \dots, M$: fit h_m to minimise weighted error $\varepsilon_m = \sum_i w_i \mathbb{1}[h_m(\mathbf{x}_i) \neq y_i]$; set $\alpha_m = \frac{1}{2} \ln \frac{1-\varepsilon_m}{\varepsilon_m}$; update $w_i \leftarrow w_i \exp(-\alpha_m y_i h_m(\mathbf{x}_i))$ and renormalise.
3. Predict $\text{sign}(\sum_m \alpha_m h_m(\mathbf{x}))$.

Misclassified points are multiplied by e^{α_m} , correct ones by $e^{-\alpha_m}$. After the update the current learner has weighted error exactly $\frac{1}{2}$ — it is useless on the new weights, which forces the next learner to find something new.

Example 10.4.1 (Three rounds by hand)

$x = 1, \dots, 10$ with labels $+, +, -, -, -, +, +, +, -, -$. Each round picks the stump with the lowest weighted error:

round	stump	misclassified x	ε_m	α_m	train acc. of F
1	$x \leq 2.5 \Rightarrow +1$	6, 7, 8	0.3000	0.4236	0.7
2	$x \leq 8.5 \Rightarrow +1$	3, 4, 5	0.2143	0.6496	0.7
3	$x \leq 5.5 \Rightarrow -1$	1, 2, 9, 10	0.1818	0.7520	1.0

Round 1: $\varepsilon_1 = 3 \times 0.1 = 0.3000$, $\alpha_1 = \frac{1}{2} \ln \frac{0.7}{0.3} = 0.4236$. Unnormalised weights become $0.1e^{0.4236}$ for the three mistakes and $0.1e^{-0.4236}$ for the rest; they sum to $Z_1 = 0.9165$, and after normalising each mistake has weight $0.1667 (= \frac{1}{2.7})$ and each correct point $0.0714 (= \frac{1}{2.7})$: the mistakes now carry half the total weight. Three stumps classify all ten points correctly although no single stump does better than 70%. scikit-learn's `AdaBoostClassifier` (SAMME) reports the same errors and estimator weights $\ln \frac{1-\varepsilon}{\varepsilon} = 2\alpha_m$, and the same predictions.

AdaBoost is forward stagewise additive modelling with the exponential loss $e^{-yF(x)}$: the α_m formula is the exact line-search minimiser of that loss. Because e^{-m} grows exponentially for negative margins, AdaBoost pours weight onto mislabelled or outlying points — its main weakness on noisy data.

§10.5 Gradient boosting

Prototypical example for this section: Fit a stump to the residuals, add half of it, repeat.

§10.5.i Functional gradient descent

Treat the n predictions $F(x_1), \dots, F(x_n)$ as the parameters. Gradient descent on the total loss $\sum_i \ell(y_i, F(x_i))$ would move each prediction along its **pseudo-residual** $r_i = -\partial \ell / \partial F(x_i)$. But we need a function that generalises to new x , so fit a small regression tree h_m to (x_i, r_i) and step along it:

$$F_m(x) = F_{m-1}(x) + \nu h_m(x), \quad 0 < \nu \leq 1 \text{ (shrinkage, learning rate).}$$

For squared loss $\frac{1}{2}(y - F)^2$ the pseudo-residual is the ordinary residual $y - F$; for log-loss with F the log-odds it is $y - p$; for absolute loss it is $\text{sign}(y - F)$, which is why boosting can use robust losses.

Example 10.5.1 (Two rounds by hand)

$x = (1, 2, 3, 4)$, $y = (1, 3, 6, 8)$, squared loss, stumps, $\nu = 0.5$. $F_0 = \bar{y} = 4.50$. Residuals $(-3.5, -1.5, 1.5, 3.5)$; the best stump splits at $x \leq 2.5$ and predicts $(-2.5, -2.5, 2.5, 2.5)$. Add half: $F_1 = (3.25, 3.25, 5.75, 5.75)$. New residuals $(-2.25, -0.25, 0.25, 2.25)$; the next stump predicts $(-2.25, 0.75, 0.75, 0.75)$, so $F_2 = (2.125, 3.625, 6.125, 6.125)$. `GradientBoostingRegressor(n_estimators=2, learning_rate=0.5, max_depth=1)` returns exactly these.

```
code/ml/c10_boosting.py: gradient boosting for squared loss

1 def gb_regress(X, y, M=100, nu=0.1, depth=3, seed=0):
2     rs = np.random.RandomState(seed)           # one RNG shared by all trees,
        ↪ as sklearn does
3     X = X.astype(np.float32)                   # sklearn grows its trees on
        ↪ float32 inputs
4     F0 = y.mean()
5     F = np.full(len(y), F0)
6     trees = []
7     for _ in range(M):
8         r = y - F                               # negative gradient of 1/2 (y -
        ↪ F)²
9         t = DecisionTreeRegressor(max_depth=depth, random_state=rs).fit(X, r)
10        F += nu * t.predict(X)
11        trees.append(t)
12    return lambda Z: F0 + nu * sum(t.predict(Z.astype(np.float32)) for t in trees)
```

On the Friedman-1 regression benchmark this loop reproduces the test predictions of scikit-learn's `GradientBoostingRegressor` to 10^{-8} , once it shares one random state across trees and grows them on `float32` inputs as scikit-learn does. For log-loss each leaf takes a Newton step,

$$\sum_{\text{leaf}} (y_i - p_i) / \sum_{\text{leaf}} p_i(1 - p_i),$$

instead of the mean residual; our version of that matches the decision function of `GradientBoostingClassifier` as well.

Remark 10.5.2 — Boosting *can* overfit: training loss keeps falling with M . Control it with a small ν (0.01–0.1) plus early stopping on validation loss, shallow trees (depth 3–8 or 8–64 leaves), row subsampling ([stochastic gradient boosting](#)) and column subsampling. Small ν needs more trees: $\nu \times M$ is roughly constant at the optimum.

§10.6 XGBoost

Prototypical example for this section: Derive the optimal leaf value $-G/(H + \lambda)$ and the split gain, then check them against the library on six points.

§10.6.i Second-order objective

At round t , XGBoost adds a tree f_t to minimise

$$\sum_i \ell(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2,$$

where T is the number of leaves and w_j the leaf values. Expand the loss to second order with $g_i = \partial \ell / \partial \hat{y}$ and $h_i = \partial^2 \ell / \partial \hat{y}^2$ at the current prediction:

$$\sum_i \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \gamma T + \frac{1}{2} \lambda \sum_j w_j^2 + \text{const.}$$

Grouping examples by leaf (I_j = examples in leaf j , $G_j = \sum_{I_j} g_i$, $H_j = \sum_{I_j} h_i$):

$$\sum_j \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T.$$

Each bracket is a one-dimensional quadratic, so

Theorem 10.6.1 (Leaf weight and structure score)

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad \text{objective}^* = -\frac{1}{2} \sum_j \frac{G_j^2}{H_j + \lambda} + \gamma T,$$

and splitting a leaf into L and R changes the objective by

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma.$$

For squared loss $\frac{1}{2}(y - \hat{y})^2$: $g = \hat{y} - y$, $h = 1$, so H_j is a count and w^* is a shrunk mean residual. For log-loss: $g = p - y$, $h = p(1 - p)$, a Newton step like Friedman's leaf update. λ shrinks leaf values (and makes leaves with tiny Hessian sums harmless); γ is a minimum gain per extra leaf (pre-pruning); `min_child_weight` is a minimum H_j per leaf.

Example 10.6.2 (Squared loss, by hand and by library)

$x = 1, \dots, 6$, $y = (1, 2, 2, 6, 7, 9)$, initial prediction 0.5, $\lambda = 1$, $\gamma = 0$, one depth-1 tree, learning rate 1. Gradients $g = \hat{y} - y = (-0.5, -1.5, -1.5, -5.5, -6.5, -8.5)$, $h_i = 1$, $G = -24.0$. Bracketed gains (without the $\frac{1}{2}$) for each midpoint threshold: 1.5 : 9.88, 2.5 : 15.85, 3.5 : 25.84, 4.5 : 8.91, 5.5 : -6.12. The best split is $x < 3.5$ with $G_L = -3.5$, $H_L = 3$, $G_R = -20.5$, $H_R = 3$:

$$w_L = \frac{3.5}{3+1} = 0.8750, \quad w_R = \frac{20.5}{3+1} = 5.1250, \quad \frac{3.5^2}{4} + \frac{20.5^2}{4} - \frac{24^2}{7} = 25.8393.$$

`xgboost.train` (exact method) chooses the same threshold, its leaves hold the same weights, its `Cover` is $\sum h_i = 6$, and its reported `Gain` is 25.8393 — the bracket *without* the $\frac{1}{2}$ (the paper's `Gain` is 12.9196). Without the split the single leaf would be $24/7 = 3.4286$.

Example 10.6.3 (Log-loss)

Labels (0, 1, 0, 0, 1, 1) at $x = 1, \dots, 6$, initial probability 0.5: $g_i = 0.5 - y_i$, $h_i = 0.25$. The best split is $x < 4.5$: $G_L = 1.0$, $H_L = 1.00$, $G_R = -1.0$, $H_R = 0.50$, so $w_L = -1/2 = -0.5000$, $w_R = 1/1.5 = 0.6667$ and the predicted probabilities are $\sigma(w) = 0.3775$ and 0.6608 — both matching the library. Setting `gamma` just below the reported gain 1.1667 keeps the split (2 leaves); just above prunes it (1 leaf). In the library, a split survives iff the reported gain exceeds `gamma`.

§10.6.ii The engineering that made it win

- **Shrinkage and column subsampling**, as in stochastic gradient boosting.
- **Sparsity-aware split finding**. Each split learns a *default direction* for missing values by trying both and keeping the better, scanning only non-missing entries. With 20% of entries missing, the root of our first tree splits on `f23` and sends missing values to the “yes” branch: yes.

- **Approximate and histogram splits.** Instead of scanning every distinct value, bucket each feature into quantiles (weighted by h_i , the “weighted quantile sketch”) or fixed histograms (`tree_method="hist"`, default 256 bins), turning split search from $O(n)$ per feature into $O(\text{bins})$.
- **Systems:** column blocks in compressed, pre-sorted format; cache-aware access; parallel split search across features; out-of-core computation.

The tree dump for that missing-value example, as printed by the library:

```
output (generated by build.sh)
0: [f23<876.5] yes=1,no=2,missing=1
  1: [f7<0.0559800006] yes=3,no=4,missing=3
    3: leaf=0.387992859
    4: leaf=-0.498231322
  2: [f12<2.35500002] yes=5,no=6,missing=6
    5: leaf=-0.106682248
    6: leaf=-0.766024709
```

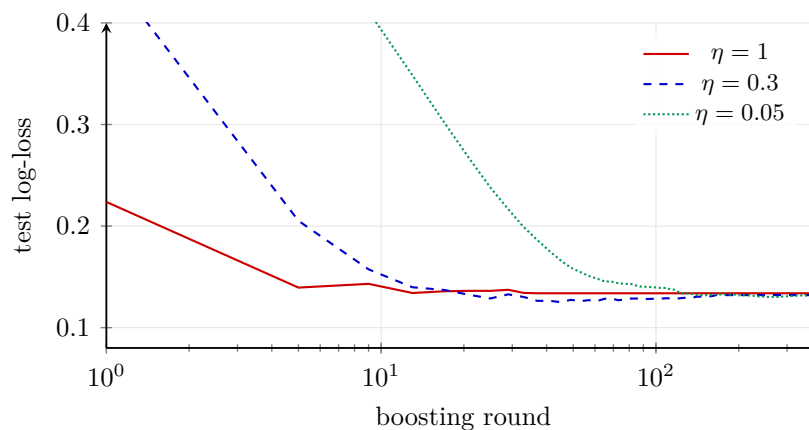


Figure 10.1: XGBoost on the breast-cancer data: a large learning rate reaches its best test loss in 11 rounds and then degrades; $\eta = 0.3$ is best at round 45 (loss 0.1251); $\eta = 0.05$ needs 263 rounds. Smaller steps need more trees and are more forgiving.

§10.7 LightGBM and CatBoost

Prototypical example for this section: Same accuracy, an order of magnitude less time on a million rows.

LightGBM [Ke+17]: histogram-based splits by default; *leaf-wise* (best-first) tree growth — always split the leaf with the largest gain, controlled by `num_leaves` rather than depth, which reaches lower loss for the same number of leaves but can overfit small data (cap `max_depth` or raise `min_data_in_leaf`); *gradient-based one-side sampling* (keep all large-gradient rows, subsample small-gradient ones); *exclusive feature bundling* (merge sparse features that are rarely non-zero together); native categorical splits.

CatBoost [Pr+18]: *ordered target statistics* for categorical features (encode each row’s category using only rows that come before it in a random permutation, avoiding target leakage), *ordered boosting* to reduce the prediction shift from reusing the same rows for

gradients and fitting, and *oblivious* (symmetric) trees that use the same split at every node of a level, which are fast to evaluate and act as a regulariser.

On a 3000-row synthetic problem, LightGBM reaches test accuracy 0.968 and XGBoost (hist) 0.964; differences of this size are noise — tuning matters more than the library.

§10.8 Stacking and the bias–variance story

Prototypical example for this section: Logistic regression, a random forest and k -NN, combined by a logistic regression trained on their out-of-fold predictions.

Stacking trains a meta-model on the base models' predictions. The meta-model must be trained on *out-of-fold* predictions (each base model predicts only rows it was not trained on); otherwise it learns to trust whichever base model overfits most. Blending uses a single hold-out set instead of folds.

model	test accuracy
lr	0.885
rf	0.960
knn	0.973
stack (LR on out-of-fold predictions)	0.971

Here the stack does not beat its best member: stacking helps when base models are good *and* make different errors, and costs complexity always.

§10.8.i Bagging versus boosting, with caveats

	bagging / random forest	boosting (AdaBoost, GBM, XGBoost)
training	parallel, independent	sequential, each fixes the previous
base learner	strong, low bias, high variance (deep trees)	weak, high bias (stumps, shallow trees)
main effect	reduces variance	reduces bias (and variance, via shrinkage)
overfitting	robust; more trees never hurt	can overfit with too many rounds; needs early stopping
noise	robust	AdaBoost sensitive to label noise; GBM less so with robust losses
weights	equal votes	learner weights α_m or shrinkage ν

Caveats an interviewer may push on: boosting also reduces variance when it averages many shrunk trees; a random forest's bias is slightly above a single tree's; and the boosted-stumps row in the bias–variance table above shows boosting cutting both terms when the target is additive (stumps can represent sums of one-feature functions exactly).

Average decorrelated low-bias models to kill variance (forests); add up shrunk corrections along the loss gradient to kill bias (boosting); XGBoost is that with a second-order step, λ on leaf values and γ on leaves.

§10.9 A few problems to think about

Problem 10A (Bootstrap probabilities). A dataset has $n = 4$ rows. (a) What is the probability that row 1 is absent from a bootstrap sample? (b) That all four rows appear? (c) The expected number of distinct rows?



[hint · solution](#)

Problem 10B (An AdaBoost round). Five points have equal weights; the first stump misclassifies exactly one point. Compute ε_1 , α_1 and the normalised weights after the update.

[hint · solution](#)

Problem 10C (XGBoost leaf). A leaf receives four examples with squared-error gradients $g = (-2, -1, 0.5, -1.5)$ and $h = 1$ each. With $\lambda = 2$, what is its weight? What would it be with $\lambda = 0$, and what does λ do?

[hint · solution](#)

Problem 10D (Split gain). Log-loss, all current probabilities 0.5, $\lambda = 1$, $\gamma = 0$. A candidate split sends labels $(1, 1, 1, 0)$ left and $(0, 0)$ right. Compute the XGBoost gain (with the $\frac{1}{2}$).

[hint · solution](#)

Problem 10E (Which are true?). (a) Random forests reduce bias compared with a single deep tree. (b) Boosting trains its learners sequentially. (c) Increasing the number of trees in a random forest causes overfitting. (d) XGBoost uses second-order derivatives of the loss. (e) OOB error requires a separate validation set. (f) MDI importance is biased towards high-cardinality features.

[hint · solution](#)

Problem 10F (Variance of a forest). Each tree has prediction variance 4 and pairwise correlation 0.25. What is the variance of the average of 10 trees? Of infinitely many? What correlation would bring the limit down to 0.5?

[hint · solution](#)

III

Learning Without (Enough) Labels

Part III: Contents

11	Unsupervised Learning	95
11.1	<i>k</i> -means	95
11.2	Hierarchical clustering	97
11.3	DBSCAN	97
11.4	Gaussian mixtures and EM	98
11.5	PCA	99
11.6	ICA, t-SNE and UMAP	100
11.7	Anomaly detection	101
11.8	Association rules	101
11.9	A few problems to think about	102
12	Semi- and Self-Supervised Learning	103
12.1	Assumptions	103
12.2	Self-training	103
12.3	Co-training and label propagation	103
12.4	Consistency regularisation	104
12.5	When unlabelled data hurts	104
12.6	Contrastive learning in one page	105
12.7	A few problems to think about	106

11 Unsupervised Learning

Without labels there is no test accuracy to hide behind, so interviewers ask about mechanics and assumptions: when does k -means stop, why does it need k and scaling, what does EM actually alternate, why standardise before PCA. OA papers favour the k -means stopping question and the PCA scaling question; both are answered here with numbers.

§11.1 k -means

Prototypical example for this section: Seven points, two starting centroids, one assign-and-update step by hand.

§11.1.i Lloyd's algorithm

k -means seeks centroids $\mu_1, \dots, \mu_k$ and an assignment $c(i)$ minimising the **inertia** (within-cluster sum of squares)

$$J = \sum_i \left\| \mathbf{x}_i - \mu_{c(i)} \right\|^2.$$

Lloyd's algorithm alternates two steps, each of which minimises J over one block of variables with the other fixed:

1. **Assign:** $c(i) = \arg \min_k \|\mathbf{x}_i - \mu_k\|$ (best c for fixed μ).
2. **Update:** $\mu_k = \text{mean of points assigned to } k$ (the minimiser of $\sum_{c(i)=k} \|\mathbf{x}_i - \mu\|^2$ for fixed c).

Example 11.1.1 (One iteration by hand)

Initial centroids (1, 1) and (5, 7):

point	distance to μ_1	distance to μ_2	cluster
(1, 1)	0.00	7.21	1
(1.5, 2)	1.12	6.10	1
(2.8, 4)	3.50	3.72	1
(5, 7)	7.21	0.00	2
(3.5, 5)	4.72	2.50	2
(4.5, 5)	5.32	2.06	2
(3.5, 4.5)	4.30	2.92	2

New centroids are the cluster means: (1.77, 2.33); (4.12, 5.38) (`KMeans(max_iter=1)` agrees). Inertia over the iterations: 32.49, 11.57, 8.92, converged after 3 assignment passes.

§11.1.ii Why it stops, and the stopping conditions

Each step cannot increase J , $J \geq 0$, and there are finitely many assignments, so Lloyd's algorithm terminates — at a *local* minimum, not necessarily the global one (which is NP-hard to find). In practice it stops when any of these holds:

- no point changes cluster (assignments are stable — the classical criterion);
- centroids move less than a tolerance (scikit-learn: `tol` on the centroid shift);
- the decrease in inertia falls below a threshold;
- a maximum number of iterations (`max_iter`) is reached.

Remark 11.1.2 — Trap. “ k -means stops when the inertia reaches zero” (only if $k = n$) or “when every cluster has the same number of points” (never a criterion). Also, k -means always converges but may converge to a poor local optimum: hence multiple restarts (`n_init`) and smart seeding.

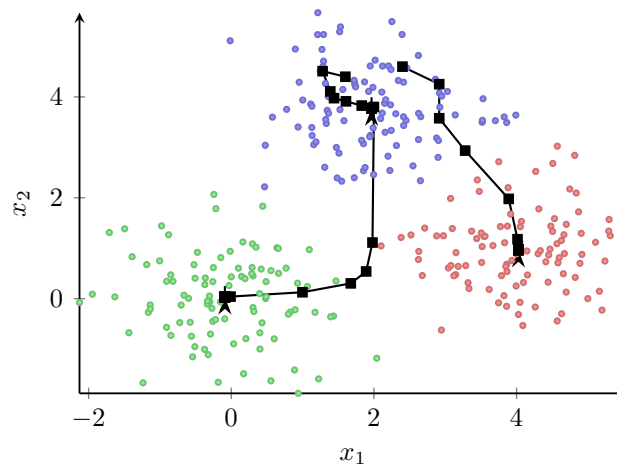


Figure 11.1: Lloyd iterations from a deliberately bad start (all three centroids inside one blob). The centroids (squares) walk to the cluster centres; inertia decreases at every step.

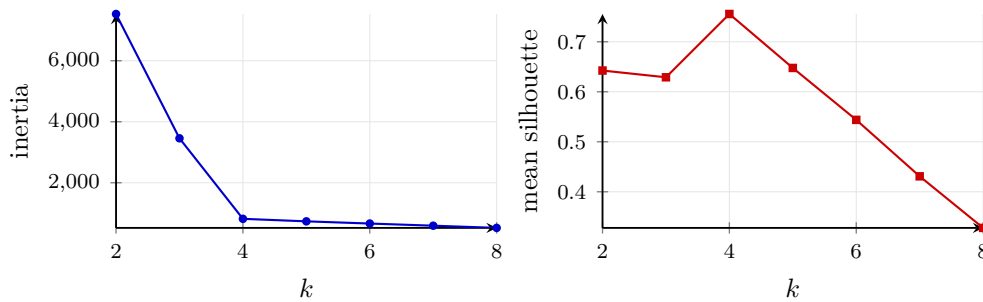
§11.1.iii Initialisation: k -means++

k -means++ [ArVa07] picks the first centroid uniformly at random, then each next one with probability proportional to $D(\mathbf{x})^2$, the squared distance to the nearest centroid already chosen. It spreads the seeds out and comes with an $O(\log k)$ approximation guarantee in expectation. On 8 well-separated blobs, single runs with random initialisation ended more than 5% above the best inertia found in 0.92 of 50 seeds (worst run $18.41 \times$ the best); with k -means++ in 0.14 (worst $1.47 \times$).

§11.1.iv Choosing k ; assumptions

Inertia always decreases with k , so look for an **elbow**, or use the **silhouette**: for point i with mean intra-cluster distance $a(i)$ and mean distance $b(i)$ to the nearest other cluster, $s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1]$. For $(0, 0)$ in clusters $\{(0, 0), (0, 1), (1, 0)\}$ and

$\{(5, 5), (5, 6), (6, 5)\}$: $a = 1.000$, $b = 7.564$, $s = 0.8678$ (matches `silhouette_samples`). On four blobs the mean silhouette peaks at $k = 4$.



k -means implicitly assumes spherical clusters of similar size and spread (it is EM for a Gaussian mixture with equal, isotropic, vanishing variances). It uses Euclidean distance, so features must be scaled; it is sensitive to outliers (means are not robust — k -medoids uses actual points); and it cannot find non-convex shapes (Figure 11.2).

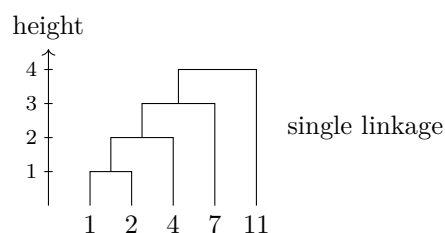
§11.2 Hierarchical clustering

Prototypical example for this section: Five numbers 1, 2, 4, 7, 11 merged bottom-up under three linkages.

Agglomerative clustering starts with singletons and repeatedly merges the two closest clusters; the history is a **dendrogram**, cut at any height to get a flat clustering — no k needed in advance. Distance between clusters (**linkage**): *single* (closest pair; finds chains, suffers from “chaining”), *complete* (farthest pair; compact clusters), *average* (mean pairwise), *Ward* (smallest increase in within-cluster variance; closest in spirit to k -means).

Example 11.2.1 (Merge heights)

For 1, 2, 4, 7, 11: single linkage merges at heights 1, 2, 3, 4 ($\{1, 2\}$, then 4 joins at distance 2, then 7 at 3, then 11 at 4); complete linkage at 1, 3, 4, 10; average linkage at 1, 2.5, 4, 6.66667 (all three match `scipy.cluster.hierarchy.linkage`).



Cost: naive agglomeration is $O(n^3)$ time and $O(n^2)$ memory for the distance matrix, so it is used on thousands, not millions, of points.

§11.3 DBSCAN

Prototypical example for this section: Two interleaved half-moons that k -means cuts in half.

DBSCAN [Es+96] groups points in dense regions. With radius ε and threshold `min_samples`: a **core point** has at least `min_samples` points (itself included) within ε ; a **border point** is within ε of a core point but is not core; everything else is **noise**. Clusters are connected components of core points plus their borders.

Example 11.3.1 (Labelling points)

Points $(0, 0), (0, 1), (1, 0), (1, 1), (2.2, 1), (5, 5)$, $\varepsilon = 1.5$, `min_samples` = 4. Neighbour counts: 4, 4, 4, 5, 2, 1, so the kinds are core, core, core, core, border, noise.

Strengths: arbitrary shapes, number of clusters not specified, explicit noise. Weaknesses: one global density threshold (clusters of different densities break it — HDBSCAN and OPTICS address this), choosing ε (look at the sorted distance to the k -th neighbour), and high dimensions. On two moons, k -means reaches adjusted Rand index 0.274 and DBSCAN 1.000.

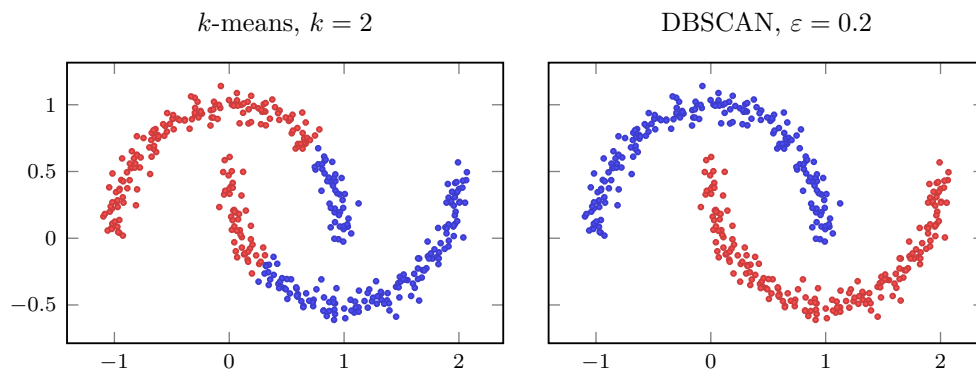


Figure 11.2: k -means draws a straight boundary between two centroids; DBSCAN follows density.

§11.4 Gaussian mixtures and EM

Prototypical example for this section: A histogram with two bumps: estimate both bumps without knowing which point came from which.

§11.4.i The model and the difficulty

A **Gaussian mixture** has density $p(\mathbf{x}) = \sum_k \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$. If we knew each point's component z_i , fitting would be per-component MLE; we do not, and the log-likelihood $\sum_i \log \sum_k \pi_k \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ has a sum inside the log with no closed-form maximiser.

§11.4.ii EM

Expectation–maximisation [DeLaRu77] alternates:

- **E-step:** compute **responsibilities**

$$r_{ik} = \mathbb{P}(z_i = k \mid \mathbf{x}_i) = \frac{\pi_k \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}.$$

- **M-step:** re-estimate with soft counts $N_k = \sum_i r_{ik}$: $\pi_k = N_k/n$, $\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_i r_{ik} \mathbf{x}_i$, $\boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_i r_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top$.

Why it works. For any distribution q over z , Jensen's inequality gives

$$\log p(\mathbf{x}) = \log \sum_z q(z) \frac{p(\mathbf{x}, z)}{q(z)} \geq \sum_z q(z) \log \frac{p(\mathbf{x}, z)}{q(z)},$$

a lower bound (the ELBO) with gap $\text{KL}(q \| p(z | \mathbf{x}))$. The E-step sets $q = p(z | \mathbf{x})$, making the bound tight; the M-step maximises the bound over the parameters. So the likelihood never decreases: it is coordinate ascent on the ELBO, converging to a local maximum or saddle. $\square$

code/ml/c11_unsup.py: one EM step for a 1-D mixture

```
1 def em_step(x, pi, mu, var):
2     # E-step: responsibilities r_ik = pi_k N(x_i | mu_k, var_k) / sum_j (...)
3     dens = pi * norm.pdf(x[:, None], mu, np.sqrt(var))
4     r = dens / dens.sum(1, keepdims=True)
5     # M-step: weighted MLEs
6     Nk = r.sum(0)
7     pi = Nk / len(x)
8     mu = (r * x[:, None]).sum(0) / Nk
9     var = (r * (x[:, None] - mu) ** 2).sum(0) / Nk
10    return pi, mu, var, np.log(dens.sum(1)).sum()
```

Example 11.4.1 (EM, checked)

Equal weights, means -1 and 1 , unit variances: for $x = -1, 0, 2$ the responsibilities of the first component are $(0.881, 0.500, 0.018)$ — a point midway gets 0.5 . On 500 points drawn from $0.6\mathcal{N}(-2, 0.8^2) + 0.4\mathcal{N}(2.5, 1.2^2)$, EM converges in 21 iterations to $\pi = (0.601, 0.399)$, $\mu = (-2.041, 2.518)$, $\sigma^2 = (0.632, 1.268)$, identical to scikit-learn's `GaussianMixture` started from the same point; the log-likelihood rises at every iteration.

k -means is the “hard” limit: responsibilities rounded to $0/1$, all covariances $\sigma^2 \mathbf{I}$ with $\sigma \rightarrow 0$. GMMs give soft assignments, elliptical clusters, and a density usable for anomaly detection; choose k with BIC (Chapter 13). Pitfalls: a component can collapse onto one point (variance $\rightarrow 0$, likelihood $\rightarrow \infty$), hence a small `reg_covar`; and EM is sensitive to initialisation (usually from k -means).

§11.5 PCA

Prototypical example for this section: Ten 2-D points that lie near a line: one number per point keeps 96% of the variance.

§11.5.i Two derivations

Maximum variance. Find the unit vector $\mathbf{u}$ maximising the variance of the projections of centred data, $\mathbf{u}^\top \mathbf{S} \mathbf{u}$ with $\mathbf{S}$ the sample covariance, subject to $\mathbf{u}^\top \mathbf{u} = 1$. The Lagrangian $\mathbf{u}^\top \mathbf{S} \mathbf{u} - \lambda(\mathbf{u}^\top \mathbf{u} - 1)$ gives $\mathbf{S} \mathbf{u} = \lambda \mathbf{u}$: $\mathbf{u}$ is an eigenvector and the variance achieved is λ , so take the top eigenvector. Each next component is the top eigenvector orthogonal to the previous ones.

Minimum reconstruction error / SVD. With centred $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, the best rank- k approximation (Eckart–Young, [Chapter 2](#)) uses the top k right singular vectors, which are the eigenvectors of $\mathbf{X}^\top\mathbf{X} = (n-1)\mathbf{S}$, and $\lambda_j = \sigma_j^2/(n-1)$. Libraries use the SVD: it never forms $\mathbf{X}^\top\mathbf{X}$ and is more stable.

Example 11.5.1 (PCA by hand)

Ten points with mean (1.81, 1.91) and covariance $\mathbf{S} = \begin{pmatrix} 0.6166 & 0.6154 \\ 0.6154 & 0.7166 \end{pmatrix}$. Its eigenvalues are 1.2840 and 0.0491; PC1 = (0.6779, 0.7352) explains 0.9632 of the variance. The first point (2.5, 2.4) projects to $z_1 = (2.5 - 1.81, 2.4 - 1.91) \cdot \text{PC1} = 0.8280$. Eigenvalues, PC1 (up to sign) and the projection match `sklearn.decomposition.PCA`, and equal $\sigma_j^2/(n-1)$ from the SVD.

§11.5.ii Why standardise first

PCA maximises *variance*, which depends on units. In the wine data, `proline` has variance 99167 while `hue` has 0.052. Unscaled, PC1 explains 0.9981 of the variance and is essentially the `proline` axis (loading 0.9998) — a unit conversion, not structure. After standardising, PC1 explains 0.3620, 8 components reach 90%, and k -means on the first two components recovers the wine cultivars with adjusted Rand index 0.895 versus 0.371 unscaled.

Remark 11.5.2 — Trap. “Always standardise before PCA” is the expected answer when features have different units; the precise answer is: standardise when variance differences are artefacts of units (PCA on the correlation matrix); do not, when all features share a unit and their variances are meaningful (pixel intensities, returns on similar assets). And always *centre*. Other traps: PCA is unsupervised, so the top components need not be the most predictive; it is linear; and the components are only defined up to sign.

Choose k by cumulative explained variance (e.g. 90–95%), a scree-plot elbow, or downstream validation score. Uses: compression, de-noising, visualisation, decorrelating features for linear models (PCA regression), speeding up distance-based methods.

§11.6 ICA, t-SNE and UMAP

Prototypical example for this section: Two speakers recorded by two microphones: *unmix the voices*.

Independent component analysis assumes $\mathbf{x} = \mathbf{A}\mathbf{s}$ with *statistically independent, non-Gaussian* sources $\mathbf{s}$ and finds an unmixing matrix maximising non-Gaussianity (FastICA uses negentropy approximations). PCA only decorrelates (second-order statistics) and cannot recover the sources; mixing a square wave and a sine wave, ICA recovers both with correlation at least 0.998 while PCA’s best match is only 0.804. ICA recovers sources up to order and scale, and fails for Gaussian sources (any rotation of independent Gaussians is still independent).

t-SNE [MaHi08] turns pairwise distances into neighbour probabilities (Gaussian in input space, with bandwidths set by a *perplexity*, roughly the effective number of neighbours) and finds a 2-D embedding whose Student- t neighbour probabilities match them, minimising a KL divergence. The heavy tail lets moderately distant points spread

out, which separates clusters. **UMAP** builds a fuzzy k -NN graph and optimises a cross-entropy between graph and embedding; it is faster and often preserves more global structure. For both: use for *visualisation*; cluster sizes and inter-cluster distances in the picture are not meaningful; results depend on perplexity/neighbours and the random seed; there is no **transform** for new points in standard t-SNE.

§11.7 Anomaly detection

Prototypical example for this section: Ten scattered points among 300 normal ones: score how easy each is to isolate.

Isolation forest [LiTiZh08] builds random trees by choosing a random feature and a random threshold between its min and max, until each point is alone. Anomalies are few and different, so they are isolated near the root: short average path length $h(\mathbf{x})$. The score is $2^{-\mathbb{E}[h(\mathbf{x})]/c(n)}$ with $c(n) = 2H(n-1) - 2(n-1)/n$ the average path length of an unsuccessful binary-search-tree lookup ($c(256) = 10.245$ for the default sub-sample of 256). On our data the 10 planted outliers average score 0.713 against 0.410 for inliers, and all 10 of the 10 highest scores are planted outliers. Alternatives: GMM density (low likelihood), one-class SVM, local outlier factor (density relative to neighbours), reconstruction error of PCA or an autoencoder.

§11.8 Association rules

Prototypical example for this section: Six shopping baskets: does buying butter predict buying bread?

For itemsets A, B over N transactions:

$$\begin{aligned} \text{support}(A) &= \frac{\#\{t : A \subseteq t\}}{N}, & \text{confidence}(A \Rightarrow B) &= \frac{\text{support}(A \cup B)}{\text{support}(A)}, \\ \text{lift}(A \Rightarrow B) &= \frac{\text{confidence}(A \Rightarrow B)}{\text{support}(B)}. \end{aligned}$$

Example 11.8.1 (Baskets)

Transactions: {bread, milk}, {bread, butter, milk}, {bread, butter}, {milk, tea}, {bread, butter, milk}, {bread, tea}. Support(bread) = 0.8333, support(butter) = 0.5000, support(bread, butter) = 0.5000. Confidence(butter $\Rightarrow$ bread) = 1.0000, but confidence(bread $\Rightarrow$ butter) = 0.6000: confidence is not symmetric. Lift (symmetric) = 1.2000 > 1: positively associated; lift(milk, tea) = 0.7500 < 1.

Apriori [AgSr94] finds frequent itemsets level by level using *anti-monotonicity*: every subset of a frequent itemset is frequent, so a candidate with an infrequent subset is pruned without counting. With minimum support 1/3 it returns {bread}; {butter}; {milk}; {tea}; {bread, butter}; {bread, milk}; {butter, milk}; {bread, butter, milk} — the same as brute-force enumeration.

Remark 11.8.2 — Trap. High confidence alone is misleading when the consequent is common: “anything $\Rightarrow$ bread” has high confidence because 83% of baskets contain bread. Lift corrects for this; lift = 1 means independence.

k -means is coordinate descent on inertia (it always stops, maybe badly); EM is coordinate ascent on a likelihood bound; PCA is an eigenproblem on the covariance — so scale before it unless units agree.

§11.9 A few problems to think about

Problem 11A (One k -means step). 1-D points 1, 2, 3, 8, 9, 10, 25 with initial centroids 2 and 9. Do one assignment and update. What happens next, and what would k -means with $k = 2$ do to the outlier 25?

[hint](#) · [solution](#)

Problem 11B (PCA explained variance). A covariance matrix has eigenvalues 4.0, 2.5, 1.0, 0.3, 0.2. How many components keep at least 90% of the variance? What fraction does PC1 explain?

[hint](#) · [solution](#)

Problem 11C (PCA of a 2×2 covariance). Centred data have covariance $\begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$. Find the principal directions, the variances along them, and the projection of $\mathbf{x} = (2, 0)$ onto PC1.

[hint](#) · [solution](#)

Problem 11D (Association rule). In 1000 orders, 200 contain chips, 300 contain cola, 120 contain both. Compute support, confidence of chips $\Rightarrow$ cola, and lift. Is the rule useful?

[hint](#) · [solution](#)

Problem 11E (EM responsibility). A 1-D mixture has $\pi = (0.3, 0.7)$, means 0 and 4, both variances 1. Compute the responsibility of component 1 for $x = 1.5$.

[hint](#) · [solution](#)

12 Semi- and Self-Supervised Learning

Labels are expensive; raw data are cheap. Placement interviews at product companies increasingly ask “you have 1000 labelled and a million unlabelled examples — what do you do?” The honest answer starts with the assumptions that make unlabelled data useful, and with the fact that it can hurt.

§12.1 Assumptions

Prototypical example for this section: Two half-moons, six labelled points, six hundred unlabelled ones.

Unlabelled points tell you about $p(\mathbf{x})$. They help predict y only if $p(\mathbf{x})$ carries information about $p(y \mid \mathbf{x})$. Three standard ways of saying so:

- **Smoothness**: points close in a high-density region should share a label.
- **Cluster** (low-density separation): the decision boundary should pass through low-density regions, so each cluster tends to be one class.
- **Manifold**: data lie near a low-dimensional manifold, and distances *along* it are the meaningful ones.

When these hold, unlabelled data reshape the boundary; when they fail, they mislead it.

§12.2 Self-training

Prototypical example for this section: Train on the labelled set, label the unlabelled points you are most sure of, retrain, repeat.

Self-training (pseudo-labelling): fit a model on labelled data; predict the unlabelled data; add predictions above a confidence threshold as **pseudo-labels**; refit; repeat. It is simple and model-agnostic (scikit-learn’s `SelfTrainingClassifier`). The failure mode is **confirmation bias**: early mistakes become training labels and are reinforced, especially when the initial model is poor or poorly calibrated. Mitigations: high thresholds, class-balanced selection, re-labelling every round (not freezing pseudo-labels), and the teacher–student variants below.

§12.3 Co-training and label propagation

Prototypical example for this section: A web page described by its text and, independently, by the anchor text of links pointing to it.

Co-training [BIMi98] needs two *views* of each example, each sufficient on its own and conditionally independent given the label. Two classifiers, one per view, label unlabelled examples for each other: an example confidently labelled by the text model teaches the link model, which sees it differently.

Label propagation builds a graph with edge weights $W_{ij} = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ and lets labels flow along it. With labelled nodes clamped, the fixed point is the

harmonic solution: every unlabelled node's label distribution is the weighted average of its neighbours',

$$F_u = (D_{uu} - W_{uu})^{-1} W_{ul} Y_l, \quad D = \text{diag}\left(\sum_j W_{ij}\right).$$

code/ml/c12_semisup.py: label propagation in closed form

```
1 def harmonic(X, y_semi, gamma):
2     """Label propagation fixed point: f_u = (D_uu - W_uu)^{-1} W_ul Y_l (labels
   ↪ clamped)."""
3     W = np.exp(-gamma * ((X[:, None] - X[None]) ** 2).sum(-1))
4     L = y_semi >= 0
5     Yl = np.eye(2)[y_semi[L]]
6     D = np.diag(W.sum(1))
7     Fu = np.linalg.solve((D - W)[np.ix_(~L, ~L)], W[np.ix_(~L, L)] @ Yl)
8     F = np.zeros((len(X), 2)); F[L] = Yl; F[~L] = Fu
9     return F
```

On 120 moon points with four labels this matches the iterative `LabelPropagation` of scikit-learn to 10^{-4} and labels 0.900 of the points correctly. **Label spreading** adds a regularisation α that lets clamped labels be partly overwritten, which tolerates label noise.

§12.4 Consistency regularisation

Prototypical example for this section: An image and its slightly cropped, recoloured copy should get the same prediction.

Modern semi-supervised deep learning adds an unsupervised loss that penalises a model whose predictions change under perturbations of an *unlabelled* input — a direct implementation of the smoothness assumption. Examples: the Π -model (two stochastic passes should agree), Mean Teacher (a student matches an exponential moving average of its own weights), and FixMatch (pseudo-label a weakly augmented view only if confident, train the strongly augmented view to predict it). The total loss is supervised cross-entropy plus $\lambda \times$ consistency loss, with λ ramped up as the model improves.

§12.5 When unlabelled data hurts

Prototypical example for this section: One cloud of points whose labels flip across a line through its densest region.

Example 12.5.1 (Helps and hurts, measured)

Six labelled points (three per class), 594 unlabelled, mean test accuracy over 20 random draws:

data	LR (6 labels only)	RBF-SVM	self-training (6 labels + 594 unlabelled)	label spreading
two moons (gap between classes)	0.816	0.818	0.727	0.937
blobs labelled across the clusters	0.841	0.679	0.563	0.733

On two moons the classes are separated by a low-density gap and label spreading lifts accuracy from 0.818 to 0.937. On the second dataset the clusters run along x_1 but the label depends on the sign of x_2 : the cluster assumption is false, and label spreading (0.733) loses to a plain logistic regression trained on the six labelled points (0.841), whose linear bias happens to be right. Self-training with an RBF-SVM made things worse in *both* cases (0.727 on moons): with six labels the first model is weak, and its confident mistakes are fed back as truth.

Remark 12.5.2 — Trap. “More data never hurts.” More *labelled* data from the same distribution essentially never hurts; *unlabelled* data can, when the method’s assumption about $p(\mathbf{x})$ and $p(y \mid \mathbf{x})$ is wrong, or through confirmation bias. Always compare against the supervised baseline on a labelled validation set — which, with few labels, should be cross-validated.

§12.6 Contrastive learning in one page

Prototypical example for this section: Two augmented views of the same image should be closer to each other than to any other image in the batch.

Self-supervised **contrastive learning** learns an encoder f without labels: for each example build two views (augmentations); embeddings of the same example are *positives*, all other pairs in the batch are *negatives*. The **InfoNCE** loss for anchor i with positive i^+ is

$$\ell_i = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_{i^+})/\tau)}{\sum_j \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)},$$

a softmax cross-entropy whose “correct class” is the positive; sim is cosine similarity and τ a temperature (small τ sharpens the distribution and focuses on hard negatives).

Example 12.6.1 (InfoNCE on three items)

Normalised view-1 and view-2 embeddings for three items give, at $\tau = 0.5$, the similarity matrix $\begin{pmatrix} 1.99 & 0.97 & -1.95 \\ 1.37 & 1.98 & -1.52 \\ -1.96 & -0.79 & 1.90 \end{pmatrix}$ (rows: anchors; diagonal: positives). Row 1’s softmax puts 0.7239 on its positive; the mean loss over the three rows is 0.2869, identical to `torch.nn.functional.cross_entropy` applied to the matrix with targets (0, 1, 2).

After pre-training, a linear classifier on the frozen embeddings (“linear probe”) or light fine-tuning needs far fewer labels. What the augmentations leave unchanged is what the representation learns to ignore, so augmentation design *is* the inductive bias. The same recipe pairs images with captions in image–text models; see *Deep Learning* for SimCLR- and CLIP-style training.

Unlabelled data help only through an assumption linking $p(\mathbf{x})$ to $p(y \mid \mathbf{x})$; check it against a supervised baseline, and beware of a model grading its own homework.

§12.7 A few problems to think about

Problem 12A (Pick the method). For each scenario name the assumption you rely on and a method: (a) 200 labelled and 50 000 unlabelled product photos; (b) a citation graph with 1% of papers labelled by field; (c) e-mails described by body text and by sender metadata.

[hint](#) · [solution](#)

Problem 12B (Harmonic label). An unlabelled node is connected to labelled neighbours with weights 2 (class A), 1 (class A) and 3 (class B) and to nothing else. After label propagation, what is its probability of class A?

[hint](#) · [solution](#)

Problem 12C (InfoNCE temperature). An anchor has cosine similarity 0.8 with its positive and 0.5 with each of 3 negatives. Compute the InfoNCE loss at $\tau = 1$ and $\tau = 0.1$. Which temperature makes the loss more sensitive to the negatives?

[hint](#) · [solution](#)

IV

Doing It Properly

Part IV: Contents

13	Evaluation and Model Selection	109
13.1	The confusion matrix	109
13.2	ROC and PR curves	110
13.3	Regression and probabilistic metrics	110
13.4	Cross-validation	111
13.5	Leakage	112
13.6	AIC, BIC and hyper-parameter search	112
13.7	Calibration	114
13.8	A few problems to think about	114
14	Statistics Interviewers Ask	117
14.1	Distributions	117
14.2	Estimators and confidence intervals	117
14.3	Hypothesis tests	118
14.4	Chi-square and ANOVA	119
14.5	Multiple testing	119
14.6	Correlation, causation and Simpson	120
14.7	A few problems to think about	120
15	Data Preparation and Feature Engineering	123
15.1	Scaling	123
15.2	Encoding categoricals	124
15.3	Missing values and outliers	124
15.4	Feature selection	125
15.5	Imbalance	125
15.6	Text features and TF-IDF	126
15.7	Time series in one page	127
15.8	Recommenders in one page	127
15.9	A few problems to think about	127
16	Optimisation for ML	129
16.1	Gradient descent on a quadratic	129
16.2	Momentum	129
16.3	Newton and quasi-Newton	130
16.4	Coordinate descent, SGD and duality	130
16.5	A few problems to think about	131

13 Evaluation and Model Selection

Most failed ML projects do not fail at modelling; they fail at measurement — the wrong metric, a leaky split, a test set consulted twenty times. Interviewers know this, so expect: precision and recall from a confusion matrix in one minute, ROC AUC by hand, “accuracy is 97%, is the model good?”, and “how did you choose the hyper-parameters without fooling yourself?”

§13.1 The confusion matrix

Prototypical example for this section: 1000 transactions, 60 frauds; the model flags 50, of which 40 are fraud.

		predicted	
		positive	negative
actual	positive	TP = 40	FN = 20
	negative	FP = 10	TN = 930

metric	formula	value	question it answers
accuracy	$\frac{TP+TN}{P+R}$	0.9700	fraction correct
precision (PPV)	$\frac{TP}{TP+FP}$	0.8000	of the flagged, how many are fraud?
recall (sensitivity, TPR)	$\frac{TP}{TP+FN}$	0.6667	of the frauds, how many did we catch?
specificity (TNR)	$\frac{TN}{TN+FP}$	0.9894	of the genuine, how many passed?
FPR	$\frac{FP}{FP+TN}$	0.0106	false-alarm rate
F1	$\frac{2PR}{P+R}$	0.7273	harmonic mean of P and R
F_β	$\frac{(1+\beta^2)PR}{\beta^2P+R}$	$F_2 = 0.6897$	recall weighted β times as much
balanced accuracy	$\frac{TPR+TNR}{2}$	0.8280	accuracy with equal class weight
MCC	$\frac{TP \cdot TN - FP \cdot FN}{\sqrt{(P+R)(P+FP)(TNR+TN)(TP+FN)}}$	0.7149	correlation of predictions and labels

(All values checked against `sklearn.metrics`; $F_{0.5} = 0.7692$ favours precision.) A model predicting “not fraud” for everyone scores accuracy 0.940 — almost as good — with recall 0. With imbalanced classes accuracy is nearly meaningless.

Remark 13.1.1 — Trap. Which to optimise is a business decision: recall when misses are expensive (cancer screening, fraud), precision when false alarms are expensive (spam filtering into a hidden folder, recommending a loan rejection). Precision depends on the prevalence; recall and specificity do not. A model with the same TPR/FPR deployed where fraud is ten times rarer has much lower precision (recall the base-rate example of [Chapter 2](#)).

§13.1.i Macro, micro, weighted

For K classes compute per-class precision/recall/F1 one-vs-rest, then average: **macro** (plain mean: every class equal), **weighted** (mean weighted by class support), **micro** (pool all TP/FP/FN first: for single-label multiclass, micro-F1 = accuracy).

Example 13.1.2 (Three classes)

Confusion matrix (rows true) $\begin{pmatrix} 50 & 3 & 2 \\ 10 & 25 & 5 \\ 2 & 3 & 10 \end{pmatrix}$. Per-class F1: (0.855, 0.704, 0.625). Macro-F1 = 0.7280, weighted = 0.7687, micro = accuracy = 85/110 = 0.7727. The small third class drags macro-F1 down; use macro when minority classes matter.

§13.2 ROC and PR curves

Prototypical example for this section: Eight scored examples, swept threshold by threshold.

A scoring classifier gives a curve, not a point. Sweeping the threshold from high to low traces the **ROC curve** (TPR against FPR) and the **precision–recall curve**.

Example 13.2.1 (ROC by hand)

Scores 0.95, 0.85, 0.80, 0.70, 0.55, 0.45, 0.40, 0.20 with labels 1, 1, 0, 1, 0, 1, 0, 0 (4 positives, 4 negatives). Predicting positive for score $\geq t$:

threshold t	label at t	TPR	FPR	precision
0.95	1	0.25	0.00	1.000
0.85	1	0.50	0.00	1.000
0.80	0	0.50	0.25	0.667
0.70	1	0.75	0.25	0.750
0.55	0	0.75	0.50	0.600
0.45	1	1.00	0.50	0.667
0.40	0	1.00	0.75	0.571
0.20	0	1.00	1.00	0.500

Each positive moves the curve up by 1/4, each negative right by 1/4. Area by trapezoids: 0.8125. Average precision (mean of the precision at each positive's rank) = $(1 + 1 + 0.75 + 0.667)/4 = 0.8542$. Both match scikit-learn.

Theorem 13.2.2 (AUC is a ranking probability)

$\text{AUC} = \mathbb{P}(\text{score of a random positive} > \text{score of a random negative})$, counting ties as $\frac{1}{2}$ (the normalised Mann–Whitney U statistic).

Above, 13 of the 16 positive–negative pairs are correctly ordered: $13/16 = 0.8125$. Consequences: AUC is threshold-free and invariant to any monotone transformation of the scores (so it says nothing about calibration); 0.5 is random ranking; below 0.5 means the ranking is inverted.

Remark 13.2.3 — Trap. “AUC 0.9, so the model is great.” With 1% positives, the same model has average precision 0.192: FPR is tiny relative to the huge negative class, but in absolute numbers those false positives swamp the true ones. For rare positives, report PR-AUC (average precision) and precision at the operating recall.

§13.3 Regression and probabilistic metrics

Prototypical example for this section: Five house-price predictions scored five ways.

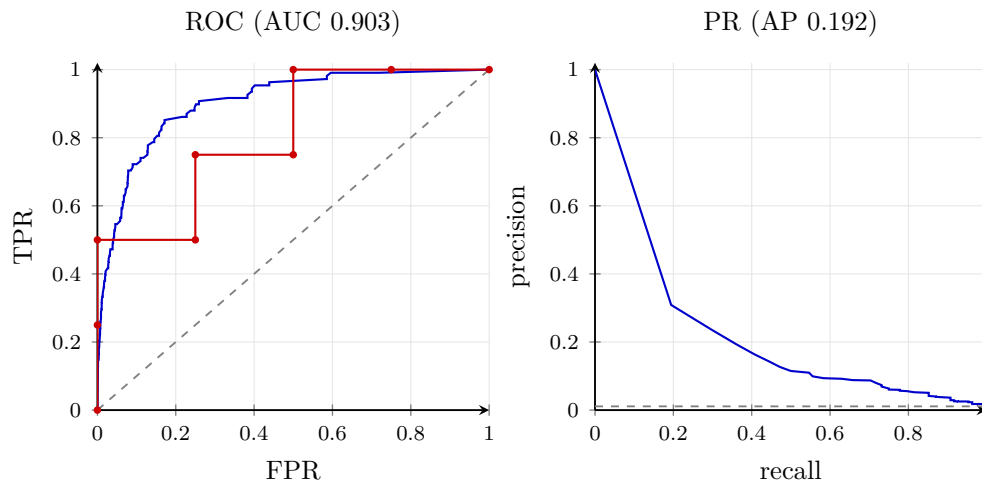


Figure 13.1: Blue: a logistic regression on data with 1% positives. The ROC curve looks excellent; the PR curve shows that at most recalls the majority of flagged cases are false alarms (dashed: precision of random guessing, the prevalence). Red: the eight-point hand example.

For $y = (3, 5, 2.5, 7, 10)$ and $\hat{y} = (2.5, 5, 4, 8, 9)$: $\text{MAE} = 0.800$, $\text{MSE} = 0.900$, $\text{RMSE} = 0.9487$ (same units as y), $\text{MAPE} = 20.19\%$, $R^2 = 0.8816$. MSE punishes large errors (and is minimised by the mean), MAE treats all errors linearly (minimised by the median), MAPE is undefined at $y = 0$ and penalises over-prediction of small values asymmetrically; RMSLE suits targets spanning orders of magnitude.

Probabilistic classifiers should also be judged as probabilities. **Log-loss** $-\frac{1}{n} \sum \log \hat{p}_{y_i}$ and the **Brier score** $\frac{1}{n} \sum (\hat{p}_i - y_i)^2$ are *proper* scoring rules: they are minimised in expectation by the true probabilities. On the imbalanced data the logistic model has log-loss 0.0434 versus 0.0598 for always predicting the base rate.

§13.4 Cross-validation

Prototypical example for this section: Five folds, each used once for validation; but what if the rows are days?

k -fold CV: split into k folds, train on $k - 1$, validate on the remaining one, average. $k = 5$ or 10 is the usual compromise: **leave-one-out** ($k = n$) is nearly unbiased but has high variance and costs n fits (except for models with closed-form LOO, like linear smoothers). Variants:

- **Stratified** k -fold keeps class proportions in each fold (default for classification).
- **Group** k -fold keeps all rows of a group (patient, user) in one fold.
- **Time-series** (forward-chaining) splits train on the past and validate on the future, with an optional gap.
- **Repeated** CV averages over several random partitions to reduce variance.

Example 13.4.1 (Shuffled CV on a time series)

A trending series, random-forest regressor on the time index. Shuffled 5-fold CV

reports MAE 0.294: each validation day is surrounded by training days, so the model interpolates. Forward-chaining CV reports 1.116: the model must extrapolate the trend, which a forest cannot do. The second number is what production will see.

§13.5 Leakage

Prototypical example for this section: 5000 pure-noise features, 60 rows, “92% cross-validated accuracy”.

Leakage is information in training that will not be available at prediction time. A catalogue:

1. **Preprocessing on all data before splitting:** scaling, imputation, PCA, feature selection, target encoding, SMOTE.
2. **Target leakage:** a feature computed after the outcome (“refund issued” to predict fraud; “days in hospital” to predict admission).
3. **Temporal leakage:** random splits of time-ordered data; features using future aggregates.
4. **Group leakage:** the same entity in train and test.
5. **Duplicates** or near-duplicates across the split.
6. **Tuning on the test set** (Chapter 1).

Example 13.5.1 (Feature selection outside the loop)

60 rows, 5000 features of pure noise, random labels. Selecting the 20 features most correlated with the labels on *all* the data and then cross-validating a logistic regression gives accuracy 0.883. Putting the selection inside a **Pipeline**, refitted within each fold, gives 0.517 — chance, the truth.

Remark 13.5.2 — Trap. “My validation score is much better than expected” is a leakage alarm, not a celebration. The standard defence is a **Pipeline** (every fitted transform lives inside the CV loop) plus a split that mirrors deployment (by time, by group).

§13.6 AIC, BIC and hyper-parameter search

Prototypical example for this section: Choose a polynomial degree for 60 points without a validation set.

§13.6.i Information criteria

With $\hat{L}$ the maximised likelihood, k the number of free parameters and n the sample size:

$$\text{AIC} = 2k - 2 \ln \hat{L}, \quad \text{BIC} = k \ln n - 2 \ln \hat{L}.$$

Lower is better. Both penalise parameters; BIC’s penalty $\ln n$ exceeds AIC’s 2 once $n \geq 8$, so BIC picks smaller models. AIC [Ak74] estimates expected predictive (KL) loss; BIC [Sc78] approximates the log marginal likelihood and is *consistent* (picks the true model as $n \rightarrow \infty$ if it is among the candidates).

Example 13.6.1 (Degree selection)

Data from a cubic plus noise, $n = 60$ ($\ln n = 4.094$); Gaussian likelihood with $\hat{\sigma}^2 = \text{RSS} / n$, $k = \text{degree} + 2$:

degree	k	$\ln \hat{L}$	AIC	BIC
1	3	-31.53	69.07	75.35
2	4	-31.40	70.80	79.18
3	5	-17.54	45.07	55.55
4	6	-17.35	46.70	59.26
5	7	-17.35	48.69	63.35
6	8	-17.23	50.47	67.22
7	9	-17.08	52.16	71.01
8	10	-17.08	54.16	75.10

Both choose degree 3; past it, $\ln \hat{L}$ creeps up but not by enough to pay for the extra parameters, and BIC’s penalty grows faster. scikit-learn’s `GaussianMixture.bic` equals $-2 \ln \hat{L} + k \ln n$ with $k = \text{means} + \text{covariances} + \text{free weights}$, as the script checks.

Remark 13.6.2 — Trap. “Lower BIC means a more accurate model.” BIC compares models *fitted to the same data* by a likelihood-based trade-off; it is not an accuracy, and comparing BIC across different datasets or different likelihood definitions is meaningless. “AIC and BIC need a validation set” — no, that is their point.

§13.6.ii Search and nested CV

Grid search tries every combination; **random search** samples configurations and, with the same budget, explores more values of the parameters that matter; **Bayesian optimisation** fits a surrogate of score versus hyper-parameters and picks promising points; successive halving / Hyperband stop bad configurations early. Whatever the search, its best CV score is optimistically biased (the selection-optimism of Chapter 1). **Nested CV** wraps the whole search in an outer CV loop to estimate the performance of the *procedure*.

Example 13.6.3 (Optimism of the inner score)

An RBF-SVM tuned over a 6×6 grid on 120 noisy rows, averaged over six independent training sets: the best inner CV accuracy averages 0.768, nested CV estimates 0.750, and the tuned models score 0.749 on 10 000 fresh rows. Nested CV told the truth; the grid’s own best score did not.

§13.7 Calibration

Prototypical example for this section: Of the e-mails a model scores at 0.8, are 80% spam?

A classifier is **calibrated** if, among examples given probability p , a fraction p are positive. Check with a **reliability diagram** (bin predictions, plot observed frequency against mean prediction) or the **expected calibration error** $\sum_b \frac{n_b}{n} |\text{acc}_b - \text{conf}_b|$. Logistic regression is usually well calibrated; naive Bayes (over-confident, Chapter 6), SVM scores, boosted trees and modern deep nets often are not. Fix on held-out data with **Platt scaling** (a sigmoid on the scores) or **isotonic regression** (a monotone step function; needs more data).

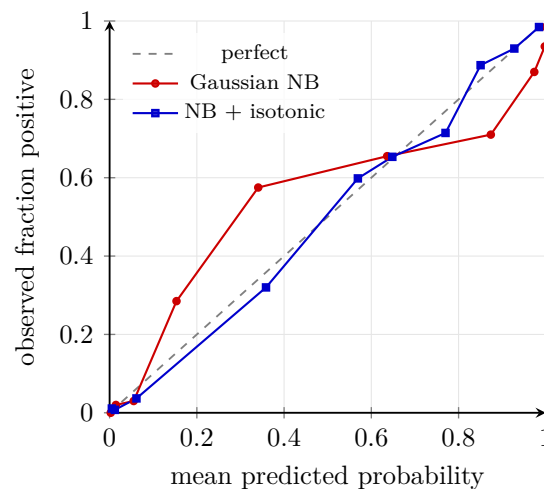


Figure 13.2: Naive Bayes on correlated features pushes probabilities to the extremes (ECE 0.0774, Brier 0.1241). Isotonic calibration fixes it (ECE 0.0232, Brier 0.1134) while leaving AUC unchanged (0.9176 vs 0.9169).

Pick the metric from the cost of each error, the split from how the model will be deployed, and never let a number you report have helped choose the model.

§13.8 A few problems to think about

Problem 13A (Metrics from a confusion matrix). A COVID test on 500 people: TP = 45, FN = 5, FP = 30, TN = 420. Compute accuracy, precision, recall, specificity and F1.

hint · solution

Problem 13B (AUC by counting). Positives score 0.9, 0.6, 0.4; negatives score 0.7, 0.3. Compute the ROC AUC.

hint · solution

Problem 13C (Precision at a new prevalence). A classifier has TPR 0.9 and FPR 0.05. What is its precision when 10% of cases are positive? When 0.5% are?

hint · solution

Problem 13D (BIC comparison). Two models on $n = 100$ points: model A has 3 parameters and $\ln \hat{L} = -120$; model B has 6 parameters and $\ln \hat{L} = -114$. Which does AIC prefer? BIC?

[hint](#) · [solution](#)

Problem 13E (Spot the leak). A churn model uses: tenure, plan type, number of support calls in the last 30 days, “account closure reason”, average monthly bill, and the customer’s churn probability from last quarter’s model. Train/test split is random over customer-months. List the problems.

[hint](#) · [solution](#)



14 Statistics Interviewers Ask

Data-science and analytics rounds (and many ML rounds) spend ten minutes on classical statistics: a distribution question, a confidence interval, “what is a p-value”, a test choice, an A/B sample size. The answers are short, but the follow-ups punish vague ones. Everything below is computed with `scipy.stats` and, where it matters, by hand alongside.

§14.1 Distributions

Prototypical example for this section: Ten loan applications each default with probability 0.2: chance of exactly three defaults?

distribution	mean	variance	typical question
Bernoulli(p)	p	$p(1-p)$	one click
Binomial(n, p)	np	$np(1-p)$	$\mathbb{P}(X = 3)$, $n = 10$, $p = 0.2$: $\binom{10}{3}0.2^30.8^7 = 0.2013$
Geometric(p)	$1/p$	$(1-p)/p^2$	trials until first success: mean 5 at $p = 0.2$
Poisson(λ)	λ	λ	calls per minute; $\mathbb{P}(X = 0)$ at $\lambda = 2$: $e^{-2} = 0.1353$
Uniform(a, b)	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	random tie-breaking
Exponential(λ)	$1/\lambda$	$1/\lambda^2$	waiting time; memoryless
Normal(μ, σ^2)	μ	σ^2	68–95–99.7 rule

Facts on call: $\mathbb{P}(\text{at least one default in 10}) = 1 - 0.8^{10} = 0.8926$; $\mathbb{P}(|Z| < 1) = 0.6827$ and $\mathbb{P}(|Z| < 1.96) = 0.9500$; the exponential is *memoryless*, $\mathbb{P}(T > s + t \mid T > s) = \mathbb{P}(T > t)$ (rate 0.5, $t = 2$: 0.3679); Poisson is the limit of Binomial($n, \lambda/n$); sums of independent normals are normal.

§14.2 Estimators and confidence intervals

Prototypical example for this section: Ten measurements of a delivery time: give a 95% interval for the mean.

An **estimator** $\hat{\theta}$ has bias $\mathbb{E}\hat{\theta} - \theta$, variance, and $\text{MSE} = \text{bias}^2 + \text{variance}$ (Chapter 4). The sample mean is unbiased with standard error $\sigma/\sqrt{n}$; the sample variance with divisor $n - 1$ is unbiased (Chapter 2).

Definition 14.2.1. A 95% **confidence interval** is produced by a procedure that, over repeated samples, covers the true parameter 95% of the time. For a mean with unknown σ : $\bar{x} \pm t_{0.975, n-1} s/\sqrt{n}$.

Example 14.2.2 (Mean)

$n = 10$, $\bar{x} = 12.310$, $s = 0.5953$, $\text{SE} = 0.1882$, $t_{0.975, 9} = 2.262$ (not 1.960: small sample): interval $[11.884, 12.736]$, as `scipy.stats.t.interval` gives.

Example 14.2.3 (Proportion and bootstrap)

120 clicks in 400 impressions: $\hat{p} = 0.30$, $SE = \sqrt{\hat{p}(1 - \hat{p})/n} = 0.0229$, Wald interval $[0.2551, 0.3449]$. For statistics without a formula (a median, a ratio, AUC) use the **bootstrap**: resample with replacement, recompute, take percentiles. For the median of 60 skewed values (7.59) the 95% percentile interval is about $[5.38, 10.25]$, agreeing with `scipy.stats.bootstrap`.

Remark 14.2.4 — Trap. “There is a 95% probability that the true mean lies in $[11.88, 12.74]$ ” is the classic wrong reading: the parameter is fixed and this particular interval either contains it or not; the 95% describes the procedure. (A Bayesian credible interval does admit the probability statement, under its prior.)

§14.3 Hypothesis tests

Prototypical example for this section: A new checkout page: did conversion really go up, or is it noise?

§14.3.i The vocabulary

Null hypothesis H_0 (no effect) versus alternative H_1 . Choose a test statistic whose distribution under H_0 is known. The **p-value** is the probability, *assuming* H_0 , of a statistic at least as extreme as the observed one. Reject H_0 if $p < \alpha$.

	H_0 true	H_0 false
reject	Type I error (rate α)	correct (power $1 - \beta$)
do not reject	correct	Type II error (rate β)

Power rises with effect size, sample size and α , and falls with noise.

Remark 14.3.1 — Trap. A p-value is *not* $\mathbb{P}(H_0 \text{ true} \mid \text{data})$, not the probability the result is due to chance, and not an effect size. $p = 0.03$ with $n = 10^6$ can be a negligible effect; $p = 0.2$ with $n = 20$ can hide a large one. “Not significant” is not “no effect”. Fix α before looking, and do not stop an A/B test the moment p dips below 0.05 (peeking inflates Type I error).

§14.3.ii z and t

Example 14.3.2 (z -test, known σ)

Mean load time should be 50 ms; $n = 64$ requests average 52 ms with known $\sigma = 8$: $z = \frac{52-50}{8/\sqrt{64}} = 2.00$, two-sided $p = 0.0455$. Reject at 5%. The power of this test against a true mean of 52 is only 0.5160.

- **One-sample t** (σ unknown): $t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$, $n - 1$ d.f. For the delivery times against $\mu_0 = 12$: $t = 1.647$, $p = 0.1340$.
- **Two-sample Welch t** (unequal variances, the safe default): $t = \frac{\bar{a} - \bar{b}}{\sqrt{s_a^2/n_a + s_b^2/n_b}}$ with Welch–Satterthwaite d.f. Scores $\bar{a} = 83.29$ vs $\bar{b} = 76.00$: $t = 3.084$, d.f. 11.31, $p = 0.0101$.

- **Paired t :** the same units measured twice; test the differences. Six students before and after coaching: $t = 5.937$, $p = 0.00194$. Treating the same data as two independent samples gives $p = 0.6320$: the between-student variation swamps the consistent small gains.

Each statistic and p-value above is recomputed by hand in `code/ml/c14_stats.py` and checked against `ttest_1samp`, `ttest_ind(equal_var=False)` and `ttest_rel`. Use z when σ is known or n is large; non-parametric alternatives (Mann–Whitney, Wilcoxon signed-rank) when normality is doubtful and n small.

§14.3.iii A/B test sample size

For two proportions $p_1 \rightarrow p_2$ at significance α (two-sided) and power $1 - \beta$:

$$n \approx \frac{\left(z_{1-\alpha/2} \sqrt{2\bar{p}(1-\bar{p})} + z_{1-\beta} \sqrt{p_1(1-p_1) + p_2(1-p_2)} \right)^2}{(p_2 - p_1)^2} \quad \text{per group.}$$

Detecting a lift from 10% to 12% at $\alpha = 0.05$, power 0.8 needs 3841 users per arm; simulating 4000 such experiments rejects H_0 in 0.800 of them. Halving the detectable lift roughly quadruples n .

§14.4 Chi-square and ANOVA

Prototypical example for this section: Does conversion depend on the app version?

Example 14.4.1 (Test of independence)

Version A: 30 of 100 converted; version B: 45 of 100. Expected counts under independence (row total $\times$ column total / grand total, row by row): 37.5, 62.5, 37.5, 62.5. $\chi^2 = \sum \frac{(O-E)^2}{E} = 4.8000$ on $(2-1)(2-1) = 1$ d.f., $p = 0.0285$ (with Yates' continuity correction, scipy's default for 2×2 : $p = 0.0409$). Goodness of fit: a die rolled 60 times with counts 8, 12, 9, 11, 6, 14 gives $\chi^2 = 4.20$ on 5 d.f., $p = 0.5210$ — no evidence of bias. Rule of thumb: expected counts ≥ 5 , else Fisher's exact test.

Example 14.4.2 (One-way ANOVA)

Three groups (5, 6, 7, 6), (8, 9, 7, 8), (6, 5, 6, 7). Between-group sum of squares 10.667 (2 d.f.), within-group 6.000 (9 d.f.), $F = \frac{10.667/2}{6/9} = 8.000$, $p = 0.0101$ (matches `f_oneway`). ANOVA says *some* mean differs; post-hoc tests (Tukey) say which.

§14.5 Multiple testing

Prototypical example for this section: Twenty metrics on a dashboard, each tested at 5%.

With 20 independent true nulls, $\mathbb{P}(\text{at least one false positive}) = 1 - 0.95^{20} = 0.6415$. **Bonferroni** controls this family-wise error by testing each at α/m ; it is conservative. **Benjamini–Hochberg** controls the *false discovery rate*: sort p-values, find the largest k with $p_{(k)} \leq \frac{k}{m}\alpha$, reject the k smallest. For the eight sorted p-values

0.001, 0.008, 0.012, 0.030, 0.041, 0.20, 0.35, 0.60,

unadjusted testing rejects 5, Bonferroni ($0.05/8$) rejects 1, and BH, whose thresholds $\frac{k}{8} \cdot 0.05$ are 0.00625, 0.0125, 0.01875, 0.025, 0.03125, 0.0375, 0.04375, 0.05, rejects 3 (matching `false_discovery_control`). The same logic applies to trying many features, models or hyper-parameters on one validation set.

§14.6 Correlation, causation and Simpson

Prototypical example for this section: A treatment better in every subgroup, yet worse overall.

Covariance carries units and scale; correlation is scale-free and measures *linear* association (Chapter 2). Pearson's r is sensitive to outliers; Spearman's ρ (Pearson on ranks) measures monotone association and is robust: nine points on a rising line plus one wild outlier give Pearson -0.251 but Spearman 0.455 .

Example 14.6.1 (Simpson's paradox)

In the classic kidney-stone data, treatment A succeeds in 0.931 of small-stone cases and 0.730 of large-stone cases; treatment B in 0.867 and 0.688. A is better in both groups, yet overall A succeeds in 0.780 versus B's 0.826: A was given mostly to the harder, large-stone cases. Stone size *confounds* the comparison; aggregate only after asking what drives treatment assignment.

Remark 14.6.2 — Trap. Correlation does not imply causation: confounders (ice-cream sales and drownings both follow summer), reverse causation, selection effects. In ML terms: a feature can be highly predictive without being a lever you can pull. Causal questions need randomised experiments or explicit causal assumptions.

State H_0 , pick the test that matches the data's structure (paired? proportions? counts?), report effect size and interval alongside p , and correct for how many things you tested.

§14.7 A few problems to think about

Problem 14A (Poisson). A server gets on average 3 errors per hour. Probability of no error in an hour? Of at least 2?

[hint · solution](#)

Problem 14B (Which test?). Name the test: (a) average spend of the same 50 users before and after a redesign; (b) click-through of two independent ad variants; (c) whether city and preferred payment method are related; (d) mean delivery time across four warehouses.

[hint · solution](#)

Problem 14C (t statistic). Sample of 16 response times: mean 210 ms, standard deviation 40 ms. Test $H_0 : \mu = 200$ two-sided at 5%.

[hint · solution](#)

Problem 14D (Sample size intuition). An A/B test powered to detect a 2-point lift (10% to 12%) needs about 3840 users per arm. Roughly how many for a 1-point lift (10% to 11%)? Why?

[hint](#) · [solution](#)

15 Data Preparation and Feature Engineering

In practice, more model quality comes from features and clean data than from the choice of algorithm, and most production bugs live here. OA papers test the crisp parts (which models need scaling, target versus one-hot encoding, IDF arithmetic, missing values); interviews test judgement (“this feature is 40% missing, what do you do?”).

§15.1 Scaling

Prototypical example for this section: Wine data: k -NN at 66% accuracy before standardising, 96% after.

- **Standardisation** $(x - \mu)/\sigma$: zero mean, unit variance; the default.
- **Min-max** $(x - \min)/(\max - \min)$ to $[0, 1]$: bounded inputs (pixels, some neural nets); one outlier squashes everything else.
- **Robust** $(x - \text{median})/\text{IQR}$: resistant to outliers.

For the column 10, 12, 11, 13, 12, 100:

- standardised: $-0.50, -0.43, -0.47, -0.40, -0.43, 2.24$;
- min-max: $0.00, 0.02, 0.01, 0.03, 0.02, 1.00$ — the five ordinary values are crushed near 0;
- robust: $-1.33, 0.00, -0.67, 0.67, 0.00, 58.67$ — they stay spread and the outlier is obviously extreme.

Log or Box-Cox transforms handle skew before scaling.

Example 15.1.1 (Who needs scaling)

5-fold accuracy on the wine data, raw versus standardised inside a pipeline:

model	raw	standardised
k -NN ($k = 5$)	0.663	0.961
SVM (RBF)	0.657	0.983
logistic regression (L2)	0.950	0.983
PCA(2) + logistic	0.674	0.961
decision tree	0.927	0.927
random forest	0.972	0.972

Distance- and margin-based models (k -NN, SVM, k -means), variance-based PCA, and penalised or gradient-trained linear models depend on scale (logistic regression also needed far more iterations unscaled). Trees and tree ensembles split on thresholds and are invariant to any monotone rescaling. Unpenalised OLS gives the same predictions either way.

§15.2 Encoding categoricals

Prototypical example for this section: A “city” column with 1500 distinct values and a churn label.

- **One-hot**: one binary column per level (drop one to avoid the dummy-variable trap in unpenalised linear models). Safe and interpretable; dimension explodes with cardinality and rare levels are noisy.
- **Ordinal/label encoding**: integers $0, 1, 2, \dots$. Fine for trees and for genuinely ordered categories (small < medium < large); for linear models it invents a spurious order and spacing.
- **Frequency encoding, feature hashing** (hash levels into a fixed number of columns; collisions are the price of bounded memory, useful for huge or streaming vocabularies), learned **embeddings** (neural nets).
- **Target (mean) encoding**: replace a level by a (smoothed) mean of the target over rows with that level — one dense column regardless of cardinality.

§15.2.i Target encoding and its leakage

With n_c rows in level c , level mean $\bar{y}_c$, global mean $\bar{y}$ and smoothing m :

$$\text{TE}(c) = \frac{n_c \bar{y}_c + m \bar{y}}{n_c + m},$$

a shrinkage towards the global mean that is strong for rare levels (the Beta-prior estimate of [Chapter 2](#) again). A level seen 3 times, always positive, with $\bar{y} = 0.280$ and $m = 10$, encodes to 0.4462, not 1 (matches scikit-learn’s `TargetEncoder(smooth=10)`).

Example 15.2.1 (The leak, measured)

4000 rows; the label depends only on a numeric feature x ; a 1500-level “ID-like” category is pure noise. Logistic regression on x alone: test AUC 0.750. Add the category, target-encoded with means computed on the full training set (each row’s own label included): *training* AUC 0.946, coefficient on the encoding 7.02, *test* AUC 0.585 — worse than ignoring the column. With levels of about three rows, the encoding largely *is* the row’s own label. Cross-fitted (out-of-fold) encoding, as `TargetEncoder.fit_transform` does, gives training AUC 0.738, test AUC 0.750 and a near-zero coefficient (−0.22): the noise is correctly ignored.

Remark 15.2.2 — Trap. “Target encoding always beats one-hot for high cardinality.” Only if done out-of-fold (or with CatBoost’s ordered statistics) and with smoothing; naive target encoding is a textbook leak that inflates validation scores and hurts production. One-hot plus regularisation is the safe baseline; for trees, ordinal or native categorical handling is often enough.

§15.3 Missing values and outliers

Prototypical example for this section: 28% of applicants skip the income question — mostly high earners.

Missingness mechanisms: **MCAR** (completely at random: deleting rows only costs data), **MAR** (depends on observed features: model-based imputation works), **MNAR** (depends on the missing value itself: the fact of missingness is information). Options: drop rows (small, MCAR), drop columns (mostly empty and uninformative), mean/median/mode imputation (fast, shrinks variance and distorts correlations), k -NN or iterative model-based imputation, a *missing-indicator* column, or models that handle NaN natively (XGBoost, LightGBM, recent scikit-learn trees). Fit imputers on training folds only.

Example 15.3.1 (Missingness is a feature)

Income missing for 0.284 of applicants, more often for high earners; the target rises with income. Mean imputation alone: test AUC 0.710 (and the income standard deviation drops from 14.87 to 10.05). Mean imputation plus an `is_missing` indicator: 0.808.

Outliers: flag with the IQR rule (outside $[Q_1 - 1.5 \text{ IQR}, Q_3 + 1.5 \text{ IQR}]$; for the column above $Q_1 = 11.25$, $Q_3 = 12.75$, upper fence 15.00, so 100 is flagged), z -scores (fragile: the outlier inflates σ), or isolation forests ([Chapter 11](#)). Then decide: data error (fix or drop), genuine extreme (keep; use robust losses, log transforms or winsorising), or the signal itself (fraud).

§15.4 Feature selection

Prototypical example for this section: 50 features, 5 informative: which methods find them?

Filter methods score features one at a time (correlation, ANOVA F , mutual information, χ^2): fast, ignore interactions and redundancy. **Wrapper** methods search subsets using a model (forward/backward selection, recursive feature elimination): expensive, can overfit the validation data. **Embedded** methods select while fitting (lasso, tree importances). On 600 rows with 5 informative of 50 features: mutual information picks 3 of the 5 in its top 5, RFE with logistic regression 5 of 5, and an ℓ_1 logistic regression keeps 8 features including 4 true ones. Any selection that looks at labels must run inside the CV loop ([Chapter 13](#)).

§15.5 Imbalance

Prototypical example for this section: 5% positives; SMOTE before cross-validation reports $F1$ 0.94.

Options, in order of preference: choose the metric and threshold properly (PR curves, cost-based threshold, [Chapter 5](#)); class weights (`class_weight="balanced"`, `scale_pos_weight`); resampling — random under-sampling of the majority, random over-sampling, or **SMOTE** (synthetic minority points interpolated between a minority point and one of its minority neighbours):

code/ml/c15_dataprep.py: SMOTE in ten lines

```
1 def smote(X, y, k=5, ratio=1.0, seed=0):
2     """Add synthetic minority points on segments between a minority point and one
3     ↪ of its k minority neighbours."""
4     r = np.random.default_rng(seed)
```

```

4  Xm = X[y == 1]
5  n_new = int(ratio * ((y == 0).sum() - len(Xm)))
6  nn = NearestNeighbors(n_neighbors=k + 1).fit(Xm).kneighbors(Xm, return_distance
    ↪ =False)[: , 1:]
7  i = r.integers(0, len(Xm), n_new)
8  j = nn[i, r.integers(0, k, n_new)]
9  lam = r.random((n_new, 1))
10 Xs = Xm[i] + lam * (Xm[j] - Xm[i])
11 return np.vstack([X, Xs]), np.r_[y, np.ones(n_new, int)]

```

Example 15.5.1 (SMOTE leakage)

1000 rows, 5% positives, 3-NN classifier. SMOTE the whole dataset, then cross-validate: F1 0.936. SMOTE only inside each training fold, evaluate on untouched folds: F1 0.330. In the leaky version synthetic points sit in the validation folds between their own parents in the training folds, and 3-NN finds them trivially.

Remark 15.5.2 — Trap. Resample *only the training data*, never the validation or test data (their class balance must match deployment), and remember resampling changes predicted probabilities. SMOTE also interpolates in feature space, which is meaningless for categorical features and risky in high dimensions.

§15.6 Text features and TF-IDF

Prototypical example for this section: Three short documents; why does “the” get a low weight and “cat” a high one?

Bag-of-words counts word occurrences (optionally n -grams). **TF-IDF** reweights a term’s count in a document by its rarity across the corpus:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t), \quad \text{idf}(t) = \log \frac{N}{\text{df}(t)} \quad (\text{textbook}),$$

with N documents and $\text{df}(t)$ the number containing t . scikit-learn’s default is the smoothed $\text{idf}(t) = \ln \frac{1+N}{1+\text{df}(t)} + 1$, followed by L2-normalising each document vector.

Example 15.6.1 (IDF by hand)

Documents: “the cat sat on the mat”, “the dog sat on the log”, “cats and dogs” ($N = 3$). “the” appears in 2 documents: textbook IDF $\ln(3/2) = 0.4055$, smoothed $\ln(4/3) + 1 = 1.2877$. “cat” appears in 1: $\ln 3 = 1.0986$, smoothed $\ln 2 + 1 = 1.6931$ (note “cats” is a different token without stemming). In document 1, “the” occurs twice: unnormalised tf-idf $2 \times 1.2877 = 2.5754$ against 1.6931 for “cat”. The smoothed IDFs and the normalised matrix match `TfidfVectorizer`. With $\log_{10}$ the textbook IDF of “cat” would be 0.4771: always state the base.

TF-IDF vectors are sparse and high-dimensional, compared with cosine similarity and fed to linear models or naive Bayes; they ignore word order beyond n -grams and know no synonyms — the gap embeddings fill.

§15.7 Time series in one page

Prototypical example for this section: Forecast tomorrow's orders from the last 30 days.

A time series is not i.i.d.: values are autocorrelated, may trend and have seasonality. **Stationarity** (constant mean, variance and autocorrelation over time) is assumed by classical models; differencing $y_t - y_{t-1}$ removes trends, seasonal differencing $y_t - y_{t-s}$ removes seasonality, and the augmented Dickey–Fuller test checks for a unit root. **ARIMA**(p, d, q) = autoregression on p lags + d differences + a moving average of q past errors; read p from the partial autocorrelation and q from the autocorrelation plots. For ML models, build lag features, rolling statistics (computed only from the past), calendar features and holiday flags, then validate with forward-chaining splits (Chapter 13); remember tree ensembles cannot extrapolate a trend, so model differences or detrend first.

§15.8 Recommenders in one page

Prototypical example for this section: Five users, four items, a sparse rating matrix: predict the blanks.

Content-based recommenders use item features; **collaborative filtering** uses the rating matrix itself: neighbourhood methods (item–item cosine similarity of rating columns) or **matrix factorisation** $R \approx \mathbf{U}\mathbf{V}^\top$ with k -dimensional user and item factors, fitted on observed entries only [KoBeVo09]:

$$\min_{\mathbf{U}, \mathbf{V}} \sum_{(u,i) \text{ observed}} (r_{ui} - \mathbf{u}_u^\top \mathbf{v}_i)^2 + \lambda(\|\mathbf{U}\|_F^2 + \|\mathbf{V}\|_F^2).$$

Alternating least squares fixes $\mathbf{V}$ and solves a ridge regression for each user, then swaps. On a 5×4 toy matrix ($k = 2$, $\lambda = 0.1$) each user update equals **Ridge** on that user's ratings, the regularised objective decreases every sweep, the training RMSE reaches 0.048, and the model predicts 1.92 for user 1's unseen item 3. Practicalities: biases per user and item, implicit feedback (clicks, not ratings), the cold-start problem for new users/items (fall back to content or popularity), and evaluation by ranking metrics (precision@ k , NDCG) on time-split data.

Fit every transform inside the training fold; scale for distances and penalties, not for trees; treat missingness and rare categories as information to be estimated carefully, not noise to be papered over.

§15.9 A few problems to think about

Problem 15A (IDF). A corpus has 1000 documents; “refund” appears in 10, “the” in 990. Compute textbook IDF (natural log) for both, and the tf-idf of “refund” in a document where it occurs 3 times.

hint · solution

Problem 15B (Smoothed target encoding). Global conversion rate 5%. Pincode A has 2 visits and 1 conversion; pincode B has 400 visits and 60. Compute smoothed encodings with $m = 20$.

hint · solution

Problem 15C (Scaling, true or false). (a) Standardising features changes a random forest's predictions. (b) k -means results depend on feature units. (c) PCA should be run on standardised data when features have different units. (d) Gradient descent on linear regression converges faster after standardising. (e) The scaler should be fitted on train and test together for consistency.

[hint](#) · [solution](#)



16 Optimisation for ML

Every model in this book is fitted by minimising something. This short chapter collects the facts about *how* that you need for follow-up questions: why gradient descent zig-zags, what momentum buys, when Newton or L-BFGS is the right tool, and why SGD needs a decaying step. Deep-learning optimisers (Adam and friends) are covered in *Deep Learning*.

§16.1 Gradient descent on a quadratic

Prototypical example for this section: Minimise $\frac{1}{2}(x^2 + 10y^2)$ starting from $(1, 1)$.

Near a minimum every smooth loss looks quadratic, $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x}$ with $\mathbf{A} \succ 0$ having eigenvalues $\mu = \lambda_{\min} \leq \dots \leq L = \lambda_{\max}$. Gradient descent $\mathbf{x} \leftarrow \mathbf{x} - \eta \mathbf{A}\mathbf{x}$ multiplies the component along eigenvector i by $1 - \eta\lambda_i$ each step (Chapter 3, Problem 3F). Hence:

- it converges iff $0 < \eta < 2/L$; at $\eta = 0.21 > 2/L = 0.20$ the iterate's norm after 20 steps is 6.7 and growing;
- the best constant step $\eta = 2/(\mu + L)$ contracts the error by $\frac{\kappa-1}{\kappa+1}$ per step, where $\kappa = L/\mu$ is the **condition number** (the script confirms the ratio 9/11 for $\kappa = 10$);
- the number of iterations to a fixed accuracy grows linearly in κ .

Geometrically, with $\kappa \gg 1$ the level sets are long thin ellipses; the gradient points across the valley, not along it, so the iterates bounce between the walls (Figure 16.1). Standardising features and preconditioning reduce κ .

§16.2 Momentum

Prototypical example for this section: A ball rolling down the valley keeps its speed along the floor and damps the side-to-side bounce.

Heavy-ball momentum keeps a velocity: $\mathbf{v} \leftarrow \beta\mathbf{v} - \eta\nabla f(\mathbf{x})$, $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}$. Oscillating gradient components cancel in the running sum; consistent ones accumulate. With optimally tuned η, β on a quadratic the contraction factor improves from $\frac{\kappa-1}{\kappa+1}$ to $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$: iterations scale with $\sqrt{\kappa}$ instead of κ . Iterations to reach $\|\mathbf{x}\| < 10^{-6}$ from $(1, 1)$:

κ	GD, $\eta = 1/L$	GD, $\eta = \frac{2}{\mu+L}$	heavy ball	GD rate	momentum rate
10	132	71	27	0.8182	0.5195
100	1375	709	95	0.9802	0.8182
1000	13809	7082	321	0.9980	0.9387

Nesterov's accelerated gradient evaluates the gradient at the look-ahead point $\mathbf{x} + \beta\mathbf{v}$ and attains the $\sqrt{\kappa}$ rate for general smooth convex functions.

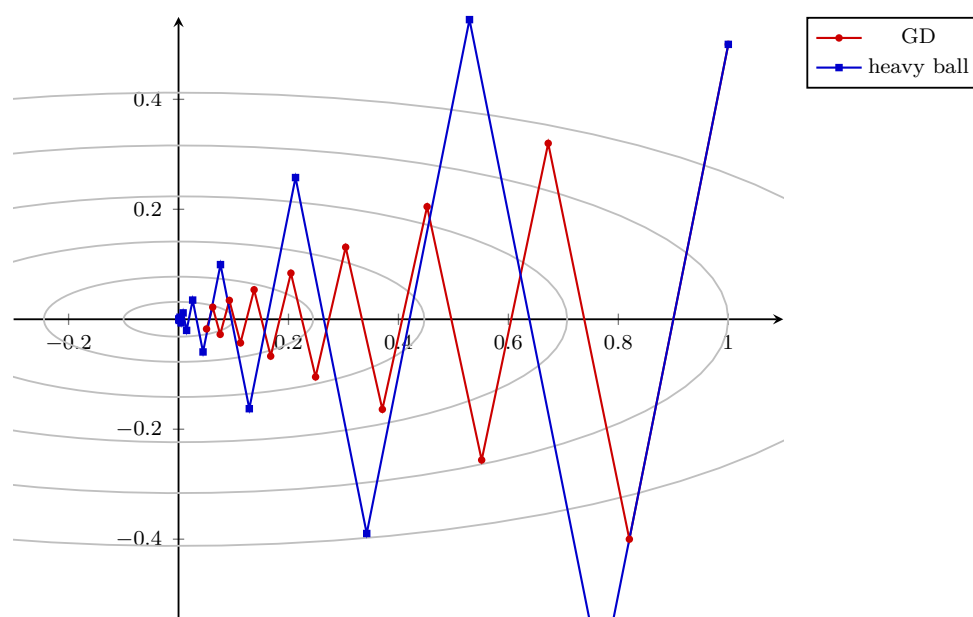


Figure 16.1: $f = \frac{1}{2}(x^2 + 10y^2)$, $\kappa = 10$, 15 steps from $(1, 0.5)$: gradient descent ($\eta = 2/(\mu + L)$) zig-zags across the narrow direction and ends at distance 0.0539 from the optimum; tuned heavy-ball momentum ends at 0.0008.

§16.3 Newton and quasi-Newton

Prototypical example for this section: One Newton step solves a quadratic exactly.

Newton's method uses curvature: $\mathbf{x} \leftarrow \mathbf{x} - \mathbf{H}^{-1} \nabla f$. On a quadratic it jumps to the minimiser in one step (checked); near a minimum of a smooth function it converges *quadratically* (the number of correct digits doubles each step), and it is invariant to linear rescaling of the parameters, so conditioning does not matter. Costs: forming and solving with the $d \times d$ Hessian, $O(d^3)$ per step; and far from the minimum, or where $\mathbf{H}$ is not PD, it can go the wrong way — hence damping and line searches. Logistic regression's IRLS (Chapter 5) is Newton's method.

Quasi-Newton methods build a Hessian (inverse) approximation from successive gradient differences: BFGS keeps a dense $d \times d$ matrix; **L-BFGS** keeps only the last m (say 10) pairs and costs $O(md)$ per step — the default solver for scikit-learn's logistic regression. On the Rosenbrock function (a curved, ill-conditioned valley) from $(-1.2, 1)$, plain gradient descent with step 10^{-3} needs 26312 iterations to get the gradient norm below 10^{-5} ; BFGS needs 32, L-BFGS 36 and Newton-CG 83.

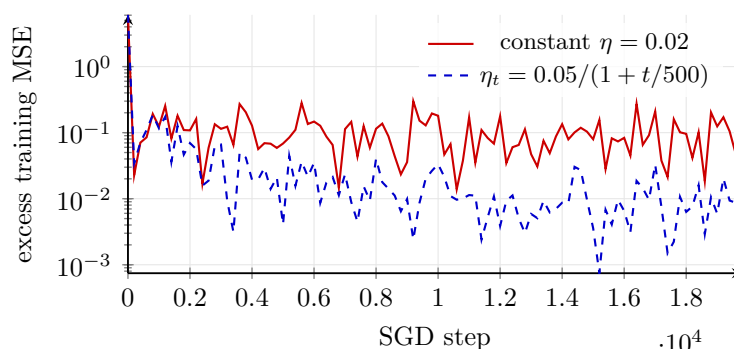
§16.4 Coordinate descent, SGD and duality

Prototypical example for this section: One data point per update: fast, noisy, and never quite settles at a constant step.

Coordinate descent minimises over one coordinate at a time; it shines when each one-dimensional problem has a closed form, as for the lasso (Chapter 4) and SMO's two-variable steps (Chapter 8).

Stochastic gradient descent replaces ∇f by the gradient on one example or a mini-batch: an unbiased but noisy estimate, n times cheaper per step. With a constant step, the iterates reach a noise ball around the optimum whose size is proportional to η ;

convergence requires a decaying step ($\sum \eta_t = \infty$, $\sum \eta_t^2 < \infty$, e.g. $\eta_t \propto 1/t$) or averaging of iterates.



Linear regression with 2000 rows: the constant step plateaus at excess MSE ≈ 0.0986 ; the decaying step keeps improving (0.0091). Rates to remember for convex problems: GD on smooth convex $O(1/t)$, on strongly convex $O(c^t)$ (linear); SGD $O(1/\sqrt{t})$ and $O(1/t)$ respectively.

Duality (Chapter 2) turns a constrained primal into a dual whose value lower-bounds it; for convex problems with Slater's condition they coincide. Practical uses: the SVM dual exposes kernels and sparsity; the *duality gap* is a certificate of how far an iterate is from optimal, which is exactly the stopping test scikit-learn's lasso and SVM solvers use.

Condition number sets gradient descent's speed; momentum turns κ into $\sqrt{\kappa}$; curvature (Newton, L-BFGS) removes it at a price per step; SGD trades exactness for cheap steps and needs a decaying rate.

§16.5 A few problems to think about

Problem 16A (Largest step). For $f(x, y) = 2x^2 + 8y^2$, what is the largest constant learning rate for which gradient descent converges, and what is the condition number?

[hint · solution](#)

Problem 16B (Newton in one dimension). Apply two Newton steps to minimise $f(x) = x^4 - 3x$ from $x_0 = 1$.

[hint · solution](#)



Interview Mode

Part V: Contents

17	Implement It	135
17.1	Linear models	135
17.2	Neighbours and Bayes	136
17.3	Trees and forests	136
17.4	Clustering and PCA	138
17.5	SVM	139
18	Rapid-fire	141
18.1	Rapid-fire answers	141
19	Problems	143
19.1	Problems	143

17 Implement It

“Implement k -means / a decision tree / logistic regression in NumPy, no libraries” is a standard 30–45 minute round. Interviewers watch for three things: a correct core loop, vectorised NumPy instead of nested Python loops where it matters, and awareness of edge cases (empty clusters, ties, numerical stability, the intercept). Every listing below lives in `code/ml/c17_implement.py` and is asserted against scikit-learn on real data; the largest absolute difference found is in the last column.

from-scratch implementation	max ours – scikit-learn
linear regression (normal equations)	1.1e−13
ridge regression (closed form)	1.4e−14
logistic regression (gradient descent)	1.7e−09
k -nearest neighbours	0.0e+00
Gaussian naive Bayes	3.3e−16
decision tree (CART, Gini, depth 4)	0.0e+00
random forest (bagged trees, same bootstrap draws)	0.0e+00
AdaBoost (10 stumps)	1.3e−15
k -means (Lloyd)	5.3e−15
PCA (SVD)	2.7e−14
soft-margin linear SVM (SMO)	5.7e−07
lasso (coordinate descent) [Ch. 4]	see chapter
logistic regression (Newton/IRLS) [Ch. 5]	see chapter
softmax regression [Ch. 5]	see chapter
gradient boosting (squared and log loss) [Ch. 10]	see chapter
Gaussian mixture (EM) [Ch. 11]	see chapter
label propagation [Ch. 12]	see chapter
SMOTE [Ch. 15]	see chapter

§17.1 Linear models

Prototypical example for this section: Ridge in five lines; logistic regression in eight.

linear and ridge regression

```

1 def fit_linear(X, y, lam=0.0):
2     """OLS (lam=0) or ridge; the intercept is not penalised (fit on centred data).
   ↪ """
3     xm, ym = X.mean(0), y.mean()
4     Xc, yc = X - xm, y - ym
5     w = np.linalg.solve(Xc.T @ Xc + lam * np.eye(X.shape[1]), Xc.T @ yc)
6     return w, ym - xm @ w

```

Centre, solve, recover the intercept: the intercept is unpenalised and you never invert a matrix. Say why `solve` beats `inv` (stability, cost) and when you would switch to gradient descent (d large, streaming data).

logistic regression by batch gradient descent

```

1 def fit_logistic(X, y, lr=0.5, iters=5000):
2     Xb = np.c_[np.ones(len(X)), X]

```

```

3     w = np.zeros(Xb.shape[1])
4     for _ in range(iters):
5         p = 1 / (1 + np.exp(-Xb @ w))
6         w -= lr * Xb.T @ (p - y) / len(y)      # gradient of the mean log-loss
7     return w

```

Standardise inputs first or the step size must be tiny. For production code, compute the sigmoid stably (`np.exp(-np.logaddexp(0, -z))`) and the loss with log-sum-exp; with separable data and no penalty the weights diverge (Chapter 5), which is why this check uses noisy labels. Newton/IRLS and softmax regression are in Chapter 5, the lasso by coordinate descent in Chapter 4.

§17.2 Neighbours and Bayes

Prototypical example for this section: Broadcasting the full test-by-train distance matrix in one line.

k-nearest neighbours

```

1 def knn_predict(Xtr, ytr, Xte, k=5):
2     d = ((Xte[:, None, :] - Xtr[None, :, :]) ** 2).sum(-1)      # (n_test, n_train)
3     ↪ squared distances
4     idx = np.argsort(d, axis=1, kind="stable")[:, :k]
5     return np.array([np.bincount(ytr[r]).argmax() for r in idx])

```

The broadcast builds an $n_{\text{test}} \times n_{\text{train}} \times d$ array; for big data use $\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\mathbf{a}^\top \mathbf{b}$ (one matrix product) and process the test set in chunks, or `np.argpartition` instead of a full sort. A stable sort makes ties reproducible.

Gaussian naive Bayes

```

1 def gnb_fit_predict_proba(Xtr, ytr, Xte, eps=1e-9):
2     classes = np.unique(ytr)
3     eps = eps * Xtr.var(0).max()      # sklearn's
4     ↪ var_smoothing convention
5     logp = []
6     for c in classes:
7         Xk = Xtr[ytr == c]
8         mu, var = Xk.mean(0), Xk.var(0) + eps
9         ll = -0.5 * (np.log(2 * np.pi * var) + (Xte - mu) ** 2 / var).sum(1)
10        logp.append(np.log(len(Xk) / len(Xtr)) + ll)
11    L = np.array(logp).T
12    L -= L.max(1, keepdims=True)
13    P = np.exp(L)
14    return P / P.sum(1, keepdims=True)

```

Work in log space and subtract the row maximum before exponentiating: some posteriors here are 10^{-233} . The tiny variance floor (`var_smoothing`) prevents division by zero for constant features.

§17.3 Trees and forests

Prototypical example for this section: CART with weighted Gini, reused unchanged as the forest's and AdaBoost's base learner.

CART classification tree

```

1 def gini(counts):
2     p = counts / counts.sum()
3     return 1 - (p ** 2).sum()
4
5 def build_tree(X, y, w, depth, max_depth, n_classes):
6     """w = sample weights (bootstrap counts for forests). Leaves store class
7     ↪ weights."""
8     counts = np.bincount(y, weights=w, minlength=n_classes)
9     if depth == max_depth or gini(counts) == 0:
10         return {"leaf": counts / counts.sum()}
11     best = None
12     for j in range(X.shape[1]):
13         order = np.argsort(X[:, j], kind="stable")
14         xs, ys, ws = X[order, j], y[order], w[order]
15         left = np.cumsum(np.eye(n_classes)[ys] * ws[:, None], axis=0) # class
16         ↪ weights left of each cut
17         total = left[-1]
18         for i in np.where(xs[1:] > xs[:-1])[0]: # cut
19             ↪ between distinct values only
20             L, R = left[i], total - left[i]
21             score = (L.sum() * gini(L) + R.sum() * gini(R)) / total.sum()
22             if best is None or score < best[0] - 1e-12:
23                 best = (score, j, (xs[i] + xs[i + 1]) / 2)
24     if best is None or best[0] >= gini(counts) - 1e-12:
25         return {"leaf": counts / counts.sum()}
26     _, j, t = best
27     m = X[:, j] <= t
28     return {"j": j, "t": t,
29           "l": build_tree(X[m], y[m], w[m], depth + 1, max_depth, n_classes),
30           "r": build_tree(X[~m], y[~m], w[~m], depth + 1, max_depth, n_classes)}
31
32 def tree_proba(node, x):
33     while "leaf" not in node:
34         node = node["l"] if x[node["j"]] <= node["t"] else node["r"]
35     return node["leaf"]

```

Points to make while writing it: sort each feature once per node and sweep cumulative class counts, so each feature costs $O(n \log n)$ rather than $O(n^2)$; only cut between distinct values; stop on purity, depth, or no improvement; store class proportions in leaves so the same tree gives probabilities. Supporting sample weights costs one extra argument and buys both bagging and boosting. It reproduces `DecisionTreeClassifier` exactly when no two splits tie (scikit-learn breaks ties by a random feature order).

random forest (bagging mode)

```

1 def forest_proba(Xtr, ytr, Xte, bootstrap_indices, max_depth):
2     P = 0
3     for idx in bootstrap_indices:
4         w = np.bincount(idx, minlength=len(ytr)).astype(float) # bootstrap =
5         ↪ integer sample weights
6         keep = w > 0
7         T = build_tree(Xtr[keep], ytr[keep], w[keep], 0, max_depth, 2)
8         P = P + np.array([tree_proba(T, x) for x in Xte])
9     return P / len(bootstrap_indices)

```

Given scikit-learn's own bootstrap draws (`estimators_samples_`), our averaged trees reproduce `RandomForestClassifier(max_features=None)`'s probabilities to the last

bit. A real forest additionally draws `max_features` candidate features at each split; add `jinrng.choice(d,m,replace=False)` to the split loop.

AdaBoost (SAMME, two classes)

```

1 def adaboost(X, y, M):
2     """y in {-1, +1}. Stumps are our CART trees of depth 1 fitted with sample
   ↪ weights."""
3     w = np.full(len(y), 1 / len(y)); stumps, alphas = [], []
4     yy = (y > 0).astype(int)
5     for _ in range(M):
6         T = build_tree(X, yy, w, 0, 1, 2)
7         pred = np.where([tree_proba(T, x)[1] > 0.5 for x in X], 1, -1)
8         err = w[pred != y].sum() / w.sum()
9         a = np.log((1 - err) / err)           # SAMME weight (= 2 x the
   ↪ 1/2-log convention)
10        w = w * np.exp(a * (pred != y)); w /= w.sum()
11        stumps.append(T); alphas.append(a)
12    return stumps, np.array(alphas)

```

Ten rounds give the same estimator weights and predictions as `AdaBoostClassifier`. Gradient boosting for squared and log loss is in [Chapter 10](#).

§17.4 Clustering and PCA

Prototypical example for this section: k-means in eight lines; PCA in four.

k-means (Lloyd)

```

1 def kmeans(X, C, iters=100):
2     for _ in range(iters):
3         lab = ((X[:, None] - C[None]) ** 2).sum(-1).argmin(1)
4         Cn = np.array([X[lab == k].mean(0) for k in range(len(C))])
5         if np.allclose(Cn, C):
6             break
7         C = Cn
8     return C, lab

```

Mention: *k*-means++ seeding, several restarts, and the empty-cluster edge case (a centroid with no points makes `mean` return `NaN`; re-seed it at the farthest point or keep the old centroid).

PCA via the SVD

```

1 def pca(X, k):
2     Xc = X - X.mean(0)
3     U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
4     comps = Vt[:k] * np.sign(Vt[:k, [0]])           # fix the sign convention
5     return Xc @ comps.T, comps, s[:k] ** 2 / (len(X) - 1)

```

Centre first; use the SVD of the data rather than the eigendecomposition of $\mathbf{X}^\top \mathbf{X}$; fix a sign convention before comparing components. EM for Gaussian mixtures is in [Chapter 11](#).

§17.5 SVM

Prototypical example for this section: Soft-margin linear SVM by sequential minimal optimisation.

soft-margin SVM: SMO with the maximal-violating pair

```

1 def svm_smo(X, y, C=1.0, tol=1e-6, max_iter=100000):
2     """Dual coordinate ascent on pairs (the working-set rule of LIBSVM's first
   ↪ version)."""
3     n = len(y); K = X @ X.T; a = np.zeros(n); g = -np.ones(n)      # g = gradient
   ↪ of 1/2 a^T Q a - sum a
4     Q = (y[:, None] * y[None, :]) * K
5     for _ in range(max_iter):
6         up = ((y > 0) & (a < C)) | ((y < 0) & (a > 0))              # can move y_i
   ↪ a_i up
7         dn = ((y > 0) & (a > 0)) | ((y < 0) & (a < C))
8         i = np.where(up)[0][np.argmax(-y[up] * g[up])]
9         j = np.where(dn)[0][np.argmin(-y[dn] * g[dn])]
10        if -y[i] * g[i] + y[j] * g[j] < tol:                        # KKT satisfied
   ↪ within tol
11            break
12        # move along y_i d_i = -y_j d_j that decreases the objective, then clip to
   ↪ the box
13        quad = max(Q[i, i] + Q[j, j] - 2 * y[i] * y[j] * Q[i, j], 1e-12)
14        step = (-y[i] * g[i] + y[j] * g[j]) / quad
15        lo = max(-a[i] if y[i] > 0 else a[i] - C, a[j] - C if y[j] > 0 else -a[j])
16        hi = min(C - a[i] if y[i] > 0 else a[i], a[j] if y[j] > 0 else C - a[j])
17        step = min(max(step, lo), hi)
18        di, dj = y[i] * step, -y[j] * step
19        a[i] += di; a[j] += dj
20        g += Q[:, i] * di + Q[:, j] * dj
21    w = (a * y) @ X
22    free = (a > 1e-8) & (a < C - 1e-8)
23    b = np.mean(y[free] - X[free] @ w)
24    return w, b, a

```

Each iteration picks the pair of dual variables that most violates the KKT conditions, moves along the only direction that keeps $\sum \alpha_i y_i = 0$, and clips to the box $[0, C]$ (Chapter 8); the gradient is updated in $O(n)$. Stopping when the KKT violation is below 10^{-6} reproduces `SVC(kernel="linear")`'s $(\mathbf{w}, b)$ to within 10^{-6} . If asked for a quicker answer, sub-gradient descent on the primal $\frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum \max(0, 1 - y_i \mathbf{w}^\top \mathbf{x}_i)$ (Pegasos) is ten lines, at the cost of only approximate optimality.

In a coding round: write the core update first, vectorise the inner loop, handle the intercept, numerical stability and ties out loud, and test against a known answer before optimising.

18 Rapid-fire

§18.1 Rapid-fire answers

[STUB: Rapid-fire answers]

19 Problems

§19.1 Problems

[STUB: Problems]

VI

Backmatter

A Hints to problems

- 1A.** Ask what plays the role of y at training time, and where it came from.
- 1B.** Residuals are $(1, 0, -2, 1)$.
- 1C.** Look at both the level and the gap.
- 1D.** Compare $\pi \phi(x - 2)$ with $(1 - \pi) \phi(x)$ and take logs.
- 1E.** Are the rows independent?
- 1F.** Each accuracy is approximately $\mathcal{N}(0.5, 0.25/n)$.
- 2A.** Square, and ignore the sign for strength.
- 2B.** Use the closure rules and the second-derivative test; for (f) look at the Hessian.
- 2C.** Set $\mathbf{A}\mathbf{w} - \mathbf{b} = \mathbf{0}$.
- 2D.** Use the first posterior as the new prior, or multiply likelihoods.
- 2E.** $\text{MAP} = (h + a - 1)/(n + a + b - 2)$.
- 2F.** Three cases: $a < 0$, $0 \leq a \leq 1$, $a > 1$.
- 3A.** $\bar{x} = 3$, $\bar{y} = 3$; compute $\sum (x - \bar{x})(y - \bar{y})$.
- 3B.** $b_1 = r s_y / s_x$.
- 3C.** Rank of $\mathbf{X}$; minimum-norm solutions.
- 3D.** Plug into $1 - (1 - R^2)(n - 1)/(n - p - 1)$.
- 3E.** $\mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}$ and $\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}$.
- 3F.** Write the error $\mathbf{w}_t - \hat{\mathbf{w}}$ and show it is multiplied by $\mathbf{I} - \eta \mathbf{H}_J$ each step.
- 4A.** Think about the closed form and the SVD shrinkage factors.
- 4B.** Soft-threshold: $\text{sign}(z) \max(|z| - \lambda, 0)$.
- 4C.** High training error = bias; large gap = variance.
- 4D.** Take $-\log$ of likelihood times prior and multiply by $2\sigma^2$.
- 4E.** Bias $= (c - 1)\mu$, variance $= c^2\sigma^2/n$.
- 4F.** $\mathbf{0} \in -\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda \partial \|\mathbf{w}\|_1$ at $\mathbf{w} = \mathbf{0}$.
- 5A.** $z = 0.5 \cdot 2 - 1 \cdot 1 + 0.25$.
- 5B.** Work in odds: multiply by $e^{0.4 \cdot 3}$.
- 5C.** $p = \sigma(x)$ for both points; gradient $\frac{1}{n} \sum (p_i - y_i)(1, x_i)$.
- 5D.** Only one of these is false.
- 5E.** $\partial \log p_k / \partial z_j = \mathbb{I}[k = j] - p_j$.
- 5F.** $t = c_{\text{FP}} / (c_{\text{FP}} + c_{\text{FN}})$; re-weighting distorts probabilities.

- 6A.** Multiply prior by both likelihoods for each class and normalise.
- 6B.** $(N_{cj} + 1)/(N_c + V)$.
- 6C.** Think about what each model can represent and count parameters.
- 6D.** Set the two log-densities equal.
- 7A.** Manhattan distances from $(1, 1)$ are 2, 1, 2, 2, 3.
- 7B.** Volume = edge²⁰.
- 7C.** Think about $k = 1$ versus $k = n$.
- 7D.** B is A repeated ten times.
- 8A.** Width $2/\|\mathbf{w}\|$; distance $|\mathbf{w}^\top \mathbf{x} + b|/\|\mathbf{w}\|$.
- 8B.** $\mathbf{x}^\top \mathbf{z} = 4$ and $\|\mathbf{x} - \mathbf{z}\|^2 = 3$.
- 8C.** Complementary slackness.
- 8D.** For (d): leaving out a non-support vector does not change the classifier.
- 8E.** $y\mathbf{w}^\top \mathbf{x} = -0.5 < 1$, so the hinge is active.
- 9A.** Uniform over three classes: $\log_2 3$.
- 9B.** Compute both weighted child impurities.
- 9C.** Try $x \leq 1.5$, $x \leq 2.5$, $x \leq 3.5$.
- 9D.** Think about leaves and monotone transforms.
- 10A.** (b) counts arrangements of 4 distinct rows among 4^4 equally likely samples.
- 10B.** After normalising, the mistakes carry total weight $\frac{1}{2}$.
- 10C.** $w = -G/(H + \lambda)$.
- 10D.** $g = 0.5 - y$, $h = 0.25$.
- 10E.** Three of the six are false.
- 10F.** $\rho\sigma^2 + (1 - \rho)\sigma^2/B$.
- 11A.** 25 is closer to 9.
- 11B.** Total variance is the trace, the sum of the eigenvalues.
- 11C.** Eigenvectors of $\begin{pmatrix} a & b \\ b & a \end{pmatrix}$ are $(1, \pm 1)/\sqrt{2}$.
- 11D.** Lift = $\mathbb{P}(\text{both})/(\mathbb{P}(\text{chips})\mathbb{P}(\text{cola}))$.
- 11E.** Ratio of $\pi_k \phi(x - \mu_k)$; the normalising constants cancel.
- 12A.** Graphs suggest propagation; two views suggest co-training.
- 12B.** It takes the weighted average of its neighbours' label distributions.
- 12C.** Loss = $-\log \frac{e^{0.8/\tau}}{e^{0.8/\tau} + 3e^{0.5/\tau}}$.
- 13A.** Precision uses the predicted-positive column; recall the actual-positive row.
- 13B.** Count the six positive-negative pairs ordered correctly.

13C. $\text{Precision} = \frac{\text{TPR} \cdot \pi}{\text{TPR} \cdot \pi + \text{FPR} \cdot (1 - \pi)}.$

13D. $\ln 100 \approx 4.605.$

13E. Ask, for each feature, when it becomes known; then ask whether rows are independent.

14A. $\mathbb{P}(X = k) = e^{-\lambda} \lambda^k / k!.$

14B. Paired, two proportions, contingency table, more than two means.

14C. $t = (210 - 200) / (40/4)$ with 15 d.f.; $t_{0.975, 15} \approx 2.13.$

14D. $n \propto 1/\delta^2.$

15A. $\ln(N/\text{df}).$

15B. $(n\bar{y} + m\bar{y}_{\text{global}}) / (n + m).$

15C. Only one statement about trees; one about leakage.

16A. The Hessian is $\text{diag}(4, 16).$

16B. $x \leftarrow x - f'(x)/f''(x)$ with $f' = 4x^3 - 3$, $f'' = 12x^2.$

B Solutions to problems

- 1A.** (a) supervised regression. (b) unsupervised clustering/topic modelling. (c) self-supervised: the target frame is part of the input stream. (d) reinforcement learning: only delayed reward. (e) supervised; strictly ordinal regression, but usually treated as five-class classification — in an interview say that the order matters and that a model ignoring it penalises predicting 4 for a 5 as much as predicting 1.
- 1B.** $\text{MSE} = (1 + 0 + 4 + 1)/4 = 1.5$; $\text{MAE} = (1 + 0 + 2 + 1)/4 = 1$. With target 17 the last residual is -9 : MSE becomes $(1 + 0 + 4 + 81)/4 = 21.5$, MAE becomes $(1 + 0 + 2 + 9)/4 = 3$. Squared loss is far more sensitive to outliers, which is why MAE or Huber loss is used when targets have heavy tails.
- 1C.** A overfits (gap 0.15). C underfits (both low, gap 0.01). B has the best validation accuracy and a small gap; ship B (after confirming on the untouched test set). Note that A has the best training accuracy, and that the overfitting A still beats the underfitting C on validation (0.84 versus 0.70): overfitting is not automatically worse than underfitting; compare validation scores.
- 1D.** Predict 1 iff $\pi e^{-(x-2)^2/2} > (1 - \pi)e^{-x^2/2}$, i.e. $\ln \pi - (x-2)^2/2 > \ln(1 - \pi) - x^2/2$, i.e. $2x - 2 > \ln \frac{1-\pi}{\pi}$, which is the claim. As $\pi \rightarrow 0$ the threshold goes to $+\infty$: when positives are rare you need overwhelming evidence to predict one, which is why accuracy-optimal classifiers on imbalanced data predict “negative” almost always (Chapter 13).
- 1E.** Images of the same patient land in both train and test; the network can recognise the patient (anatomy, implants, scanner artefacts) and copy that patient’s label. The test score measures memorisation, not diagnosis. Split by patient (`GroupKFold` with patient ID as the group), and ideally also by hospital/scanner if deployment will see new sites.
- 1F.** Standard deviation $= \sqrt{0.25/100} = 0.05$, so the best is about $0.5 + 2.25 \times 0.05 = 0.612$, matching the simulation 0.612 in the chapter. The optimism shrinks like $1/\sqrt{n}$ and grows like $\sqrt{2 \ln m}$: bigger validation sets help, comparing more models hurts.
- 2A.** 0.09 (9%) and 0.36 (36%). The second is stronger; the first is weaker and negative. Strength is $|r|$, direction is the sign, and r^2 is the explained fraction of variance in simple linear regression.
- 2B.** (a) convex: max of convex functions. (b) not convex: it equals x^2 on $[-1, 1]$ and 1 outside. The chord from $x = 0$ to $x = 2$ has midpoint value $(0 + 1)/2 = 0.5$, while $f(1) = 1 > 0.5$: the graph rises above the chord. (c) convex: logistic loss composed with an affine map, plus a norm. (d) not convex ($f'' = 6x < 0$ for $x < 0$). (e) convex ($f'' \geq 0$). (f) not convex: Hessian $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ has eigenvalues ± 1 , a saddle.
- 2C.** $\nabla f = \mathbf{A}\mathbf{w} - \mathbf{b}$, so $\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b}$ and $f(\mathbf{w}^*) = c - \frac{1}{2}\mathbf{b}^\top \mathbf{A}^{-1}\mathbf{b}$. If $\mathbf{A}$ is singular the minimiser is not unique (any null-space direction can be added) and may not exist if $\mathbf{b}$ has a component outside the range of $\mathbf{A}$ (f is then unbounded below). Ridge’s $+\lambda \mathbf{I}$ removes the problem.

- 2D.** Likelihood ratio of one positive is $0.95/0.10 = 9.5$. Prior odds $1/99$; after two positives the odds are $9.5^2/99 = 0.9116$, so $\mathbb{P}(D \mid ++)$ = 0.477. Updating in odds form is the fastest way to do this in an interview.
- 2E.** $\text{MLE} = 2/3 = 0.667$; $\text{MAP} = (2+1)/(3+20) = 3/23 = 0.130$. The MLE would put a 3-impression ad at the top of the ranking; the prior encodes that typical CTRs are low and shrinks small-sample estimates towards it. Use the shrunk estimate (or a lower confidence bound) for ranking.
- 2F.** The answer is the clip of a to $[0, 1]$. For $0 \leq a \leq 1$ both constraints are slack and both multipliers are 0. For $a > 1$ the constraint $x \leq 1$ binds, $x^* = 1$, and stationarity $2(1-a) + \mu_1 = 0$ gives $\mu_1 = 2(a-1) > 0$. For $a < 0$, $x^* = 0$ and the multiplier on $-x \leq 0$ is $\mu_0 = -2a > 0$. This box-constrained step is exactly the clipping in the SMO update for SVMs.
- 3A.** Centred products: $(-2)(-2) + (-1)(0) + 0 + (1)(2) + (2)(1) = 8$; $\sum(x - \bar{x})^2 = 10$. So $b_1 = 0.8$, $b_0 = 3 - 2.4 = 0.6$. Fitted: 1.4, 2.2, 3.0, 3.8, 4.6; residuals $-0.4, 0.8, -1.0, 1.2, -0.6$, $\text{RSS} = 0.16 + 0.64 + 1 + 1.44 + 0.36 = 3.6$, $\text{TSS} = 4 + 0 + 1 + 4 + 1 = 10$, $R^2 = 0.64$. At $x = 7$: 6.2.
- 3B.** $b_1 = 0.6 \cdot 10/2 = 3$, $b_0 = 60 - 3 \cdot 5 = 45$; prediction $45 + 24 = 69$. The reverse regression has slope $0.6 \cdot 2/10 = 0.12$, not $1/3$; the product of the slopes is $r^2 = 0.36$.
- 3C.** (a) $\mathbf{X}^\top \mathbf{X}$ is singular; no unique solution (any $w_1 + w_2 = c$ works). (b) It uses an SVD least-squares solver and returns the minimum-norm solution, splitting the weight equally. (c) Predictions are unchanged and well defined; only the coefficients are not identified. (d) Ridge has a unique solution and also splits the weight equally, $w_1 = w_2$.
- 3D.** A: $1 - 0.30 \cdot 59/56 = 0.6839$. B: $1 - 0.26 \cdot 59/49 = 0.6869$. B is marginally higher; the extra features carry a little signal. In practice, compare on validation data.
- 3E.** The first row of $\mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}$ is the ones column: residuals sum to zero. $\hat{\mathbf{y}}^\top(\mathbf{y} - \hat{\mathbf{y}}) = \hat{\mathbf{w}}^\top \mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{y}}) = 0$. Write $y_i - \bar{y} = (y_i - \hat{y}_i) + (\hat{y}_i - \bar{y})$, square and sum; the cross term is $\sum e_i \hat{y}_i - \bar{y} \sum e_i = 0$. Without an intercept this decomposition (and $R^2 \geq 0$) can fail.
- 3F.** With $\mathbf{H}_J = \frac{2}{n} \mathbf{X}^\top \mathbf{X}$, $\nabla J(\mathbf{w}) = \mathbf{H}_J(\mathbf{w} - \hat{\mathbf{w}})$, so $\mathbf{w}_{t+1} - \hat{\mathbf{w}} = (\mathbf{I} - \eta \mathbf{H}_J)(\mathbf{w}_t - \hat{\mathbf{w}})$. In the eigenbasis each component is multiplied by $1 - \eta \lambda_i$ per step; all shrink iff $|1 - \eta \lambda_i| < 1$ for every i , i.e. $0 < \eta < 2/\lambda_i$ for all i , i.e. $\eta < 2/\lambda_{\max}$. The slowest direction contracts by $1 - \eta \lambda_{\min}$, which is why the condition number $\lambda_{\max}/\lambda_{\min}$ governs speed.
- 4A.** (a), (b), (c) are true. (b) assumes the intercept is unpenalised. (d) false: shrinkage factors $s_j^2/(s_j^2 + \lambda)$ are never zero; that is lasso's property. (e) false: the penalty depends on the scale of each coefficient, so standardise first.
- 4B.** Lasso: 1.4 and 0.0. Ridge: 1.2 and 0.6. Lasso subtracts, ridge divides; only lasso reaches zero.
- 4C.** First: high bias (training error far above the noise floor, tiny gap). Decrease λ or add features; more data will not help much. Second: high variance (training error below the noise floor, huge gap). Increase λ , simplify, or collect more data — here more data helps.

- 4D.** $-\log p(\mathbf{w} \mid \mathcal{D}) = \frac{1}{2\sigma^2} \sum (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \frac{1}{2\tau^2} \|\mathbf{w}\|^2 + c$; multiplying by $2\sigma^2$ gives $\text{RSS} + (\sigma^2/\tau^2) \|\mathbf{w}\|^2$. A Laplace prior $p(w_j) \propto e^{-|w_j|/b}$ gives $\frac{1}{2\sigma^2} \text{RSS} + \frac{1}{b} \|\mathbf{w}\|_1$, i.e. lasso with $\lambda = 2\sigma^2/b$ in the $\text{RSS} + \lambda \|\mathbf{w}\|_1$ form.
- 4E.** $\text{MSE}(c) = (1-c)^2 \mu^2 + c^2 \sigma^2/n$, minimised at $c^* = \mu^2/(\mu^2 + \sigma^2/n) < 1$. Shrinking trades a little bias for a larger cut in variance whenever σ^2/n is not negligible relative to μ^2 . This is ridge in one line; c^* has exactly the form $s^2/(s^2 + \lambda)$.
- 4F.** At $\mathbf{w} = \mathbf{0}$ the subdifferential of $\|\mathbf{w}\|_1$ is $[-1, 1]^d$, so optimality requires a vector $\mathbf{g} \in [-1, 1]^d$ with $\mathbf{X}^\top \mathbf{y} = \lambda \mathbf{g}$, i.e. $|\mathbf{x}_{(j)}^\top \mathbf{y}| \leq \lambda$ for every column j . Since the problem is convex this is also sufficient. Practically, $\lambda_{\max} = \|\mathbf{X}^\top \mathbf{y}\|_\infty$ is where every lasso path starts (scikit-learn's grid begins there, scaled by $1/n$), and the first feature to enter is the one most correlated with $\mathbf{y}$.
- 5A.** $z = 0.25$, $p = \sigma(0.25) = 0.5622$, class 1, loss $-\ln p = 0.5759$.
- 5B.** Odds $0.2/0.8 = 0.25$; times $e^{1.2} = 3.3201$ gives 0.8300 ; probability 0.4536 .
- 5C.** $p_1 = \sigma(1) = 0.7311$, $p_2 = \sigma(-1) = 0.2689$. Errors $p - y = (0.7311, -0.7311)$. Gradient $= (0.0000, 0.7311)$; new weights $(0.0000, 0.6345)$. The slope decreases: the model was getting both points wrong.
- 5D.** (a), (b), (d), (e) are true; (c) is false — there is no distributional assumption on $\mathbf{x}$. (e) holds because only $\mathbf{w}^\top \mathbf{x}$ matters; with regularisation it fails.
- 5E.** $\log p_k = z_k - \log \sum_i e^{z_i}$, so $\partial \log p_k / \partial z_j = \mathbb{I}[k = j] - p_j$. Then $-\sum_k y_k (\mathbb{I}[k = j] - p_j) = -y_j + p_j \sum_k y_k = p_j - y_j$ since $\sum_k y_k = 1$.
- 5F.** $t = 100/5100 = 0.0196$. No: balanced weights inflate predicted fraud probabilities (roughly, the model behaves as if the prior were 50%), so the scores are not calibrated probabilities. Either train unweighted and use t , or recalibrate the weighted model on validation data ([Chapter 13](#)) before applying a cost threshold.
- 6A.** Spam: $0.3 \cdot 0.05 \cdot 0.002 = 3.0 \times 10^{-5}$. Ham: $0.7 \cdot 0.005 \cdot 0.01 = 3.5 \times 10^{-5}$. Posterior 0.4615 .
- 6B.** Positive: $1/1500 = 0.00067$. Negative: $13/1800 = 0.00722$. Ratio negative/positive $= 10.83$: seeing “refund” multiplies the odds of negative by about 11.
- 6C.** (a) False: with conditionally independent features, each $p(x_j \mid c)$ is the same for both classes in XOR (each feature is 50/50 within each class), so the model cannot distinguish them. (b) True (double counting). (c) False for multinomial/Bernoulli; Gaussian NB estimates a scale per feature, so scaling has no effect either. (d) True. (e) False: QDA has more.
- 6D.** $-\frac{x^2}{2} - \frac{1}{2} \log(2\pi) = -\frac{x^2}{8} - \frac{1}{2} \log(8\pi)$ gives $\frac{3x^2}{8} = \log 2$, $|x| = \sqrt{8 \log 2/3} = 1.360$. Predict A inside $(-1.36, 1.36)$ and B outside: unequal variances give a quadratic boundary (two thresholds), the one-dimensional QDA picture.
- 7A.** Nearest is $(1, 0)$ (distance 1): $k = 1$ gives red. For $k = 3$ the next two come from a three-way tie at distance 2: $(0, 0)$ red, $(0, 2)$ blue, $(2, 2)$ blue. Depending on the tie-break the vote is red-red-blue (red) or red-blue-blue (blue). The answer must state the tie rule. Including every tied point gives two red and two blue, still a tie; inverse-distance weighting resolves it: red $1/1 + 1/2 = 1.5$ against blue $1/2 + 1/2 = 1$, so red.

- 7B.** $0.01^{1/20} = 0.794$ and $0.5^{1/20} = 0.966$. A neighbourhood holding 1% of the data spans almost 80% of every axis: it is not local.
- 7C.** (a) true, (b) false (it decreases variance and increases bias), (c) true, (d) true (unless you build an index, which is $O(n \log n)$), (e) false.
- 7D.** Euclidean: $d(A, B) = 20.125$, $d(A, C) = 1.414$, so C is closer. Cosine distance: $d(A, B) = 0$ (same direction), $d(A, C) = 0.225$, so B is closer. For documents, B is the same content at ten times the length; cosine (or Euclidean on length-normalised vectors) is the sensible choice.
- 8A.** $\|\mathbf{w}\| = 5$, width 0.40. $\mathbf{w}^\top \mathbf{x} + b = 5$, positive side, distance 1.00; since $f(\mathbf{x}) = 5 \geq 1$ it is outside the street and has $\alpha = 0$.
- 8B.** (a) 4. (b) $5^3 = 125$. (c) $e^{-0.3} = 0.7408$.
- 8C.** Deleting $\alpha_i = 0$ points leaves the solution unchanged: they do not appear in $\mathbf{w} = \sum \alpha_i y_i \mathbf{x}_i$ and still satisfy their constraints. Deleting an $\alpha_i = C$ point (a margin violator) generally changes $\mathbf{w}$ and b ; the remaining problem is different.
- 8D.** (a) true. (b) false (it narrows it). (c) essentially true: each kernel bump becomes a spike, so $f(\mathbf{x})$ is dominated by the closest training points. (d) true: only support vectors can be misclassified when left out, so LOO error $\leq \#SV/n$. (e) true for RBF and polynomial kernels and for the penalty $\|\mathbf{w}\|^2$ in general.
- 8E.** Subgradient $\lambda \mathbf{w} - y \mathbf{x} = (0.05, -0.05) - (1, 2) = (-0.95, -2.05)$. Update $\mathbf{w} \leftarrow \mathbf{w} - 0.1 \mathbf{g} = (0.595, -0.295)$. If the margin had been ≥ 1 , only the shrinkage term $\lambda \mathbf{w}$ would act.
- 9A.** Entropy $\log_2 3 = 1.5850$, Gini $1 - 3 \cdot \frac{1}{9} = 0.6667$. Children: (10, 10, 0) has entropy 1, (0, 0, 10) has 0; weighted $\frac{20}{30} \cdot 1 = 0.667$; IG = 0.9183.
- 9B.** A: each child has entropy 0.650, IG = 0.350; Gini decrease 0.2222. B: children entropies 0 and 0.918, weighted $\frac{9}{12} \cdot 0.918$, IG = 0.311; Gini decrease 0.1667. Both prefer A, although B creates a pure node.
- 9C.** SSE for the three thresholds: 34.67, 4.00, 34.67. Best is $x \leq 2.5$ with leaves 3 and 11.
- 9D.** (a) true. (b) false. (c) true. (d) false: leaves average training targets. (e) true: $2p(1-p) \leq \frac{1}{2}$.
- 10A.** (a) $(3/4)^4 = 0.3164$. (b) $4!/4^4 = 0.0938$. (c) by linearity, $4(1 - (3/4)^4) = 2.7344$ (simulation agrees).
- 10B.** $\varepsilon_1 = 0.2$, $\alpha_1 = \frac{1}{2} \ln 4 = 0.6931$. The misclassified point gets weight $\frac{1}{2}$, each of the four others $\frac{1}{8}$.
- 10C.** $G = -4$, $H = 4$: $w = 4/6 = 0.6667$; with $\lambda = 0$, $w = 1$ (the mean residual). λ shrinks leaf values towards zero, more strongly for leaves with few examples (small H).
- 10D.** Left: $G_L = -1$, $H_L = 1$; right: $G_R = 1$, $H_R = 0.5$; parent $G = 0$. Gain $= \frac{1}{2}[\frac{1}{2} + \frac{1}{1.5} - 0] = 0.5833$.
- 10E.** True: (b), (d), (f). False: (a) — forests reduce variance; (c) — more trees only stabilise; (e) — OOB uses each tree's left-out rows.

- 10F.** $0.25 \cdot 4 + 0.75 \cdot 4/10 = 1.30$; limit $\rho\sigma^2 = 1.0$. A limit of 0.5 needs $\rho = 0.5/4 = 0.125$: once B is large, only decorrelating the trees (fewer features per split, more randomness) lowers the variance further.
- 11A.** Clusters $\{1, 2, 3\}$ and $\{8, 9, 10, 25\}$; new centroids 2 and 13. Next assignment is unchanged (8 is closer to 13 than to 2), so it has converged. The outlier drags its centroid from 9 to 13; k -medians or k -medoids would resist.
- 11B.** Total 8.0. Cumulative fractions: 0.5, 0.8125, 0.9375, $\dots$, so 3 components. PC1 explains 0.50.
- 11C.** Eigenvalues 4 (direction $(1, 1)/\sqrt{2}$) and 2 (direction $(1, -1)/\sqrt{2}$). PC1 explains 2/3. Projection: $(2, 0) \cdot (1, 1)/\sqrt{2} = \sqrt{2} = 1.4142$.
- 11D.** Support 0.12; confidence $120/200 = 0.6$; lift $0.6/0.3 = 2.0$. Customers who buy chips are twice as likely as average to buy cola: a useful rule for placement or bundling.
- 11E.** $0.3e^{-1.125}$ versus $0.7e^{-3.125}$; $r_1 = \frac{0.3e^{-1.125}}{0.3e^{-1.125} + 0.7e^{-3.125}} = 0.7600$. Although x is closer to 0 than to 4 by a wide margin, the larger prior of component 2 is not enough to win.
- 12A.** (a) Smoothness and cluster assumptions: consistency regularisation (FixMatch-style), or self-supervised pre-training followed by a linear probe. (b) Homophily (linked papers share fields, a graph version of smoothness): label propagation or a graph neural network. (c) Two roughly independent views: co-training.
- 12B.** $(2 + 1)/(2 + 1 + 3) = 0.5$: a tie, despite having more A-neighbours, because the single B-edge is strong.
- 12C.** $\tau = 1$: 1.1701. $\tau = 0.1$: 0.1392. A small temperature turns a 0.3 similarity gap into a large logit gap, so the loss approaches 0 when the positive wins clearly and grows quickly when a negative comes close: low τ concentrates the gradient on hard negatives.
- 13A.** Accuracy 0.930, precision $45/75 = 0.600$, recall $45/50 = 0.900$, specificity $420/450 = 0.933$, F1 = 0.720.
- 13B.** Pairs: $(0.9, 0.7) \checkmark$, $(0.9, 0.3) \checkmark$, $(0.6, 0.7) \times$, $(0.6, 0.3) \checkmark$, $(0.4, 0.7) \times$, $(0.4, 0.3) \checkmark$: $4/6 = 0.667$.
- 13C.** $\pi = 0.1$: $0.09/(0.09 + 0.045) = 0.667$. $\pi = 0.005$: $0.0045/(0.0045 + 0.04975) = 0.083$. Same ROC point, precision collapses: base rates matter.
- 13D.** AIC: A = 246, B = 240: B. BIC: A = 253.82, B = 255.63: A. Six units of log-likelihood buy three extra parameters under AIC's penalty of 2 each, but not under BIC's $\ln n \approx 4.6$ each.
- 13E.** "Account closure reason" exists only after churn: target leakage. Last quarter's model score may encode the label if that model was trained on overlapping outcomes. Support calls in the last 30 days must be computed strictly before the prediction date. Random splits over customer-months put the same customer in train and test (group leakage) and mix future months into training (temporal leakage): split by time, and evaluate on customers or months not seen in training.
- 14A.** $e^{-3} = 0.0498$; $1 - e^{-3}(1 + 3) = 0.8009$.

- 14B.** (a) paired t (or Wilcoxon signed-rank); (b) two-proportion z (or χ^2 on the 2×2 table); (c) χ^2 test of independence; (d) one-way ANOVA (or Kruskal–Wallis).
- 14C.** $t = 1.00$, $p = 0.3332$: do not reject. A 95% interval is $210 \pm 2.131 \cdot 10 = [188.7, 231.3]$, which contains 200.
- 14D.** About four times as many: the formula gives 14751 per arm. Standard error falls like $1/\sqrt{n}$, so halving the detectable effect requires quadrupling n .
- 15A.** $\ln 100 = 4.6052$, $\ln(1000/990) = 0.0101$; tf-idf $3 \times 4.605 = 13.816$.
- 15B.** A: $(1 + 20 \cdot 0.05)/22 = 0.0909$ (raw mean 0.5 shrunk hard). B: $(60 + 1)/420 = 0.1452$ (barely shrunk from 0.15).
- 15C.** (a) false; (b) true; (c) true; (d) true (better conditioning); (e) false — fit on training data only and apply the same transform to test data.
- 16A.** $L = 16$, $\mu = 4$: converge iff $\eta < 2/16 = 0.125$; $\kappa = 4$, and the best constant step $2/(4 + 16) = 0.1$ contracts errors by $(4 - 1)/(4 + 1) = 0.6$ per step.
- 16B.** $x_1 = 1 - 1/12 = 0.9167$; $x_2 = 0.9086$; the minimiser is $(3/4)^{1/3} = 0.9086$ — two steps already give four correct digits.

Bibliography

- [AgSr94] RAKESH AGRAWAL and RAMAKRISHNAN SRIKANT. “Fast Algorithms for Mining Association Rules”. In: *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*. 1994 (cited p. 101)
- [Ak74] HIROTUGU AKAIKE. A New Look at the Statistical Model Identification. In: *IEEE Transactions on Automatic Control*, **19**:6 (1974), pp. 716–723 (cited p. 113)
- [ArVa07] DAVID ARTHUR and SERGEI VASSILVITSKII. “k-means++: The Advantages of Careful Seeding”. In: *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2007 (cited p. 96)
- [Be+19] MIKHAIL BELKIN, DANIEL HSU, SIYUAN MA, and SOUMIK MANDAL. Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-Off. In: *Proceedings of the National Academy of Sciences*, **116**:32 (2019), pp. 15849–15854 (cited p. 47)
- [BIMi98] AVRIM BLUM and TOM MITCHELL. “Combining Labeled and Unlabeled Data with Co-Training”. In: *Proceedings of the 11th Annual Conference on Computational Learning Theory (COLT)*. 1998 (cited p. 103)
- [DeLaRu77] ARTHUR P. DEMPSTER, NAN M. LAIRD, and DONALD B. RUBIN. Maximum Likelihood from Incomplete Data via the EM Algorithm. In: *Journal of the Royal Statistical Society, Series B*, **39**:1 (1977), pp. 1–38 (cited p. 98)
- [Es+96] MARTIN ESTER, HANS-PETER KRIEGEL, JÖRG SANDER, and XIAOWEI XU. “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise”. In: *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*. 1996 (cited p. 98)
- [Ke+17] GUOLIN KE et al. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”. In: *Advances in Neural Information Processing Systems 30*. 2017 (cited p. 90)
- [KoBeVo09] YEHUDA KOREN, ROBERT BELL, and CHRIS VOLINSKY. Matrix Factorization Techniques for Recommender Systems. In: *Computer*, **42**:8 (2009), pp. 30–37 (cited p. 127)
- [LiTiZh08] FEI TONY LIU, KAI MING TING, and ZHI-HUA ZHOU. “Isolation Forest”. In: *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*. 2008 (cited p. 101)
- [MaHi08] LAURENS VAN DER MAATEN and GEOFFREY HINTON. Visualizing Data using t-SNE. In: *Journal of Machine Learning Research*, **9**: (2008), pp. 2579–2605 (cited p. 100)
- [Pr+18] LIUDMILA PROKHORENKOVA, GLEB GUSEV, ALEKSANDR VOROBEOV, ANNA VERONIKA DOROGUSH, and ANDREY GULIN. “CatBoost: Unbiased Boosting with Categorical Features”. In: *Advances in Neural Information Processing Systems 31*. 2018 (cited p. 90)
- [Sc78] GIDEON SCHWARZ. Estimating the Dimension of a Model. In: *Annals of Statistics*, **6**:2 (1978), pp. 461–464 (cited p. 113)

- [St+07] CAROLIN STROBL, ANNE-LAURE BOULESTEIX, ACHIM ZEILEIS, and TORSTEN HOTHORN. Bias in Random Forest Variable Importance Measures: Illustrations, Sources and a Solution. In: *BMC Bioinformatics*, **8**: (2007), p. 25 (cited p. 86)